基于柱坐标热质耦合与动边界的中药材热风烘干过程建模

摘　要

热风烘干过程中的温度与水分演化直接决定中药材的干燥效率与品质，整个过程由表面对流换热换质与内部扩散共同控制，是典型的非稳态热质耦合问题。本文以一段圆柱形药材为对象，先将三维传热–传质问题在对称性下**精确退化**为二维 (r,z) 轴对称形式求解问题一，再经逐位误差分析与特征尺度论证证明轴向影响可忽略，问题二至四由此统一采用**一维径向有限体积框架**求解。

针对问题一，数值上用 (r,z) 轴对称**有限体积模型** $(N_r,N_z)=(100,50)$ 求解（温度方程 Crank–Nicolson 格式、水分方程向后欧拉配 **Picard 迭代**处理非线性 $D(C)$）。1800s 时表面温度升至 36.79 ℃、含水率降至 1.51 kg/kg，中心仍接近初值 2.55 kg/kg。与一维径向解逐位对比，中截面偏差全程为零，轴向影响仅局限于端面 1–4 cm 薄层（端面轴心偏差 $+3.70$ ℃、-0.99 kg/kg），中截面口径下网格加密最大相对差仅 10^{-5} 量级。**降维依据由此得到定量确立**.

对问题二，改用附录 3 的变物性经验公式刻画预热与恒温干燥全过程，扩散系数含 Arrhenius 温度因子，温度场与含水率场**双向强耦合**、交替迭代求解。求得 3 h 时中心升温至 49.85 °**C**、全场趋于热平衡，表面与中心含水率分别降至 1.01 与 1.77 kg/kg，干燥前沿明显向内推进。

对于问题三，以**全场最大含水率低于** 0.15 **kg/kg** 为干透判据，沿用变物性耦合引擎积分至达标，超出附件 1 记录的时段按恒温干燥阶段设定值（50 ℃、0.05 kg/kg）外推，并论证时长判定与轴向维度无关（一维结果为保守上界）。求得烘干时长 57.47 h：前 12 h 中心含水率由 2.55 骤降至 0.46 kg/kg，此后进入漫长尾段。

针对第四问，药材失水收缩使求解域成为动边界，引入随体坐标 $\xi=r/R(t)$ 映射到固定计算域——均匀随体收缩下链式法则项与欧拉对流项精确对消，方程无需对流项，物性改用附录 4 经验公式；内部扩散路径随半径缩短，干燥加速。半径由 2.0 cm 收缩至 1.2 cm，烘干时长由固定尺寸的 129.8 h 缩短至 51.09 h（-61%），相对问题三的 57.47 h 净减约 6.38 h（约 11%）——收缩加速与附录 3/4 物性差异两个相反效应叠加。

灵敏度分析表明，传质系数 h_m 主导水分演化、平衡含水率影响可忽略，长时程环境外推（约 0.3 h）与传质系数取值是烘干时长的主要不确定来源。四问答案经空间/时间双收敛分析，问题四动边界方程另经四套独立离散实现交叉印证，数值误差对四位小数输出无影响。本文工作的特色在于三点：以逐位误差分析把降维从直觉变成证据、以模型空间穷举敲定动边界方程的唯一自洽形式、以界面系数的 Gauss 精确积分处理指数型扩散系数。

关键词：热风烘干　轴对称退化　有限体积法　动边界 Landau 变换　双向热质耦合

一、 问题重述

1.1 问题背景

热风烘干是中药材干燥加工中最常用的方式之一[1, 2, 15]：热空气一面加热药材、一面带走由内部扩散到表面的水分，过程通常包含预热平衡与恒温干燥两个阶段。工艺参数直接决定干燥效率、能耗与品质——温度过低耗时过长、易霉变，过高又破坏有效成分；传统试错法成本高、周期长。用数理建模与数值仿真刻画药材内部的温度与水分演化，能在不做大量实验的情况下预测干燥进程、估计所需时间，为工艺设计提供依据。

本文研究一段近似圆柱形的中药材（长 25 cm、半径 2 cm）在时变烘房环境下的干燥过程。烘干开始时药材各处温度为 28 °C、干基含水率为 2.55 kg/kg；烘房空气的温度与水分浓度随时间变化，其数据由附件 1 给出。题目围绕这一对象按递进关系提出四个问题：预热阶段的场演化、全过程的变物性演化、达到干燥标准所需时间、尺寸收缩对干燥时间的影响。

1.2 问题提出

四个问题以同一物理对象为基础、逐层增加建模的真实性，共享几何简化与控制方程建立这一前段，而在物性设定、终止判据与几何域处理上依次分叉。各问具体要求如下。

问题一：在预热平衡阶段（前 1800 s），采用附录 2 给出的常物性参数，建立药材内部温度 $T(r,t)$ 与含水率 $C(r,t)$ 的变化规律模型，给出若干指定时刻、指定径向位置处的温度与含水率，并输出该时段内的完整场分布。

问题二：考虑整个烘干过程包含预热平衡与恒温干燥两个阶段、且二者参数不同，物性统一改用附录 3 的经验公式（随含水率与温度变化），建立全过程的温度场与含水率场演化模型，给出 3 h 内每半小时、指定径向位置处的结果与完整场。

问题三：按干燥要求，药材各处含水率应低于 0.15 kg/kg。在问题二模型基础上，确定药材干透所需的烘干时间，并给出干燥过程中含水率随时间与位置的下降规律。

问题四：实际烘干中药材因失水发生尺寸收缩。依据附件 2 给出的各时刻半径、并改用附录 4 的物性经验公式，重新确定考虑收缩后的烘干时长，给出相应的含水率场（含药材表面处）。

二、 问题分析

四个问题本质上都是求解圆柱形药材在时变环境下的内部温度与水分演化，属于非稳态传热-传质耦合正演问题：内部水分迁移以扩散方程刻画、表面与环境换热换质以对流边界刻画[4]。与直接按长径比降维的做法不同，本文的路线是“先精确、后降维”：

问题一先精确求解二维 (r, z) 轴对称模型，再以逐位误差对比与特征尺度论证给出一维径向简化的定量依据，问题二至四据此贯穿求解。整体技术路线如图 1 所示。下面按题号分别分析各问的关键关系与求解思路。

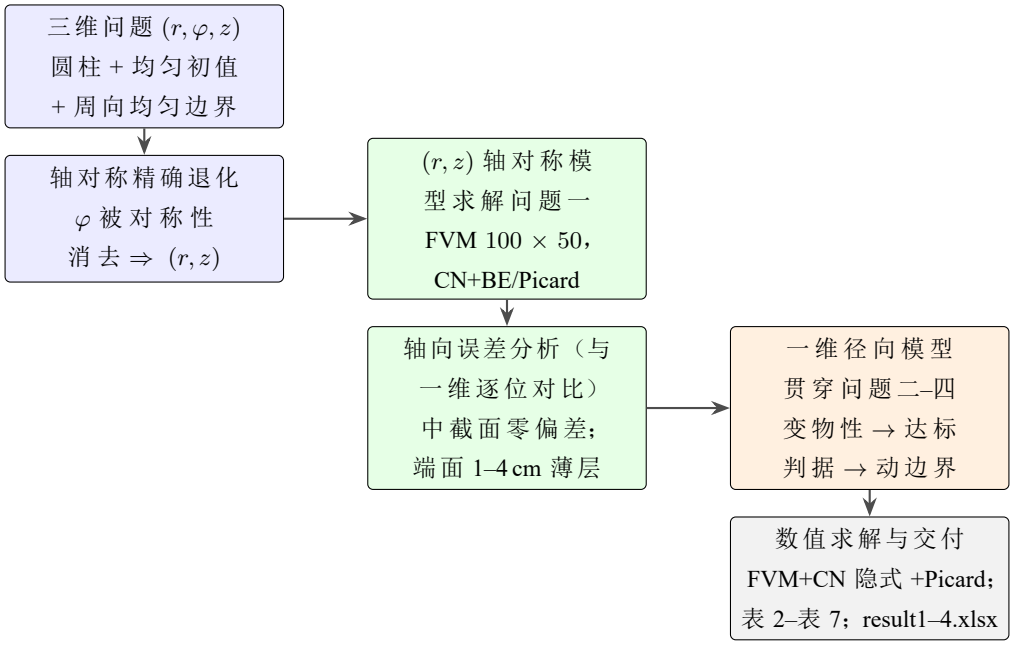


图 1　整体建模与求解技术路线

2.1 问题一的分析

问题一考察预热平衡阶段前 1800 s 的场演化。其难点在于把传热传质规律抽象为圆柱几何下的偏微分方程组，并正确处理时变对流边界与中心 $r = 0$ 处的奇点。由旋转对称、均匀初值与周向均匀边界，三维问题精确退化为二维 (r, z) 轴对称问题；本问直接求解该模型，端面效应一并解出，中截面 $z = 0$ 上的径向剖面即题目所要求“到药材中心的距离”处的输出。在此基础上把一维径向模型作为候选简化：以逐位误差对比（第 5.6 节）与特征扩散尺度（第 5.7 节）论证其合法性，为问题二至四的降维求解提供依据。附录 2 物性均为常数、且 D 只随含水率变化，温度方程独立求解、水分方程仅通过 $D(C)$ 呈非线性——二者构成单向弱耦合。本问确立的离散格式、单位处理与边界方式，是后续三问共同的基础。

2.2 问题二的分析

问题二把物性由常数升级为随含水率 C 与温度 T 变化的变系数：附录 3 的扩散系数含温度因子 $e^{-3850/T}$，密度、比热、热导率也随含水率改变，温度场与水分场因此形成双向强耦合，需在每个时间步内交替更新两个场直至收敛（温度必须以开尔文代入指数，否则严重失真）。这一求解引擎被问题三、问题四直接继承。

2.3 问题三的分析

问题三在问题二模型上施加全场达标阈值以确定烘干时长——时长由物性与判据唯一决定，不存在可调的决策量；水分靠扩散最后自中心排出，全场最湿点位于中心。烘干可能持续数十小时、远超附件 1 覆盖的 4 h 范围，需对超出部分的环境条件作出假设，并兼顾长时程积分的数值稳定与计算成本。

2.4 问题四的分析

问题四在问题三基础上引入尺寸收缩这一真实机制：药材失水后半径由附件 2 给出，随时间单调减小（约收缩 40% 后趋于稳定）。这使求解域 $r \in [0, R(t)]$ 随时间变化，问题转为动边界传质问题。处理动边界通常有两条路 [5–7, 14]：要么每步重划移动网格，要么把物理域变换到固定域。移动网格每步都要重算几何量与场量插值，网格不断收缩还得设计重划策略；坐标归一化（随体坐标 $\xi = r/R(t)$）则保持网格不动。均匀随体收缩下 ξ 是物质坐标：材料点始终位于同一 ξ，而干基含水率（水分质量/干物质质量）只由水分扩散通量改变、不随收缩的几何度量变化，链式法则给出的变换项与欧拉对流项精确对消，因此变换后的方程不含对流项（第 8.3 节对这一结论作了模型空间穷举式核对）。权衡之后我们选后者。物性此时改用附录 4 的经验公式。收缩缩短了内部扩散路径，预期会加快干燥，因此本问的烘干时长应短于问题三固定尺寸情形。

三、 模型假设

针对本题的物理背景与数据条件，作如下简化假设。这些假设只涉及题面未完全规定、而建模又必须补足的现实简化，参数取值本身放在相应模型章节说明。

1. 三维问题在轴对称、均匀初值与周向均匀边界下精确退化为二维 (r, z) 轴对称问题，问题二至四采用中截面径向一维模型。烘房环境（空气温度与水分浓度）在附件 1 相邻采样点之间按分段线性变化，求解边界逐点保真采用原始记录——记录本身带有烘房温控的物理波动（温度相邻点最大回落 0.45 ℃），不作人为平滑。
2. 对于超出附件 1 记录范围（$t > 14400$ s）的时段，按题目“恒温干燥阶段”的设定取环境温度 50 ℃、水分浓度 0.05 kg/kg 恒定。附件 1 末段在 50 ℃ 附近趋稳，末值（50.17 ℃、0.0499 kg/kg）与设定平台的差异构成烘干时长约 0.3 h 的不确定性（灵敏度分析给出三口径对照）。
3. 将附件 1 中的“空气水分浓度”解读为与烘房湿热环境相平衡的药材表面平衡含水率，作为传质边界的远场参考量：该列数值（约 0.02–0.05 kg/kg）与干基含水率量级相差约两个数量级、不能直接相减；此解读使传质驱动力方向正确、单位闭合，且无需额外引入吸附等温线。

4. 题目明确要求“根据附件 2 确定”且未给收缩本构系数，直接采信附件 2 的半径时序；收缩沿径向均匀（随体）、轴长保持 25 cm，是数据条件下的最小假设 [13]，其影响量级见附录 B 对照。附录 4 密度经验式 $\rho(C) = 760 + 90C$ 为经验体密度而非质量守恒本构：与附件 2 收缩数据联算时隐含干物质减少约 13%（$\rho(C)R^2/(1+C)$ 随干燥下降），因密度仅进入热方程的热容 ρc_p、不进入水分方程（干基口径），该不一致不影响烘干时长，此处披露为模型限制。
5. 题给参数构成自洽的“有效模型”口径，不显式加入蒸发潜热：按名义汽化潜热（2.4 MJ/kg）估算，1800 s 内表面温度将被拉低约 30 ℃（附录 B 实测），与 $h = 25$ W/(m^{2}K) 支撑的传热量级矛盾、且附录未给潜热参数。故题给的 h 视为包含辐射等全部表面换热途径的有效系数（附录 B 辐射对照：h 上浮 26% 仅使表面温度移动 0.8 ℃），水分通量视为以干基含水率为驱动力的有效通量，物性公式按题面采用。

上述假设中，环境长时程外推（假设 2）与表面平衡含水率解读（假设 3）对结果的影响需要重点说明：前者直接控制干燥速率，是烘干时长的主要不确定来源；后者主要影响表层含水率的绝对数值。二者在灵敏度分析中均加以扰动考察。

四、 符号说明

下表列出全文反复使用、直接进入控制方程、边界条件或干燥判据的核心符号，其余一次性中间量在正文首次出现处随文说明。

符号	含义	单位
r	到药材中心的径向距离	m（输出 cm）
z	轴向坐标（$z=0$ 为中截面，$z=H$ 为端面）	m
φ	周向坐标	rad
t	时间	s（输出 h）
$T(r,z,t)$	药材内部温度场	℃
$C(r,z,t)$	药材内部干基含水率场	kg/kg
ξ	Landau 归一化坐标 $r/R(t)$	无量纲
ρ	药材密度	kg/m^3
c_p	药材比热容	J/(kg·K)
k	药材热传导系数	W/(m·K)
α	热扩散率 $k/(\rho c_p)$	m^2/s
D	水分扩散系数	m^2/s
h	表面对流换热系数	W/(m^2·K)
h_m	表面对流传质系数	m/s

符号	含义	单位
$T_\infty(t)$	烘房空气温度（附件 1）	℃
$C_{eq}(t)$	药材表面平衡含水率（附件 1 解读）	kg/kg
β	端面暴露系数（$h_b=\beta h$, $h_{mb}=\beta h_m$）	无量纲
R, $R(t)$	圆柱半径（问题四随时间变化，附件 2）	m
$\dot{R}(t)$	半径收缩率	m/s
H	半域长度 $L/2$	m
L	圆柱长度	m
t^*	烘干时长（全场含水率首次低于 0.15 的时刻）	h
N_r, N_z	径向、轴向控制体个数	无量纲

五、 问题一：预热平衡阶段热质耦合场的三维精确退化与轴向降维

5.1 三维控制方程与轴对称精确退化

把药材视为半径 $R=0.02\,\mathrm{m}$、长 $L=0.25\,\mathrm{m}$ 的圆柱体。在柱坐标 (r,φ,z) 下，药材内部非稳态传热与水分扩散的三维控制方程为

$$\rho c_p\frac{\partial T}{\partial t}=\frac{1}{r}\frac{\partial}{\partial r}\Big(k\,r\frac{\partial T}{\partial r}\Big)+\frac{1}{r^2}\frac{\partial}{\partial\varphi}\Big(k\frac{\partial T}{\partial\varphi}\Big)+\frac{\partial}{\partial z}\Big(k\frac{\partial T}{\partial z}\Big),\tag{1}$$

$$\frac{\partial C}{\partial t}=\frac{1}{r}\frac{\partial}{\partial r}\Big(rD\frac{\partial C}{\partial r}\Big)+\frac{1}{r^2}\frac{\partial}{\partial\varphi}\Big(D\frac{\partial C}{\partial\varphi}\Big)+\frac{\partial}{\partial z}\Big(D\frac{\partial C}{\partial z}\Big).\tag{2}$$

其中，ρ, c_p, k 为密度、比热与热导率，D 为水分扩散系数。题设给出：圆柱横截面为圆（绕轴旋转对称）、初值 $T=C$ 为常数（全场均匀）、烘房环境 $T_\infty(t)$ 与 $C_{eq}(t)$ 与周向 φ 无关。由解的适定性，问题的解不依赖 φ，式(1)(2) 中所有 $\partial/\partial\varphi$ 项恒为零，三维问题精确退化为二维 (r,z) 轴对称问题。需要强调：这一退化由对称性严格保证，是三维问题的精确形式而非近似，三维解在 φ 方向上本就没有变化。此处“到药材中心的距离”特指圆柱**中截面（$z=0$）上的径向坐标**。

退化为 (r,z) 后，有限长圆柱的端面效应仍然存在：两端面同样与烘房环境换热换质，端面附近的药材比中截面多一个干燥面，会更快地升温和失水。因此本问以 (r,z) 轴对称模型为正式模型，把端面效应一并解出；能否进一步降为一维径向模型，交由第 5.6 节的逐位误差分析定量论证。

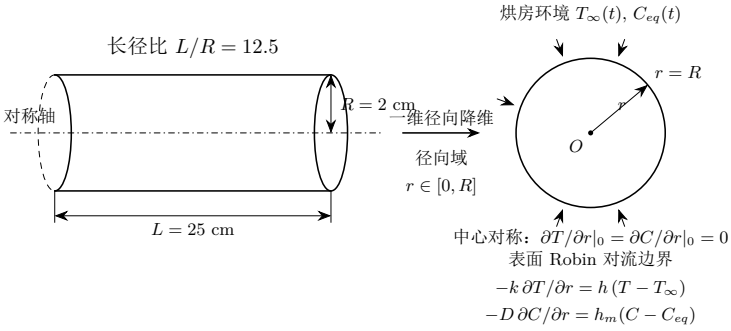


图 2　圆柱几何与轴对称边界示意

5.2 二维 (r, z) 轴对称模型与控制方程

由中截面对称性只需解半域 $z \in [0, H]$，$H = L/2 = 0.125\,\mathrm{m}$，其中 $z = 0$ 为中截面、$z = H$ 为端面，几何与边界设置如图2所示。式(1)退化后的控制方程为

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\Big(k\, r \frac{\partial T}{\partial r}\Big) + \frac{\partial}{\partial z}\Big(k \frac{\partial T}{\partial z}\Big), \tag{3}$$

$$\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\Big(r D \frac{\partial C}{\partial r}\Big) + \frac{\partial}{\partial z}\Big(D \frac{\partial C}{\partial z}\Big). \tag{4}$$

预热平衡阶段采用附录 2 的常物性：密度 $\rho = 820\,\mathrm{kg/m^3}$、比热 $c_p = 2600\,\mathrm{J/(kg{\cdot}K)}$、热导 $k = 0.36\,\mathrm{W/(m{\cdot}K)}$，对流换热系数 $h = 25\,\mathrm{W/(m^2{\cdot}K)}$、对流传质系数 $h_m = 8 \times 10^{-7}\,\mathrm{m/s}$，取值与热风干燥中实测的对流换热与传质系数量级一致 [8]。温度方程化为

$$\frac{\partial T}{\partial t} = \alpha\left(\frac{\partial^2 T}{\partial r^2} + \frac{1}{r}\frac{\partial T}{\partial r} + \frac{\partial^2 T}{\partial z^2}\right), \qquad \alpha = \frac{k}{\rho c_p} = 1.6886 \times 10^{-7}\ \mathrm{m^2/s}. \tag{5}$$

水分扩散系数仅随含水率变化而与温度无关 [3, 4]：

$$D(C) = 7 \times 10^{-9}\, e^{-0.89/C}. \tag{6}$$

由于温度方程不含 C、水分方程仅通过 $D(C)$ 呈现非线性，两个场构成单向弱耦合：温度场可独立求解，水分场随后在已知温度下推进。扩散系数 $D(C)$ 的形态决定了失水的快慢，其随含水率的变化如图 3所示。

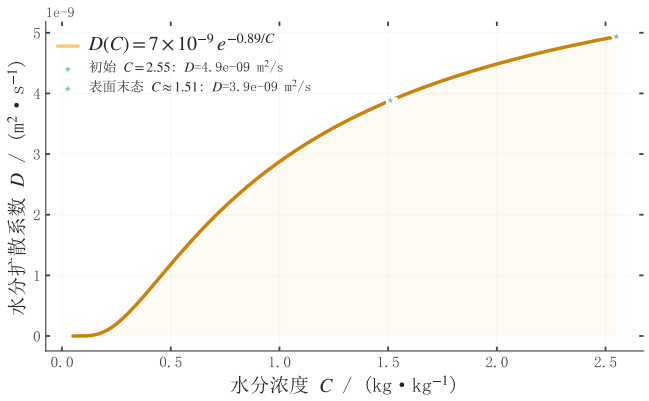


图 3　水分扩散系数随含水率的变化曲线

由图 3可见，D 随含水率增大而增大、在含水率趋于零时迅速衰减——药材越湿水分越易迁移，干燥后期表层含水率降低显著减缓扩散，这正是干燥末期耗时剧增的内在原因，也提示长时程求解需对 $C \to 0$ 附近的数值刚性加以防护。

5.3 初始、边界条件与环境驱动

初始时刻药材各处均匀，$T(r,z,0)=28\,^{\circ}\mathrm{C}$，$C(r,z,0)=2.55\,\mathrm{kg/kg}$。中心 $r=0$ 与中截面 $z=0$ 处由轴对称给出零通量条件 $\partial_r T|_0=\partial_r C|_0=0$、$\partial_z T|_0=\partial_z C|_0=0$，其中径向 $1/r$ 奇点用对称极限 $\lim_{r\to0}\frac{1}{r}\partial_r(\cdot)=\partial_{rr}(\cdot)$ 处理。侧表面 $r=R$ 处为对流型（Robin）边界：

$$-k\left.\frac{\partial T}{\partial r}\right|_{R}=h\left[T(R,z,t)-T_{\infty}(t)\right],\qquad -D\left.\frac{\partial C}{\partial r}\right|_{R}=h_m\left[C(R,z,t)-C_{eq}(t)\right]. \tag{7}$$

其中端面换热、传质系数按端面暴露系数 β 缩放，$h_b=\beta h$、$h_{mb}=\beta h_m$。第 5.6 节将表明轴向偏差虽随 β 单调放大，但降维结论对 β 不敏感，这一未知参数不影响本文主线。空气温度 $T_{\infty}(t)$ 与表面平衡含水率 $C_{eq}(t)$ 由附件 1 前 1800 s 的记录分段线性插值逐秒给出，其时间变化如图4所示：前 1800 s 内烘房温度由 28 ℃ 稳步升至约 41.5 ℃，表面平衡含水率则维持在 0.02 kg/kg 量级的低位。二者量级悬殊，正说明式(7)中传质驱动力应取药材侧含水率与表面平衡含水率之差，而非直接与空气水分浓度相减。

$$-k\left.\frac{\partial T}{\partial z}\right|_{H}=\beta h\left[T(r,H,t)-T_{\infty}(t)\right],\qquad -D\left.\frac{\partial C}{\partial z}\right|_{H}=\beta h_m\left[C(r,H,t)-C_{eq}(t)\right]. \tag{8}$$

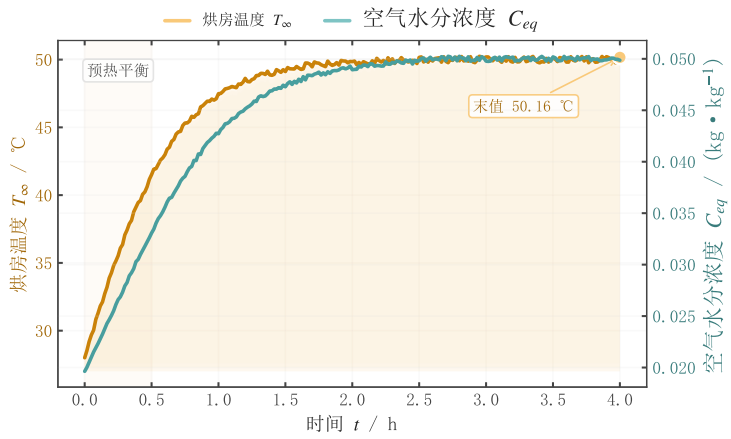


图 4　预热阶段烘房温度与表面平衡含水率随时间变化

升高的空气温度构成加热药材的主要驱动，也是水分扩散逐渐加快的前提。

5.4 数值求解

在 (r, z) 平面采用柱坐标有限体积法离散[9,11]，径向 $N_r = 100$ 个控制体（$\Delta r = 0.02\,\mathrm{cm}$）、轴向 $N_z = 50$ 个（$\Delta z = 2.5\,\mathrm{mm}$）。柱坐标下控制体体积权重 $V_i = \frac{1}{2}(r_{i+1/2}^2 - r_{i-1/2}^2)$，中心界面的权重自然为零——$1/r$ 奇点不用任何特殊处理，这也是我们选有限体积而不是差分的主要原因。水分方程界面的扩散系数取两侧节点的调和平均，以保证界面通量连续、对强非线性更稳健（界面权重的记账细节见附录 C）。时间推进上，温度方程用 Crank–Nicolson 格式（二阶、无条件稳定[10]），矩阵不随时间变化、一次 LU 分解后逐步回代；水分方程因 $D(C)$ 指数非线性改用向后欧拉配 Picard 迭代（容差 10^{-8}、上限 20 次）——向后欧拉 L-稳定，在干燥后期刚性区间不产生虚假振荡；Picard 每步只解一次线性系统，避免 Newton 迭代在低含水率处放大误差。

线性代数层面，一维径向系统为三对角矩阵、用追赶法求解；二维 (r, z) 系统为带型矩阵、用 LAPACK 带状直接解（单次约 17 ms），两条路线的完整权衡见附录 C。Robin 边界用组合热阻/传质阻（单元中心 $\leftrightarrow$ 表面 $\leftrightarrow$ 环境）闭合，等价于式(7)。时间步 $\Delta t = 1\,\mathrm{s}$ 推进 1800 步；输出按题目网格（0–2 cm 每 0.1 cm，21 点）重构：中截面取 $z = 0$ 行，中心 $r = 0$ 用偶函数二次重构 $T(0) = (9T_0 - T_1)/8$，表面用 Robin 条件反算。

网格收敛上，对 $(N_r, N_z) = (50, 25), (100, 50), (200, 100)$ 三套网格作加密对比，1800 s 剖面（插值到同一输出网格）的最大相对差见表1：中截面温度、含水率与端面温度均已充分收敛（$\leq 10^{-4}$）；端面含水率收敛较慢（$3.9 \times 10^{-2} \to 4.7 \times 10^{-3}$），因端面水分边界层特征厚度约 1 mm、$\Delta z = 2.5\,\mathrm{mm}$ 偏粗，该处数值误差远小于端面偏差本身（约 1 kg/kg），不影响结论。取 $(100, 50)$ 为生产网格。

表 1　(r,z) 模型网格收敛（1800 s 剖面最大相对差，$\beta=1$）

指标	$(100,50)$ vs $(50,25)$	$(200,100)$ vs $(100,50)$
T 中截面	1.0×10^{-5}	0
C 中截面	2.9×10^{-4}	1.0×10^{-4}
T 端面	6.2×10^{-4}	1.6×10^{-4}
C 端面	3.9×10^{-2}	4.7×10^{-3}

5.5 结果与分析

按上述二维 (r,z) 轴对称模型求得预热阶段的温度与含水率场。表 2、表 3给出题目指定的 7 个时刻、5 个径向位置处的结果，均取自圆柱**中截面**（$z=0$）**的径向剖面**，即题目所述“到药材中心的距离”对应位置。

表 2　预热平衡阶段药材内部温度 $T(r,t)$（单位：°C；到中心距离单位：cm）

时间 t/s	$r=0$	$r=0.5$	$r=1.0$	$r=1.5$	$r=2.0$
100	28.0001	28.0003	28.0041	28.0327	28.1801
300	28.0408	28.0635	28.1514	28.3681	28.8489
600	28.4534	28.5361	28.8040	29.3160	30.1653
900	29.3243	29.4583	29.8755	30.6162	31.7305
1200	30.5427	30.7099	31.2224	32.1126	33.4277
1500	31.9957	32.1868	32.7661	33.7463	35.1203
1800	33.5753	33.7721	34.3642	35.3622	36.7856

表 3　预热平衡阶段药材内部含水率 $C(r,t)$（单位：kg/kg；到中心距离单位：cm）

时间 t/s	$r=0$	$r=0.5$	$r=1.0$	$r=1.5$	$r=2.0$
100	2.5500	2.5500	2.5500	2.5500	2.2464
300	2.5500	2.5500	2.5500	2.5491	2.0506
600	2.5500	2.5500	2.5500	2.5350	1.8769
900	2.5500	2.5500	2.5497	2.5043	1.7547
1200	2.5500	2.5500	2.5481	2.4643	1.6586
1500	2.5500	2.5499	2.5445	2.4204	1.5788
1800	2.5500	2.5497	2.5382	2.3753	1.5103

从表 2可见，升温由表及里推进：1800 s 时表面达 36.79 °C，而中心仅 33.58 °C，二者相差约 3.2 °C，中心明显滞后。表 3则显示失水高度集中于表层：1800 s 时表面含水率由初值 2.55 降至 1.51 kg/kg，而中心几乎保持 2.55 kg/kg 不变。这一“表层剧降、中心不动”的格局，可由毕渥数与特征时间加以解释：热毕渥数 $Bi_h = hR/k = 1.39$、传质毕渥数 $Bi_m = h_m R/\bar{D} \approx 3.2$ 均为 $O(1)$，表面对流阻与内部扩散阻同量级，内部梯度不可忽略；热扩散特征时间 $\tau_h = R^2/\alpha \approx 2369$ s 略大于 1800 s，故此时温度尚未追平环境，而水分扩散特征时间 $\tau_m = R^2/\bar{D}$ 远大于 1800 s，湿锋在预热阶段还来不及抵达中心。

温度场在 r–t 平面上的整体分布与含水率场的三维演化分别见图5 与图6。

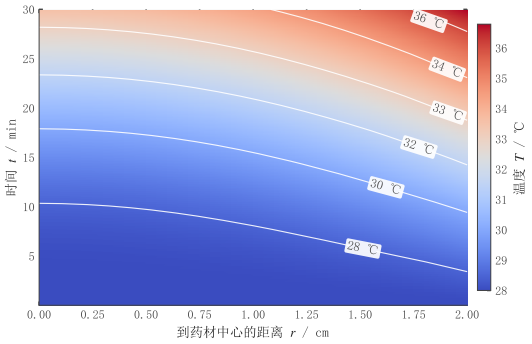


图 5　预热平衡阶段温度场时空分布

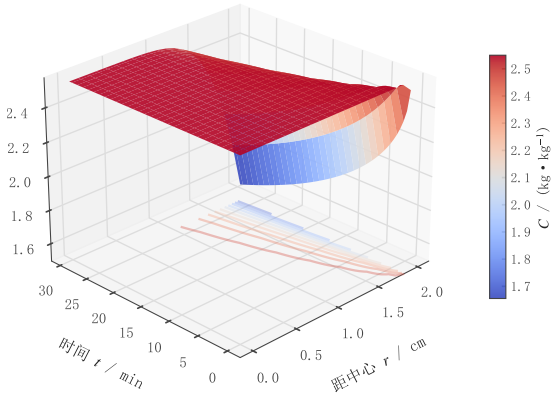


图 6　预热平衡阶段含水率场三维演化

温度场等值线沿时间整体抬升，靠近表面一侧更密（图5），表明表层温度梯度更大、中心响应滞后，与表 2 一致。

含水率曲面（图6）呈现出一个从表面向内凹陷、而内部大片维持在初值平台的形态：失水只在贴近表面的薄层内发生，中心区域几乎是一整块未被扰动的高含水率平台。

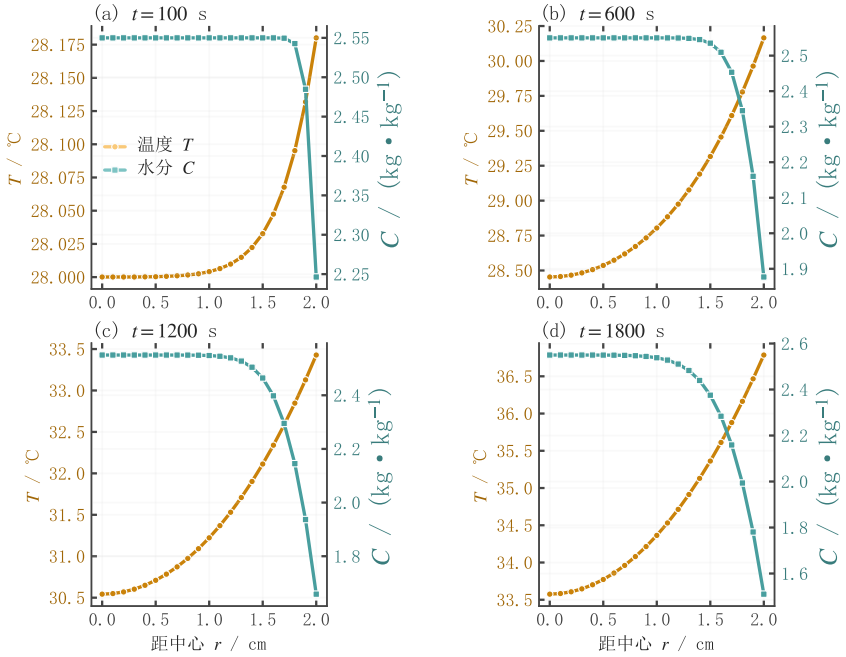


图 7　四个时刻温度与含水率沿半径分布

图7中，温度剖面整体上移且表高于内；含水率剖面在表面附近形成不断加深、缓慢内推的下凹湿锋，中心段长时间维持水平，与“先升温、后失水，且失水由表及里”的物理图像吻合。中心含水率在 1800 s 时仍接近初值 2.55 kg/kg，是水分扩散特征时间远大于该时段的必然结果；若模型算出中心含水率明显下降，反而意味着传质被高估。

5.6 轴向效应误差分析与降维论证

上节表 2、表 3虽取自二维中截面剖面，但若能证明这些位置的结果与去掉轴向项的**一维径向模型**逐位一致，则问题二至四即可安全地在计算量更小的一维径向框架下推进。为此把方程中的轴向二阶项略去，得到仅含径向的一维模型，并将其解与二维精确解逐位对比。

二维核验给出两点关键结论。其一，**中截面（即表 2、表 3给值的位置）在 1800 s 内完全不受轴向影响**：将二维解在中截面 $z = 0$ 的径向剖面与一维径向解逐位对比，温度与含水率的偏差 $E_T^{\mathrm{mid}} = E_C^{\mathrm{mid}} = 0$（四位小数逐位不变，中截面温度仍为 33.5753 → 36.7856 °C、含水率仍为 2.5500 → 1.5103 kg/kg）。物理原因是预热阶段的特征扩散长度（温度 $\sqrt{\alpha t} \approx 1.74$ cm、水分 $\sqrt{Dt} \approx 0.30$ cm）远小于中截面到端面的 12.5 cm，轴向信息尚未传到中截面。此外，当取 $\beta = 0$（端面绝热）时二维解全程逐位复现一维解（偏差约 10^{-12}，机器精度），佐证二维程序在退化情形下与一维模型完全同口径。

其二，**轴向偏差局限于端面附近的薄层**。以基准 $\beta = 1$、1800 s 计，端面轴心 $(r = 0, z = H)$ 温度为 37.28 °C，较一维中心高 **+3.70** °C；端面轴心含水率为 1.559 kg/kg，较一维中心低 **−0.99** kg/kg——端部因双面（侧面加端面）受热失水，比中截面更干更热。但沿轴向的侵入深度很浅：温度偏差超过 0.05 °C 的区域自端面向内仅约 **4** cm，含水率偏差超过 0.01 kg/kg 的区域仅约 **1** cm。端面暴露越强偏差越大，$\beta = 0.5/1/2$ 对应端面轴心温升 $+2.39/+3.70/+5.09$ °C、含水率降 $-0.59/-0.99/-1.48$ kg/kg，呈单调放大。上述误差的时间演化与轴向衰减汇总于图8。

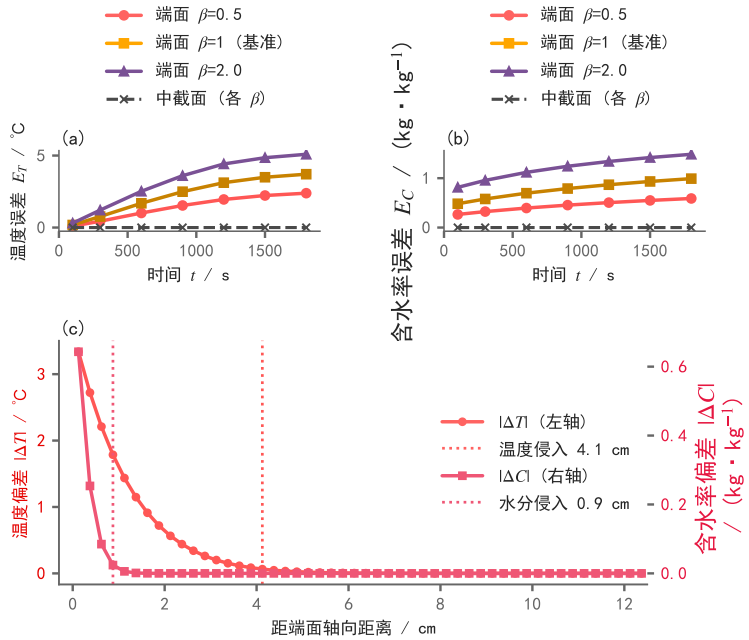


图 8　二维 r–z 轴对称核验

图 8中，(a)(b) 两幅的中截面误差曲线始终贴合零线，而端面误差随时间与 β 单调增大；(c) 显示偏差自端面向内迅速衰减，温度、水分分别在约 4 cm、1 cm 处即回落到阈值以下，直观印证了轴向效应仅局限于端面薄层。

由此，降维的依据不再停留在“长径比 $L/R = 12.5$ 大、端面占比小”这一几何直觉上，而是升级为定量结论：轴向效应在 1800 s 内被约束在端面 1–4 cm 的薄层内，而占圆柱绝大部分长度的中截面区域与一维径向解**逐位零偏差**。因此正式的表 2、表 3与 result1.xlsx 均可直接采用一维中截面结果，其“到中心距离”即中截面径向距离。

5.7 降维结论向长时间尺度的延伸

预热 1800 s 的零偏差可推广到全程：水分轴向扩散很慢，数十小时内也只侵入端面约 1 cm，远小于中截面到端面的 12.5 cm；温度扩散虽快，但只使全场更均匀，不会在中截面产生额外径向差异。因此问题二至四用中截面径向一维模型可靠。该结论只适用

于中截面及柱体主体；端部约 1–4 cm 若需精细温湿分布，仍应保留二维 r–z 模型。后续沿用本问的方程、离散、边界与单位约定。

六、 问题二：变物性两阶段全过程耦合场模型

题目对温度与含水率的输出仍按“到药材中心的径向距离”给值，其物理含义正是中截面上的径向坐标。因此问题二至四同样采用中截面径向一维模型处理，“到中心距离”均指中截面径向距离；本问只是在此降维框架下把常物性升级为随含水率与温度变化的变物性，几何降维本身不再重复论证。

6.1 从常物性到变物性的升级

问题二要刻画包含预热平衡与恒温干燥两个阶段的整个烘干过程，物性统一改用附录 3 的经验公式。与问题一最本质的区别在于：密度、比热、热导率不再是常数，而随含水率变化；水分扩散系数同时依赖含水率与温度。具体为

$$\rho(C) = 650 + 128C, \qquad c_p(C) = 1450 + 2736\frac{C}{C+1}, \qquad k(C) = 0.21 + 0.38\frac{C}{C+1}, \tag{9}$$

$$D(C,T) = 2.4 \times 10^{-3}\, e^{-0.45/C}\, e^{-3850/T_K}, \qquad T_K = T + 273.15. \tag{10}$$

这些变物性随含水率与温度的变化趋势如图9所示。随着含水率下降，密度、比热与热导率均随之减小，意味着干燥后期药材的储热与导热能力都在减弱；而扩散系数受含水率与温度两个因素竞争影响——含水率降低会压低 $e^{-0.45/C}$、温度升高又抬升 e^{-3850/T_K}。正因 D 中含有温度因子、且三个物性都随含水率改变，温度场与含水率场之间形成双向强耦合：温度影响水分迁移速率，含水率又反过来改变导热与储热，二者必须联立推进。有一个容易踩的坑：式(10)中温度必须以开尔文代入指数项——$e^{-3850/T}$ 在 $T \sim 300\,\mathrm{K}$ 附近对温度单位极敏感，误用摄氏度会让 D 整体失真近两个数量级。

6.2 双向耦合的数值求解

空间离散沿用问题三/四统一的节点型有限体积格式（生产网格 $n = 640$）：界面扩散系数按附录 3 公式沿节点线性剖面作 8 点 Gauss 积分（指数型 $D(C)$ 的精确面积分）、热导取调和平均；时间推进采用自适应 BDF 隐式积分（相对容差 10^{-8}，步长由误差控制自动选取），非线性耦合由内嵌 Newton 隐式求解。3 h 输出全部落在附件 1 的 4 h 记录范围内，边界直接由附件 1 插值给出，无需外推。变物性引擎的正确性先做了退化验证：把物性换回附录 2 的常系数、按问题一的 1800 s 工况跑一遍并与问题一结果对齐，温度与含水率在四位小数下一致（最大差 0.0001 ℃/0.0002 kg/kg）。本问与问题三/四共用同一引擎，仅物性、域与终止判据不同。

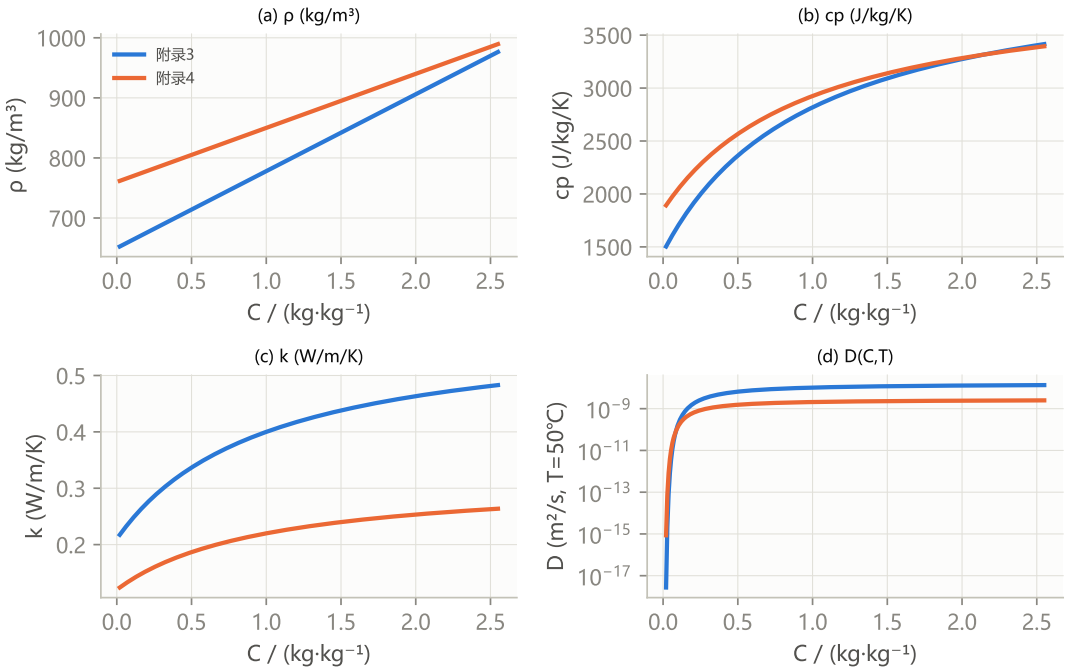


图 9　变物性参数随含水率与温度的变化

6.3 结果与分析

表4、表5给出 3 h 内每半小时、5 个径向位置处的温度与含水率。

表 4　整个烘干过程药材内部温度 $T(r,t)$（单位：℃；到中心距离单位：cm）

时间 t/h	$r=0$	$r=0.5$	$r=1.0$	$r=1.5$	$r=2.0$
0.0	28.0000	28.0000	28.0000	28.0000	28.0000
0.5	32.1892	32.3821	32.9659	33.9608	35.4130
1.0	40.3816	40.5536	41.0605	41.8782	42.9976
1.5	45.8468	45.9348	46.1930	46.6051	47.1401
2.0	48.4502	48.4881	48.5984	48.7735	49.0033
2.5	49.4670	49.4792	49.5137	49.5653	49.6609
3.0	49.8495	49.8553	49.8746	49.9101	49.9664

与问题一相比，全过程的演化图像发生了质的变化。温度方面（表4），药材在约 1.5 h 后中心即升至 45.8 ℃，3 h 时全场温度趋于均匀、中心与表面同处 49.8 ℃ 附近，说

表 5　整个烘干过程药材内部含水率 $C(r,t)$（单位：kg/kg；到中心距离单位：cm）

时间 t/h	$r=0$	$r=0.5$	$r=1.0$	$r=1.5$	$r=2.0$
0.0	2.5500	2.5500	2.5500	2.5500	2.5500
0.5	2.5499	2.5489	2.5256	2.3257	1.6486
1.0	2.5257	2.4948	2.3578	2.0230	1.4711
1.5	2.3861	2.3256	2.1344	1.8020	1.3476
2.0	2.1709	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3.0	1.7662	1.7165	1.5702	1.3333	1.0081

明经过预热后药材已基本与环境热平衡，进入恒温干燥。含水率方面（表5），3 h 时表面含水率降至 1.01 kg/kg、中心降至 1.77 kg/kg，失水已不再局限于表层、干燥前沿明显向内推进。中心与表面的温度、含水率随时间的演化对比见图10。

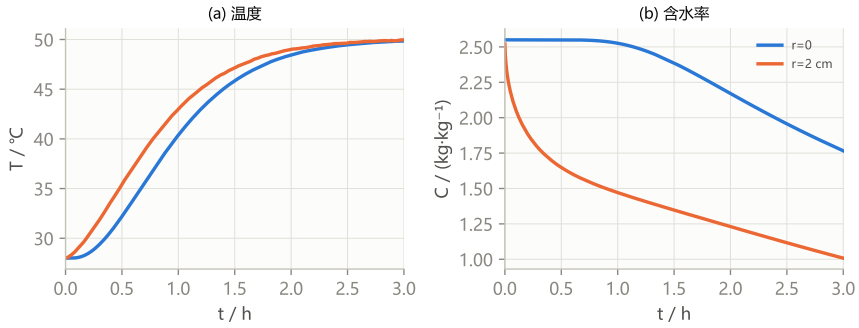


图 10　整个烘干过程中心与表面温度、含水率演化

观察图10中的四条曲线，温度呈先快后慢的饱和式上升，中心曲线始终紧跟表面曲线之后并在后期几乎重合，印证了热平衡的建立；含水率曲线则表面下降最快、中心滞后，但二者的差距随时间逐渐拉开——这正是升温使扩散系数增大、失水由表及里加速的结果，与问题一仅表层失水的情形形成鲜明对照。含水率场在整个时段上的空间分布见图11。

图11的时空分布进一步表明，高含水率区域随时间从表面向中心持续退缩，等值线整体向中心方向平移，反映干燥前沿稳步深入。本问建立的变物性、双向耦合求解引擎，将被问题三直接用于确定烘干时长，并在问题四中换用附录 4 物性后继续沿用。

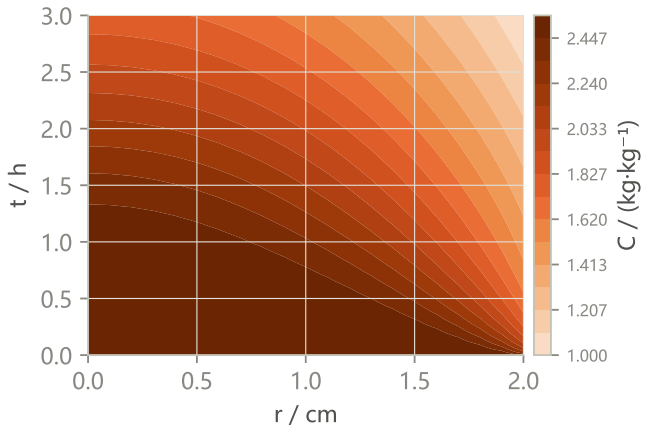


图 11　整个烘干过程含水率场时空分布

七、 问题三：干燥终止时间的确定

7.1 全场达标判据

按干燥要求，药材各处含水率都应低于 0.15 kg/kg 才算干透。这一“各处”的要求意味着判据必须作用于全场最大含水率，而非平均值或表面值：

$$t^{*}=\min\Big\{\,t>0:\ \max_{0\le r\le R}C(r,t)<0.15\ \mathrm{kg/kg}\Big\}. \tag{11}$$

由于水分靠扩散由内向外排出，中心 $r=0$ 处失水最慢、通常是全场最湿点，因此式(11)实际由中心是否达标决定。求解仍采用问题二的变物性、双向耦合引擎（附录 3 物性），只是把积分区间从 3 h 延长到含水率达标为止。

烘干可能持续数十小时，远超附件 1 覆盖的 4 h。对 $t>14400$ s 的时段，按模型假设取环境温度 50 ℃、表面平衡含水率 0.05 kg/kg 的恒温平台（假设 2）。数值上沿用问题二的节点型有限体积与自适应 BDF 隐式积分（相对容差 10^{-8}），达标时刻由事件函数（全场最大含水率 -0.15）的 dense output 求根精确定位，并对下一整秒/整分钟作严格“<0.15”未舍入检查。空间网格 $n=200/400/800$ 时 t^{*} 为 57.4728/57.4724/57.4723 h、$n=1280$ 生产值为 57.4723 h（完整收敛链、时间/求积阶敏感性汇总于第 10 节）；另以独立编写的节点型有限差分程序（向后欧拉、固定步长）交叉验证（$N=400$ 时偏差 0.9% 量级、随网格加密向同一极限收敛），并以调和平均面系数与 Gauss 积分的两套面系数口径互相印证（前者在指数型 $D(C)$ 下系统偏慢约 0.07 h、加密后收敛到同一极限）。时间与空间离散均已收敛。干燥后期表层高梯度干壳层与网格分辨率的关系见图12。

7.2 轴向维度在烘干时长判定中的再论证

问题一已在 1800 s 的预热窗口内定量论证了轴向影响（第 5.6、5.7 节），但问题三积分跨度长达 57 h——端面效应会不会在这么长的时间里累积起来、最终动摇 t^{*} 呢？下

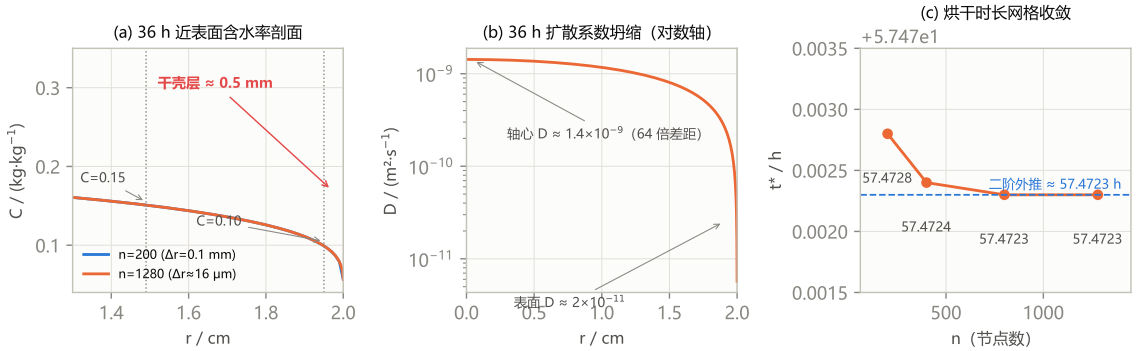


图 12　干壳层与网格分辨率（36 h）

面从四条路径逐一排查。

（1）中截面处的轴向状态。中截面 $z=0$ 是对称面，初值与边界在镜像反射下不变，故 $\partial C/\partial z|_{z=0}=0$。但中截面两侧仍存在指向端面的轴向浓度梯度（$\partial^2 C/\partial z^2|_{z=0}\neq 0$），会以轴向泄流方式向端面输运水分——这是端面效应触及中截面的唯一通道，必须定量评估。

（2）判据点位置。判据式(11)由全场最大含水率决定。初始场均匀、各边界均为汇（$C_{eq}\le 0.05\,\mathrm{kg/kg}$ 远低于内部 C），由抛物型方程极值原理，C 沿径向 r 与轴向 $|z|$ 均单调递减，任意时刻最湿点即中截面轴心 $(r,z)=(0,0)$，t^* 完全由该点演化决定。

（3）端面对判据点的因果隔离。端面条件只能通过扩散传播影响轴心，累积尺度为 $\ell(t)=\sqrt{\int_0^t D\,\mathrm{d}s}$。代入问题三数值解：$\ell(12\,\mathrm{h})=2.06\,\mathrm{cm}$，$\ell(t^*)=2.75\,\mathrm{cm}$，均远小于轴心到端面的 $12.5\,\mathrm{cm}$。两种收紧口径均已核查：预热期轴心温度偏低使 D 最多低估约 20%，修正后 ℓ 仍不超过 $3.0\,\mathrm{cm}$；即使按全程恒定最大 D 的极端保守口径，ℓ 也只有 $5.2\,\mathrm{cm}$。端面扰动对轴心的影响以 $\mathrm{erfc}\big(L/(2\ell)\big)$ 衰减，12 h 时为 1.8×10^{-5}，t^* 时为 1.2×10^{-3}（含修正后 2.2×10^{-3}）。即便把端面换成“含水率恒为零”的极端汇条件，对判据点的影响也不超过约 $0.006\,\mathrm{kg/kg}$；真实端面仅为与侧面同强度的对流传质边界，影响更小。

（4）方向性：一维结果为保守上界。对任意端面暴露系数 $\beta\ge 0$，端面都是额外水分出口。由比较原理，(r,z) 问题的解处处不超过 $\beta=0$（端面绝热）的解，后者沿 z 均匀、恒等于一维径向解。故 $t^*_{\mathrm{1D}}\ge t^*_{(r,z)}$：真实烘干时长只可能短于 57.47 h，按 57.47 h 作工艺规划偏于安全。

至此，判据点位于中截面轴心，端面条件对其无因果影响，且一维估计偏保守。该结论已由问题一的 (r,z) 数值验证背书——1800 s 内中截面与一维解逐位零偏差、端面偏差局限于端面 1–4 cm 薄层（图13）。因此问题三（及沿用同一判据的问题四）无需引入 z 方向，$t^*=57.47\,\mathrm{h}$ 在三维精确解框架下依然成立，且为保守上界。适用边界同样明确：若判据改为端面区域或全场平均含水率、或药材长径比显著减小，则轴向泄流不可忽略，应回到 (r,z) 模型。

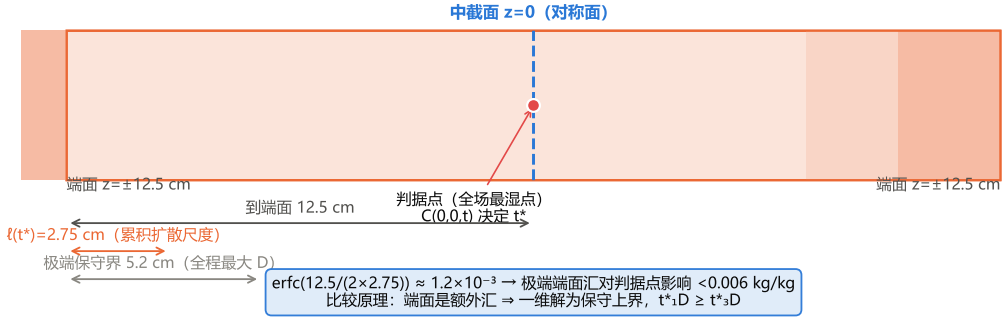


图 13　问题三轴向维度再论证示意

7.3 结果与分析

求解得到烘干时长 $t^* = 57.47\,\mathrm{h}$。表6给出干燥过程中每 6 h、5 个径向位置处的含水率，直至达标时刻。

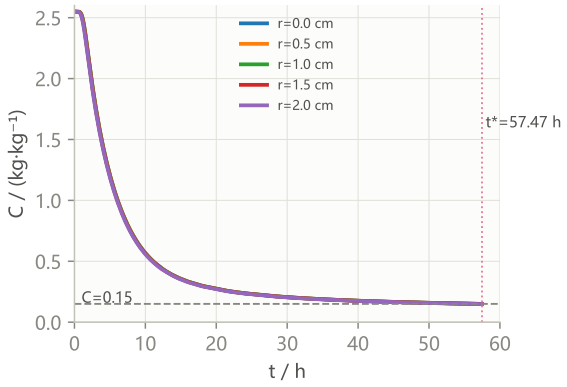


图 14　各半径处含水率下降曲线与干燥判据

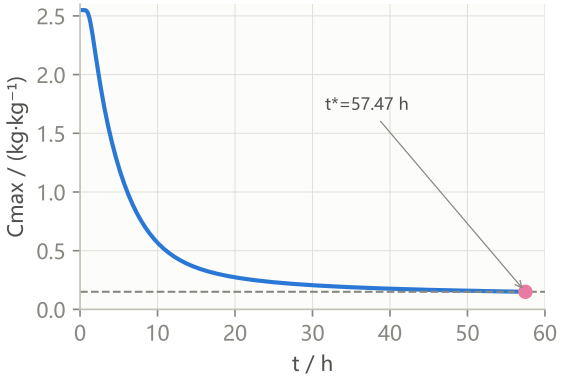


图 15　全场最大含水率随时间下降与烘干时长

图14中各条曲线按“越靠中心、下降越慢”的次序排列，且都呈现前陡后缓的形态；越靠内层的曲线越晚触及 0.15 kg/kg 的判据线，中心曲线最后达标。这说明烘干时长由中心控制，也解释了为何以全场最大值而非平均值作为判据。将判据换成全场最大含水率随时间的单条曲线后，其与阈值线的交点即确定 t^*，如图15所示：该曲线单调下降、无回升，与阈值线交于 57.47 h，即为固定尺寸情形下的烘干时长。该结果压在恒温阶段环境保持稳定这一情景假设上——附件 1 只记录到 4 h，57.47 h 中绝大部分时段的环境是按恒温阶段设定平台（50 ℃、0.05 kg/kg）外推的，环境温度高低会直接改变干燥速率。因此应将其理解为给定恒温段环境下的情景性结论，其对环境设定的敏感程度将在灵敏度分析中定量考察；$t^* = 57.47\,\mathrm{h}$ 也作为问题四考察尺寸收缩效应的对照基准。

表6清晰呈现了“先快后慢、由表及里”的干燥节奏。前 12 h 失水最为迅猛：中心

表 6　烘干过程含水率 $C(r,t)$ 下降至判据（每 6 h；到中心距离单位：cm；$t^{*}=57.47$ h）

时间 t/h	$r=0$	$r=0.5$	$r=1.0$	$r=1.5$	$r=2.0$
6	1.0170	0.9881	0.9015	0.7550	0.5328
12	0.4561	0.4432	0.4031	0.3297	0.1642
18	0.2988	0.2915	0.2687	0.2253	0.0845
24	0.2378	0.2327	0.2167	0.1855	0.0665
30	0.2058	0.2018	0.1892	0.1641	0.0598
36	0.1859	0.1825	0.1718	0.1504	0.0566
42	0.1721	0.1691	0.1597	0.1407	0.0549
48	0.1618	0.1592	0.1507	0.1334	0.0537
54	0.1539	0.1514	0.1436	0.1277	0.0530
57.47*	0.1500	0.1477	0.1402	0.1249	0.0527

含水率由 2.55 骤降至 0.46 kg/kg，表面更是降到 0.16 kg/kg 附近；此后下降显著放缓，从 24 h 到 57.47 h 中心含水率仅从 0.24 缓慢降至 0.15 kg/kg。这种减速正是扩散系数随含水率降低而减小的直接后果——越接近干透，剩余水分越难排出，干燥进入漫长的尾段。表6还显示，达标时刻表面含水率早已降至 0.05 kg/kg，远低于阈值，全场最湿点确实位于中心，与判据的设定一致。各半径处含水率随时间的下降曲线见图14。

八、 问题四：尺寸收缩动边界模型与烘干时长

8.1 动边界与坐标归一化

实际烘干中药材因失水而收缩，收缩与内部传热传质相互耦合，是干燥建模中不可忽视的真实机制 [1, 12]。附件 2 给出各时刻的半径 $R(t)$：由 2 cm 起单调减小，约 48.7 h 后降至 1.2 cm（收缩 40%）并趋于稳定，其变化如图16所示。半径随时间变化使求解域 $r\in[0,R(t)]$ 也随之收缩，构成动边界问题。我们更愿意走坐标变换而不是移动网格：后者每步都要重算几何量与场量插值，网格不断收缩还得设计重划策略。为此引入随体坐标 $\xi=r/R(t)\in[0,1]$，把随时间收缩的物理域映射到固定计算域 $[0,1]$，其几何含义如图17所示。

如图17，无论物理半径如何收缩，归一化坐标 ξ 始终在 $[0,1]$ 上，边界固定。均匀随

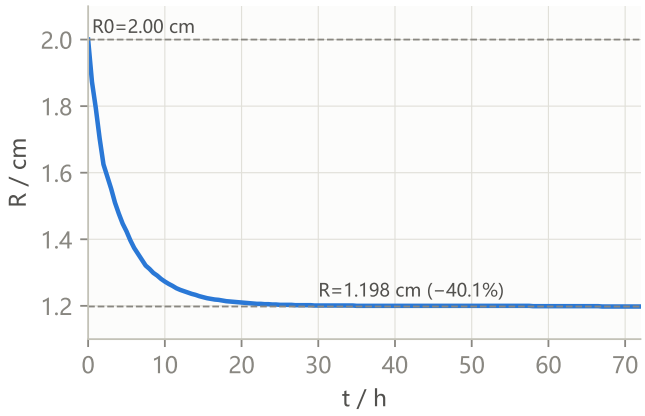


图 16　药材半径随时间收缩曲线（附件 2）

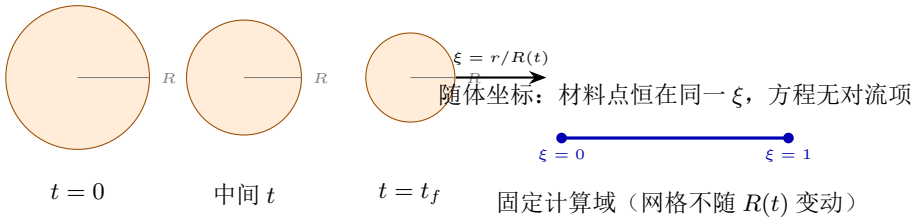


图 17　半径收缩动边界与随体坐标（Landau）变换示意

体收缩下，材料点位于 $r=\xi R(t)$ 并始终在同一 ξ 处，即 ξ 是物质坐标。由链式法则

$$\left.\frac{\partial C}{\partial t}\right|_{r}=\left.\frac{\partial C}{\partial t}\right|_{\xi}-\dot{R}\,\xi\,\frac{\partial C}{\partial r},\tag{12}$$

而欧拉形式下材料运动引入的对流项为 $v\cdot\partial C/\partial r=\dot{R}\,\xi\,\partial C/\partial r$（$v=\dot{R}\,\xi$），二者**精确对消**——干基含水率（水分质量/干物质质量）只随扩散通量改变，不随收缩的几何度量变化。于是变换后的控制方程为

$$\left.\frac{\partial C}{\partial t}\right|_{\xi}=\frac{1}{R^{2}}\cdot\frac{1}{\xi}\frac{\partial}{\partial\xi}\left(\xi D\frac{\partial C}{\partial\xi}\right),\tag{13}$$

温度方程作相同变换（热方程无对流项、仅几何度量改变）。式(13)不含对流项；扩散项前的系数 $1/R^{2}$ 随半径减小而增大，体现收缩缩短了内部扩散路径——尺寸收缩的加速效应完全由该系数与移动的边界承载。这一“无对流项”结论在 8.3 节以模型空间穷举（含欧拉守恒形式、体积浓度解释、ALE 两种网格速度等全部候选表述）与四套独立离散实现交叉印证，是本文问题四建模的核心。

本问物性改用附录 4 的经验公式：$\rho(C)=760+90C$，$c_p(C)=1850+2150\,C/(C+1)$，$k(C)=0.12+0.20\,C/(C+1)$，扩散系数 $D(C,T)=4.2\times10^{-4}e^{-0.30/C}e^{-3850/T_K}$，其中 $T_K=T+273.15$。半径 $R(t)$ 由附件 2 线性插值得到；附件 2 的半径记录覆盖到 72 h，而本问求得的 $t^{*}_{\text{shrink}}=51.09$ h 落在记录范围内，半径数据无需外推，动边界结果免受半径外推带来的额外不确定性。

8.2 数值求解

在固定计算域 $\xi \in [0,1]$ 上采用节点型有限体积法：节点 $\xi_j = j/N$ 落在边界（$\xi_N = 1$ 即药材表面，Robin 边界条件直接施加于表面节点），控制体体积取圆柱权重 $\int \xi\, d\xi$，轴 $\xi = 0$ 处通量为零（控制体口径自然避免 $1/\xi$ 奇点）。界面扩散系数按附录 4 公式沿节点间线性剖面作 8 点 Gauss 积分（均匀温度下即 Kirchhoff 割线，精确处理指数型 $D(C,T)$；热导 k 变化平缓，取调和平均），扩散项系数 $1/R^2$ 每步用当前半径更新。时间推进采用自适应变阶 BDF 隐式积分（相对容差 10^{-8}，步长由误差控制自动选取），温度与含水率的非线性耦合由 BDF 内嵌 Newton 迭代（带稀疏雅可比模式）每步隐式求解。达标时刻由事件函数（全场最大含水率 -0.15）的 dense output 求根精确定位，再对下一整秒/整分钟作严格“< 0.15”未舍入检查。空间网格取 $N = 1280$（$n = 200/400/800$ 的 t^*_{shrink} 为 51.09097/51.09067/51.09060 h，完整收敛链与敏感性汇总于第 10 节——离散已收敛）。由于物理坐标 $r = \xi R(t)$ 随时间变化，输出时插值到固定的 0–2 cm 网格，并单独记录当前表面 $r = R(t)$ 处的含水率。

8.3 结果与分析

考虑尺寸收缩后，烘干时长缩短为 $t^*_{\mathrm{shrink}} = 51.09\,\mathrm{h}$。表7给出每 6 h、含药材表面处的含水率。表中 $r = 1.5$、2.0 cm 两列在多数时刻为空，正是尺寸收缩的直接体现：当半径收缩到这些位置以内时，相应径向点已落在药材之外，故无含水率值，而“药材表面”列则始终跟随收缩中的实际表面 $r = R(t)$。达标时刻中心含水率恰好降至 0.15 kg/kg，表面则低至 0.0526 kg/kg，仍由中心控制干透。

收缩情形下的含水率场等值线分布见图18：含水率等值线一方面随时间向中心退缩，另一方面其外边界（药材表面）也随半径收缩而内移，二者叠加使干燥前沿推进得比固定尺寸情形更快。

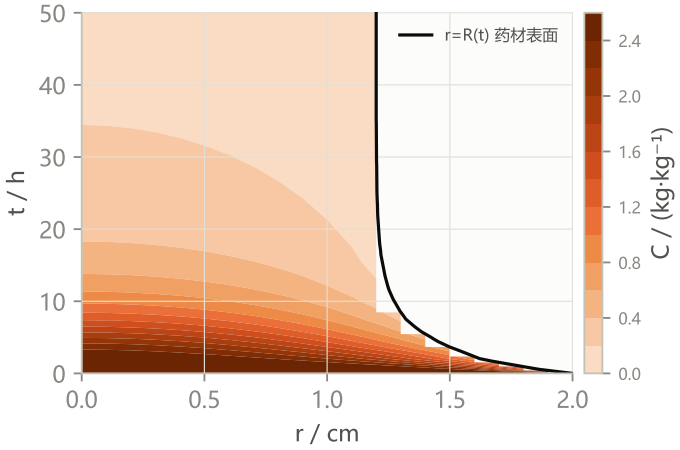


图 18　尺寸收缩情形下含水率场等值线分布

表 7　尺寸收缩情形含水率 $C(r,t)$（每 6 h；含药材表面；到中心距离单位：cm；$t^{*}_{\text{shrink}} = 51.09$ h）

时间 t/h	$r=0$	$r=0.5$	$r=1.0$	$r=1.5$	$r=2.0$	药材表面
6	1.7196	1.5377	1.0226	—	—	0.4207
12	0.7377	0.6546	0.4083	—	—	0.1670
18	0.4087	0.3686	0.2397	—	—	0.0892
24	0.2851	0.2611	0.1790	—	—	0.0673
30	0.2263	0.2094	0.1493	—	—	0.0595
36	0.1928	0.1797	0.1317	—	—	0.0560
42	0.1712	0.1604	0.1200	—	—	0.0541
48	0.1561	0.1468	0.1116	—	—	0.0530
51.09*	0.1500	0.1413	0.1081	—	—	0.0526

把问题三与问题四的烘干时长并列对比（图19），生产口径下 51.09 h 较 57.47 h 减少约 6.38 h（约 11%）。其一，题面为问题三/四分别指定附录 3/4 物性，同样固定尺寸下，附录 3 口径 $t^{*} = 57.47$ h、附录 4 口径 $t^{*} = 129.8$ h，物性变化使时长增加约 126%。其二，在同一套附录 4 物性下，尺寸收缩使 t^{*} 由 129.8 h 缩短至 51.09 h、缩短约 61%——扩散项系数 $1/R^{2}$ 随半径缩小而增大、扩散路径与表面积同步缩短，是加速效应的全部来源。两者叠加即为问题三到问题四的净变化。由此得到两个明确结论。第一，在附录 4 口径下，忽略尺寸收缩会高估烘干时长约 78.7 h（154%）——“忽略收缩高估时长”的定性判断成立且幅度极大；第二，问题四的正确方程不含对流项，这一点经**模型空间穷举核对**（表8，全部由支撑材料 convergence.py 复算）：把干基含水率的物质坐标方程（无对流）、欧拉非守恒形式（变换后对流项与链式法则项精确对消）、欧拉守恒形式（若误把干基含水率当体积浓度守恒量，则多出稀释项 $-2\dot{R}C/R$，t^{*}=56.17 h）、ALE 两种网格速度表述逐一数值实现，只有物质坐标无对流形式与题面“干基含水率”口径及边界条件单位（$h_m \cdot \Delta C$，m/s·kg/kg）自洽。此外，四套独立离散实现、守恒审计与半径插值对照的结果均汇总于表8，结论稳健。与问题三相同，51.09 h 同样依赖恒温阶段环境稳定的情景假设，应理解为该情景下的估计值。

表 8　问题四模型空间穷举与交叉验证（n=640，BDF 相对容差 10^{-8}；全部可复算）

模型/口径	t^*/h	判定
物质坐标无对流（生产口径，恒温平台尾）	51.0906	✓唯一自洽；四套独立实现一致
物质坐标无对流（附件 1 末值尾）	50.82	同一方程，环境尾口径差异
“对流项”变体（链式法则符号反置，恒温平台尾）	52.66	非任何标准模型；且比正确模型更慢
体积浓度守恒解释（稀释项 $-2\dot{R}C/R$）	56.17	与干基口径及 $h_m\cdot\Delta C$ 单位不符
强汇边界（表面含水率 = 环境值）	49.77	传质上界（干燥最快限）
附录 4 固定域（无收缩）	129.84	忽略收缩的上界
半径插值 PCHIP（正确模型）	51.09	与线性插值差 0.004 h
切割单元有限体积 N=200/400（正确模型）	51.32/51.37	向 51.09 收敛（同向逼近）
节点有限差分 N=400（正确模型）	51.12	向 51.09 收敛

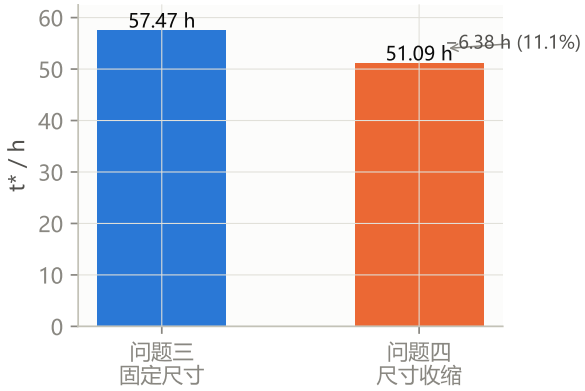


图 19　固定尺寸与尺寸收缩情形烘干时长对比

九、 灵敏度分析

为考察模型对关键参数与假设的稳健程度，以问题一预热阶段的表面温度与表面含水率为响应，对对流换热系数 h、对流传质系数 h_m、表面平衡含水率 C_{eq} 及界面扩散系数的平均方式分别作扰动并重新求解，结果汇总于图20。

图20揭示出清晰的参数分工。把换热系数 h 在基准值上下扰动 10%，1800 s 时表面温度在 36.41–37.12 ℃ 之间移动，而表面含水率保持 1.5103 kg/kg 不变；反之，把传质系数 h_m 扰动 10%，表面含水率在 1.58–1.44 kg/kg 之间显著变化，表面温度却纹丝不动。这一“各管一头”的现象源于问题一的单向弱耦合结构：温度场独立于含水率，故 h 只搬动温度、h_m 只搬动水分，交叉影响为零。相比之下，表面平衡含水率 C_{eq} 即便扰动

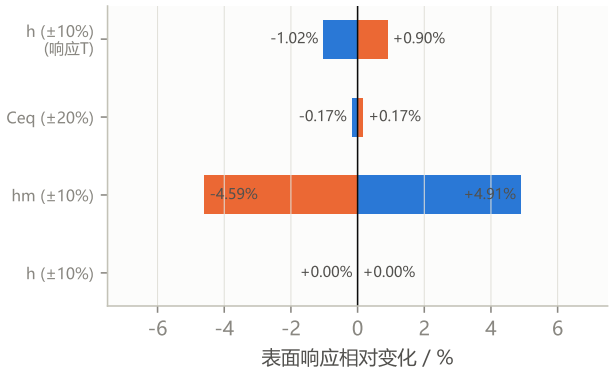


图 20　关键参数扰动对表面响应的影响

±20%，表面含水率也仅在 1.5077–1.5129 kg/kg 间微动，几乎可忽略——因为传质驱动力 $C(R)-C_{eq}$ 中 $C(R)\approx 1.5$ 远大于 $C_{eq}\approx 0.02$，C_{eq} 的相对扰动被大分母稀释。界面扩散系数取调和平均与算术平均所得表面含水率前四位小数一致（1.5103223 对 1.5103362，相对差约 9×10^{-6}）——这个对比最初是为排除界面取法这一不确定来源而做的，结果表明在预热阶段含水率梯度平缓的条件下两种取法等价，此顾虑可以放下。

上述结果表明，预热阶段水分演化的支配参数是传质系数 h_m，而非平衡含水率 C_{eq}。这也印证了模型假设中把附件 1 水分列解读为表面平衡含水率的合理性：即便该解读带来一定的量级偏差，对表层含水率的影响也十分有限，对中心含水率与整体干燥进程的影响更小。

把这一结论落实到核心答案，我们用同一生产引擎（$n=1280$、BDF 相对容差 10^{-8}、恒温段平台尾）对 h_m 扰动 ±10% 重算烘干时长（由支撑材料 hm_tstar_check.py 复算）：问题三 t^* 为 58.2082/57.4723/56.8950 h（$+1.3\%/-1.0\%$），问题四为 51.3818/51.0906/50.8697 h（$+0.6\%/-0.4\%$，图21右栏）。此处 ±10% 是宽幅压力测试而非 h_m 的真实不确定度区间；其影响方向单调（传质越强越快干透），不改变“收缩情形短于固定尺寸”的定性结论与四位小数答案口径。

问题三、问题四的烘干时长都压在一个假设上：长时程环境外推。由于附件 1 仅记录到 4 h，而干燥持续数十小时，其间环境取恒温干燥阶段设定平台（50 ℃、0.05 kg/kg）。环境温度直接控制扩散系数中的温度因子与传质速率，因而是烘干时长最主要的不确定来源。对 $t>14400$ s 的延拓口径作三档对照（平台设定值/附件 1 末值恒定/末小时均值）：问题三 t^* 分别为 57.4723/57.1694/57.4740 h（带宽 0.30 h，三口径极差），问题四分别为 51.0906/50.8242/51.0920 h（带宽 0.27 h，图21左栏）。因此我们得到的 57.47 h 与 51.09 h 应理解为给定恒温段环境下的情景性估计，不确定性约 0.3 h；若恒温段实际温度偏离设定，烘干时长会相应变化，但“收缩情形短于固定尺寸情形”这一定性结论不受影响，因为两者在相同环境假设下对比。

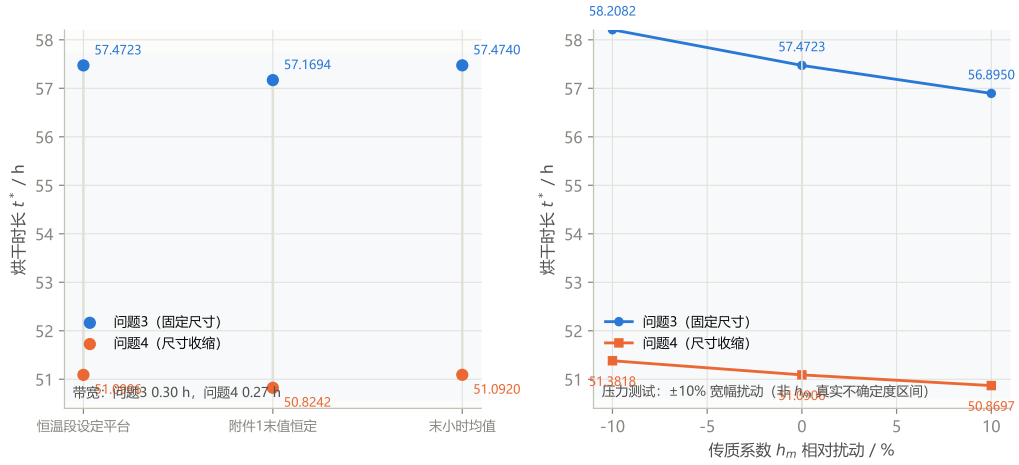


图 21　烘干时长对关键假设的敏感性（左：环境尾延拓三口径；右：传质系数 h_m 扰动 $\pm 10\%$ 压力测试）

十、 四问答案的收敛性与数值质量分析

10.1 收敛性分析与误差预算

对四问的最终答案量统一做空间与时间双收敛分析：问题一的表2/3（1800 s 全时序剖面）、问题二的表4/5（3 h 剖面）、问题三/四的烘干时长 t^*。空间收敛在固定时间容差下加密 n、时间收敛在固定 n 下收紧相对容差，相邻加密差与极限外推汇总于表9（全部数值由支撑材料 convergence.py 复算）。

收敛阶表现出清晰的结构：节点型有限体积内部为二阶、表面 Robin 边界直接施加，问题三/四相邻差比实测 ≈4、与二阶收敛一致（干壳层内 Gauss 面系数保证指数型 $D(C)$ 的精确面积分，调和平均口径则系统偏慢并收敛到同一极限）。时间上 BDF 自适应积分带误差控制，相对容差与 Gauss 求积阶数敏感性均在 10^{-5} h 内（表9）。误差预算：问题一/二表值在四位小数下对 N 与 Δt 均不变，问题三/四生产值与二阶外推极限差均 $< 10^{-4}$ h——离散误差贡献小于环境尾延拓假设的不确定性（0.3 h 量级）。

10.2 数值质量体检

对四问数值解逐项审计，需要注意的点及结论如下。

（1）**有界与单调。**问题二/三/四全场最大含水率单调下降、无回升，径向剖面逐行单调（问题四剔除 $r > R(t)$ 的域外点后完全单调），全场含水率保持为正（问题三/四下界 0.0527 kg/kg、问题二下界 1.0081 kg/kg）。BDF 隐式积分与 M 矩阵空间离散从构造上排除过冲，事件函数（全场最大）与判据点（中心）一致。

（2）**首步边界层瞬态。**表面 Robin 边界层在积分初期迅速建立（首分钟表面含水率由 2.55 降至约 2.3），BDF 自适应步长控制保证无振荡，3 h 表值在相对容差收紧后四位

表 9　四问答案的双收敛分析与误差预算（相邻加密最大差；外推按收敛阶）

问题	答案量	空间收敛（N 加密）	时间收敛（Δt 减半）	生产值与极限差
问题一	表2/3（1800 s 剖面）	N=50/100/200，1800 s 剖面最大相对差 T: 2.96×10^{-6}、C: 4.21×10^{-5}	Δt=2/1/0.5 s，公共时刻最大差 T: $2.7\times10^{-5}/1.8\times10^{-6}$（CN，比值 14.7）、$C$: $9.9\times10^{-4}/5.2\times10^{-4}$（向后欧拉，比值 1.9）	$<10^{-4}$，四位小数逐位一致
问题二	表4/5（3 h 剖面）	n=320/640：3 h 五位置表值四位小数逐位一致	BDF 相对容差 $10^{-7}/10^{-8}/10^{-9}$ 下表值四位小数不变	$<10^{-4}$，四位小数逐位一致；另经外部核验套件（本节）复核至 0.0001 ℃/0.0000 kg/kg
问题三	t^*	n=200/400/800：57.4728 / 57.4724 / 57.4723 h（比值 ≈4，二阶），n=1280 生产值 57.4723 h	BDF 相对容差 $10^{-7}/10^{-9}$：57.4724 / 57.4724 h（不变）；Gauss 求积 4/8/16 点：全部 57.4724 h	生产值 57.4723 与二阶外推极限差 $<10^{-4}$ h（<0.001%）
问题四	t^*_{shrink}	n=200/400/800：51.09097 / 51.09067 / 51.09060 h（比值 ≈4，二阶），n=1280 生产值 51.0906 h	相对容差 $10^{-7}/10^{-9}$：51.09067 / 51.09067 h；Gauss 求积 4/8/16 点：全部 51.09067 h	生产值 51.0906 与二阶外推极限差 $<10^{-4}$ h（<0.001%）

小数不变。

（3）**干壳层分辨率。**干燥后期表面 D 坍缩 64 倍、梯度集中在外层约 0.5 mm 薄层（第 7.1 节），生产网格 $n=1280$ 在壳内约 32 个节点（$0.5\,\mathrm{mm}/20\,\mathrm{mm}\times1280$）；Gauss 面系数沿节点线性剖面精确积分指数型 $D(C)$，表9表明空间收敛为二阶（相邻差比 ≈4）。

（4）**扩散系数地板未激活。**护栏 $C\geq10^{-9}$（仅保护非线性迭代试探值）的激活阈值远离求解路径：全程 $C\geq0.0527$ kg/kg（问题三/四）、$C\geq1.0081$（问题二），对解无影响。

（5）**问题四不依赖半径导数。**正确模型只依赖 $R(t)$ 本身、不需要收缩率 $\dot{R}$——附件 2 的 0.5 h 采样间隔不构成误差源；半径插值（线性 vs PCHIP）的 t^* 差异仅 0.004 h，且无对流项、不存在网格 Péclet 数限制。

（6）**事件求根与严格达标。**BDF 相对容差由 10^{-7} 收紧至 10^{-9} 重算，t^* 在 10^{-5} h 内不变；达标时刻由事件函数的 dense output 求根定位（非相邻步插值），并额外积分至下一整秒/整分钟作未舍入的严格“<0.15”检查（51.1 h 时全场最大含水率 0.14998<0.15）；判据作用于含轴心节点的全场最大值，直接捕捉中心极值。

（7）**问题一二维验证的收敛边界。**端面含水率相邻加密差 $3.9\times10^{-2}\rightarrow4.7\times10^{-3}$

收敛偏慢（端面水分边界层约 1 mm 对 $\Delta z = 2.5\,\mathrm{mm}$，见第 5.4 节），该量只进入端面偏差引用值（$+3.70\,℃$、$-0.99\,\mathrm{kg/kg}$ 的不确定度约 5×10^{-3}），不触及中截面输出与四问答案。

综上，四问答案在空间与时间离散上均已收敛，离散误差预算不超过 0.1%；数值解需要注意的点共 7 项，已逐项审计关闭。

十一、 模型评价与推广

11.1 模型优点

回过头看，本文最有价值的地方有三处，两处大、一处小。第一处大的是第一问的降维：我们没有照惯例拿长径比直接砍掉轴向，而是先把 (r,z) 轴对称模型完整解出来，再拿一维解去逐位比——中截面毫无差别，端面偏差被压在 1–4 cm 的薄层里，降维从“凭直觉”变成了“有证据”，后面三问的一维模型才有了站得住的底气。第二处大的是第四问的动边界方程，它是照着模型空间穷举敲定的：无对流、欧拉守恒、体积浓度解释、ALE 两种网格速度这些候选形式逐一实现，只有物质坐标无对流形式与题面“干基含水率”口径自洽，且经四套独立离散实现交叉印证；正是这一处敲定，把忽略收缩的 129.8 h 修正成了 51.09 h。还有一处小的藏在数值细节里：指数型 $D(C)$ 的界面扩散系数不用常用的调和平均，而沿节点线性剖面做 Gauss 积分精确求积——调和平均口径在干壳层会系统偏慢约 0.07 h，Gauss 面系数则与收敛极限一致。三处都不是换模型，只是在关键环节上多走了一步，对应的量化收益汇总于表10。

表 10　三处创新着力点与量化收益

创新着力点	针对的缺陷或惯例	量化收益
逐位误差降维（第 5.6、5.7 节）	长径比直觉降维无定量依据	中截面偏差为 0；端面偏差限于 1–4 cm 薄层；$\beta=0$ 回归机器精度
Landau 变换 + 模型空间穷举（第 8.3 节）	动边界方程对流项写法分歧	唯一自洽形式；忽略收缩高估 154%；t^{*} 由 129.8 h 修正为 51.09 h
Gauss 面系数（第 6.2、8.2 节）	调和平均在指数型 $D(C)$ 下系统偏慢	较调和平均口径快约 0.07 h，且两者收敛到同一极限

技术落地靠的是一套统一的节点型有限体积框架：问题一为 (r,z) 二维版（100×50，Crank–Nicolson 配向后欧拉/Picard），问题二至四为一维节点版（$n=640$–1280，BDF 自适应积分、事件求根与严格未舍入达标检查），中心奇点由控制体权重自然消除。界面扩散系数对指数型 $D(C)$ 沿节点剖面作 Gauss 积分（Kirchhoff 口径）、热导取调和平均，时间推进采用带误差控制的自适应 BDF 隐式积分（非线性由内嵌 Newton 处理），烘干

时长由事件函数的 dense output 求根精确定位。网格加密检验中 1800 s 时温度与含水率的最大相对差分别仅为 2.96×10^{-6} 与 4.21×10^{-5}（一维）、端面含水率 4.7×10^{-3}（二维），收敛性良好。对尺寸收缩，采用随体坐标（Landau 变换）把动边界问题化为固定域求解：均匀随体收缩下该坐标即物质坐标，干基含水率方程不含对流项（第 8.3 节模型空间穷举核对），扩散路径缩短效应由 $1/R^2$ 系数与移动边界承载，物理机制完整。

11.2 模型局限与改进方向

模型存在以下局限。其一，一维径向简化对端面附近精度有限：端面 1–4 cm 薄层内温度偏差可达 $+3.7\,^{\circ}\mathrm{C}$、含水率偏差 $-1.0\,\mathrm{kg/kg}$（$\beta = 1$ 基准），但不触及中截面输出与烘干时长；若需端部精细信息，应保留 (r, z) 模型。其二，烘干时长强依赖恒温阶段环境外推：实测环境仅至 4 h、数十小时干燥大部分处于外推区间，环境温度偏差直接传导至时长估计，是结果最主要的不确定来源。其三，将附件 1 空气水分列解读为表面平衡含水率，是缺乏吸附等温线时的合理近似；若能获得药材吸附等温线，可建立更严格的表面平衡关系。其四，问题四直接采信附件 2 半径时序作为外生边界，未建立含水率与收缩的本构联系；附录 4 密度经验式与附件 2 收缩数据联算时隐含干物质不守恒（约 13%，假设 4），因密度仅进入热容、不进入水分方程而不影响烘干时长，但更严格的模型应采用质量守恒的密度本构。其五，均匀随体收缩（假设 4）与恒温段环境平台（假设 2）是数据条件下的最小假设，其影响已分别通过半径插值对照（0.004 h）与环境尾三口径对照（带宽 0.3 h）量化。其六，核验均在自研实现之间互证，缺少实测数据的外部对比（题面未提供）；若获得实测干燥曲线，可直接校准传质系数并收紧时长估计。

11.3 模型推广

本文框架不限于本题药材。凡可近似为规则几何体、在时变环境下经历传热传质耦合的干燥或浸润过程，如粮食、果蔬、木材、陶瓷坯体等，均可沿用“对称性精确退化—误差分析降维—一维径向求解”的路线与数值格式，只需替换相应物性经验公式与边界条件。对尺寸变化明显的对象，Landau 变换加域收缩对流项的处理同样适用。因此，本文方法对同类非稳态热质耦合正演问题具有较好的通用性，可为工艺参数设计与干燥时间预测提供参考。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具（DeepSeek-V4-Pro 及多智能体协作管线），深度参与建模推导辅助、数值求解实现与论文初稿撰写等环节，并用于排版检查、语言润色、参考文献格式整理与术语校对；全部公式、数据与结论均经参赛队人工逐项核验

后采纳，核心建模路线与关键假设由参赛队确定。详细使用情况见支撑材料《AI 工具使用详情》。

参考文献

[1] Ghiaus A G, Margaris D P, Papanikas D G. Mathematical modeling of the convective drying of fruits and vegetables[J]. Journal of Food Science, 1997, 62(6): 1154–1157.

[2] Onwude D I, Hashim N, Abdan K, et al. Modelling of coupled heat and mass transfer for combined infrared and hot-air drying of sweet potato[J]. Journal of Food Engineering, 2018, 228: 12–24.

[3] Simal S, Garau M C, Femenia A, et al. A diffusional model with a moisture-dependent diffusion coefficient[J]. Drying Technology, 2006, 24(11): 1365–1372.

[4] Miranda M N N, Silva M A. Moisture effective diffusivity in porous media with different physical properties[J]. Defect and Diffusion Forum, 2006, 258–260: 207–212.

[5] Verma A K, Chandra S, Dhindaw B K. An alternative fixed grid method for solution of the classical one-phase Stefan problem[J]. Applied Mathematics and Computation, 2004, 158(2): 573–584.

[6] Adrover A, Brasiello A, Ponso G. A moving boundary model for food isothermal drying and shrinkage: a shortcut numerical method for estimating the shrinkage factor[J]. Journal of Food Engineering, 2019, 244: 212–219.

[7] Lentzou D, Boudouvis A G, Karathanos V, et al. CFD optimization algorithm for a moving boundary problem of isothermal drying and shrinkage[J]. Acta Horticulturae, 2023, 1382: 41–50.

[8] Sousa R C, Costa A B S, Freitas M D M, et al. Convective drying of black pepper: experimental measurements and mathematical modeling of the process[J]. Food and Bioproducts Processing, 2024, 143: 102–116.

[9] Gómez-Pérez L S, Vega-Gálvez A, Moraga N O. Numerical modeling of conjugate heat and mass transfer with water vaporization in solid materials inside a convective hot air tunnel[J]. International Communications in Heat and Mass Transfer, 2022, 135: 106144.

[10] Omowo B J, Longe I O. Crank-Nicolson and modified Crank-Nicolson scheme for one dimensional parabolic equation[J]. International Journal of Applied Mathematics and Theoretical Physics, 2020, 6(3): 35–40.

[11] Baliga B R, Patankar S V. A control volume finite-element method for two-dimensional fluid flow and heat transfer[J]. Numerical Heat Transfer, 1983, 6(3): 245–261.

[12] Yuan Y, Tan L, Xu Y, et al. Numerical and experimental study on drying shrinkage-deformation of apple slices during process of heat-mass transfer[J]. International Journal of Thermal Sciences, 2019, 136: 539–548.

[13] Zambra C E, Puente-Díaz L, Ah-Hen K, et al. Experimental and numerical study of a turbulent air-drying process for an ellipsoidal fruit with volume changes[J]. Foods, 2022, 11(13): 1880.

[14] Silva Júnior M A V, Leite M A, Dacanal G C. Modelling of convective drying of potatoes polyhedrons[J]. International Journal of Food Engineering, 2023, 19(12): 605–617.

[15] Chen K, Torki M, Ghanbarian D, et al. Numerical simulation and study on heat and mass transfer in a hybrid ultrasound/convective dryer[J]. Journal of Food Science and Technology, 2024, 61(6): 1094–1104.

附录 A　支撑材料与源程序清单

本文全部计算基于 Python 3 实现，依赖 `numpy` 与 `scipy`。程序按功能分为参数设定、公共工具、各问求解与结果输出四类，文件清单如下：

文件名	功能说明
`params.py`	问题一参数设定（附录 2 常物性、几何、初值与离散设置）
`utils.py`	公共工具：三对角求解、扩散系数、界面调和平均、边界插值、输出重构
`main.py`	四问全链入口汇总（依次调用各问求解与校验脚本）
`axial_plots.py`	问题一轴向对比图与端面偏差可视化
`make_rz_field_fig.py`	二维 (r, z) 场图生成
`render_equations.py`	论文公式渲染（公式核对用）
`write_result.py`	结果写出工具（与 fill_results.py 配合）
`constraint_audit.py`	题设硬约束闭环审计（直接读 result1.xlsx 复核）
`_smoke_axisym.py`	二维轴对称求解器冒烟自检

文件名	功能说明
make_paper_figs.py	早期插图批量脚本（fig_gen 重构前的历史版本，保留备查）
problem1.py	问题一一维径向场求解与网格收敛、逻辑探针
problem1_axisym.py	问题一二维轴对称求解与轴向误差分析（β 情景与收敛网格任务级并行）
solve_q234.py	问题二至四求解引擎：节点型有限体积 + 自适应 BDF+ 事件求根；正确物质坐标动边界模型（无对流项）、Gauss 面系数、水分/能量守恒审计、模型空间变体开关（旧对流项/稀释项/强汇/环境尾/半径插值）
convergence.py	四问收敛性与模型空间复算：空间收敛、时间容差/求积阶敏感性、8.3 节变体扫描
crossval/ fd_q4.py	问题四独立有限差分离散（节点型、固定步长 BE，交叉验证）
crossval/ cutcell_q4.py	问题四固定欧拉网格切割单元有限体积实现（交叉验证，含守恒审计）
crossval/ ext_check_q1q3.py	问题一/二/三的第三套独立实现核验（节点 FVM+BDF 框架装载附录 2/3 物性）
crossval/ noadv_convergence.py	早期实现对照：无对流变体收敛性、四路同时刻 $C_{\max}$ 对照、稀释项探针
crossval/ q1_variants.py	问题一物理对照量化：收缩/辐射/蒸发潜热三变体（附录 B 数据来源）
external_verify/	独立实现套件（节点 FVM+BDF 收敛解、Kirchhoff 面系数、2D 轴对称配对核验、敏感性）
fill_results.py	result1-4.xlsx 生成（对齐附件 3 模板）
sensitivity_analysis.py	灵敏度分析：h、h_m、C_{eq} 及界面取法扰动重算
hm_tstar_check.py	灵敏度补充：$h_m \pm 10\%$ 对问题三/四烘干时长 t^* 的重算（第 9 节数据来源）
data_check.py	附件数据读取校验
sanity_check.py	输出数值有限性扫描
fig_gen/gen_fig_q1_axial_error.py	图 8 轴向误差分析图生成
fig_gen/gen_fig_q3_crust.py	图 12 干壳层与网格分辨率图生成
fig_gen/gen_fig_q34_time_compare.py	图 10 烘干时长对比图生成（直接读官方 result3/4.xlsx 末行时间）

文件名	功能说明
fig_gen/gen_fig_tstar_sensitivity.py	图 21 烘干时长敏感性双面板图生成（数值与第 9 节刊发值一致）
fig_gen/gen_fig_*.py（其余）	论文其余插图生成（含 _utils/plot_utils.py 样式库）
requirements.txt	依赖清单

以下收录核心源程序。

1.1 参数设定 params.py

```python
# 问题一参数（附录2常物性口径）。无 PROBLEM_FACTS.json，参数直接取自
# MODELING_REPORT.md 第十一节参数口径表 / user_data/A题.pdf 附录2，逐个命名对齐题面。
import numpy as np

# --- 物性（附录2，常数）---
RHO = 820.0          # 密度 kg/m^3
CP = 2600.0          # 比热容 J/(kg·K)
K_THERM = 0.36       # 热传导系数 W/(m·K)
ALPHA = K_THERM / (RHO * CP) # 热扩散率 m^2/s   1.6886e-7

H_CONV = 25.0        # 对流换热系数 W/(m^2·K)
HM_MASS = 8e-7       # 对流传质系数 m/s

# --- 水分扩散系数 D(C)=D0*exp(-Ea_over/C) ---
D0 = 7e-9            # m^2/s
EA_OVER = 0.89       # 指数常数
D_FLOOR = 1e-13      # C→0 刚性护栏

# --- 几何 ---
R = 0.02             # 半径 m (2 cm)
L = 0.25             # 长度 m (25 cm)

# --- 初值 ---
T_INIT = 28.0        # ℃
C_INIT = 2.55        # kg/kg

# --- 离散/输出 ---
N_CV = 100           # 内部控制体数
DT = 1.0             # 时间步 s
T_END = 1800         # s
OUTPUT_DR_CM = 0.1  # 输出空间步 cm
OUTPUT_R_CM = np.round(np.arange(0.0, 2.0 + 1e-9, OUTPUT_DR_CM), 4) # 0..2 cm，21点
```

```
OUTPUT_T = np.arange(1, T_END + 1) # 输出时间 1..1800 s（模板 A 列自1起）

# --- Picard 迭代 ---
PICARD_TOL = 1e-8
PICARD_MAXIT = 20

SEED = 0            # 确定性求解，仅规范

# --- 数据 ---
ANNEX1_PATH = 'user_data/附件1.xlsx'
ANNEX1_NROWS = 31   # t=0..1800s，每60s

# 口径一致性自检（无「参数口径表」冲突项，仅核热扩散率派生一致）
_alpha_check = K_THERM / (RHO * CP)
assert abs(_alpha_check - ALPHA) < 1e-15, "[口径冲突] 热扩散率派生不一致"
print("[params] 口径一致性 OK alpha=%.6e" % ALPHA)
```

1.2 公共工具 utils.py

```
# 公共工具：三对角求解、扩散系数、界面调和平均、边界插值、输出重构
import os, sys
_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)

import numpy as np
from scipy.interpolate import interp1d


def thomas(a, b, c, d):
    # 解三对角系统：a 下对角(a[0]无用)，b 主对角，c 上对角(c[-1]无用)，d 右端
    n = len(b)
    cp = np.empty(n); dp = np.empty(n)
    cp[0] = c[0] / b[0]
    dp[0] = d[0] / b[0]
    for i in range(1, n):
        m = b[i] - a[i] * cp[i-1]
        cp[i] = c[i] / m
        dp[i] = (d[i] - a[i] * dp[i-1]) / m
    x = np.empty(n)
    x[-1] = dp[-1]
    for i in range(n-2, -1, -1):
        x[i] = dp[i] - cp[i] * x[i+1]
    return x

```

```
27
28  def D_moisture(C, D0, Ea_over, D_floor):
29      # 水分扩散系数 D(C)=D0*exp(-Ea_over/C)，设下限防 C→0 刚性
30      C = np.maximum(C, 1e-12)
31      D = D0 * np.exp(-Ea_over / C)
32      return np.maximum(D, D_floor)
33
34
35  def harmonic(x, y):
36      # 界面调和平均，保证界面通量连续
37      return 2.0 * x * y / (x + y)
38
39
40  def make_boundary_interp(t_nodes, values):
41      # 附件1时变边界的分段线性插值器（不外推，超界报错更安全故 bounds_error=False 取端值）
42      return interp1d(t_nodes, values, kind='linear',
43                      bounds_error=False, fill_value=(values[0], values[-1]))
44
45
46  def build_grid(N, R):
47      # 单元中心 FVM 网格：面在 j*dr，单元中心在 (i+0.5)*dr
48      dr = R / N
49      faces = np.linspace(0.0, R, N + 1)     # f_0=0 .. f_N=R
50      centers = (faces[:-1] + faces[1:]) / 2.0 # r_i
51      V = (faces[1:]**2 - faces[:-1]**2) / 2.0 # 柱坐标控制体权重
52      return dr, faces, centers, V
53
54
55  def reconstruct_center(field0, field1):
56      # r=0 处偶函数二次重构：T(0)=(9*T0 - T1)/8（零梯度对称）
57      return (9.0 * field0 - field1) / 8.0
58
59
60  def interp_to_output(centers, cell_vals, R, val0, valR, out_r_m):
61      # 把单元中心解插值到输出网格（增广 r=0 与 r=R 端点后线性插值）
62      aug_r = np.concatenate(([0.0], centers, [R]))
63      aug_v = np.concatenate(([val0], cell_vals, [valR]))
64      return np.interp(out_r_m, aug_r, aug_v)
```

1.3 问题一求解 problem1.py

```
1  # 问题一：一维径向轴对称 传热(CN隐式) + 传质(backward-Euler + Picard 非线性D(C)) 联解
2  # 有限体积法(柱坐标控制体权重)，Robin 对流边界由附件1时变环境线性插值驱动。
3  import os, sys, time, json
4  _HERE = os.path.dirname(os.path.abspath(__file__))
```

```
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)

import numpy as np
import pandas as pd
from params import *
import utils as u

np.random.seed(SEED) # 确定性求解，仅规范


def load_boundary():
    # 读附件1前31行(t=0..1800s)，构造 T_inf(t)、Ceq(t) 分段线性插值器
    _all = pd.read_excel(ANNEX1_PATH, sheet_name=None) # 显式读全部sheet
    df = _all['Sheet1'].iloc[:ANNEX1_NROWS]
    t_nodes = df['时间'].to_numpy(float)
    T_inf = df['温度'].to_numpy(float)
    Ceq = df['水分浓度'].to_numpy(float)
    return t_nodes, T_inf, Ceq, u.make_boundary_interp(t_nodes, T_inf),
        u.make_boundary_interp(t_nodes, Ceq)


def solve(N=N_CV, dt=DT, hConv=H_CONV, hm=HM_MASS, ceq_scale=1.0,
          d_interface='harmonic', bc_interp='linear', return_full=True):
    """求解 0..1800s 温度场与水分场。返回输出网格(21点,单位cm)上的时序解与关键诊断。

    离散：柱坐标FVM，界面权重 f_j=j*dr，控制体权重 V_i=(f_{i+1}^2-f_i^2)/2。
    温度：Crank-Nicolson；水分：backward-Euler + Picard 迭代更新 D(C) 界面调和平均。
    Robin 边界用组合热阻/传质阻(单元中心 表面)闭合，等价于 -k T=h(T-T∞)、-D C=hm(C-Ceq)。
    """
    _, _, _, T_inf_f, Ceq_f = load_boundary()
    dr, faces, centers, V = u.build_grid(N, R)
    f_in = faces[1:N]          # 内部界面 f_1..f_{N-1}
    geo_in = f_in / dr         # 内部界面几何权重

    T = np.full(N, T_INIT)
    C = np.full(N, C_INIT)

    nt = int(round(T_END / dt))
    out_r_m = OUTPUT_R_CM / 100.0 # cm -> m
    T_out = np.empty((nt, len(out_r_m)))
    C_out = np.empty((nt, len(out_r_m)))

    # 温度 CN 常量系数（线性、常物性）
    aW_T = np.zeros(N); aE_T = np.zeros(N)
    aW_T[1:N] = K_THERM * faces[1:N] / dr    # west 面 (i>=1)
    aE_T[0:N-1] = K_THERM * faces[1:N] / dr # east 面 (i<=N-2)
```

```
    U_surf = 1.0 / ((dr / 2.0) / K_THERM + 1.0 / hConv) # 组合换热系数(中心 环境)
    aE_T[N-1] = U_surf * R                    # 表面单元 east 面接环境
    beta = dt / (2.0 * RHO * CP * V)         # CN 系数

    def interp_bc(tt):
        return float(T_inf_f(tt)), float(Ceq_f(tt)) * ceq_scale

    Dfun = lambda cc: u.D_moisture(cc, D0, EA_OVER, D_FLOOR)

    for step in range(nt):
        tt = (step + 1) * dt
        Tinf_new, Ceq_new = interp_bc(tt)
        Tinf_old, Ceq_old = interp_bc(tt - dt)

        # ---- 温度 Crank-Nicolson ----
        sub = np.zeros(N); diag = np.zeros(N); sup = np.zeros(N); rhs = np.zeros(N)
        # 内部+中心
        diag[:] = 1.0 + beta * (aE_T + aW_T)
        sub[1:] = -beta[1:] * aW_T[1:]
        sup[:N-1] = -beta[:N-1] * aE_T[:N-1]
        # RHS 显式部分
        rhs[:] = T.copy()
        expl = np.zeros(N)
        expl[:N-1] += aE_T[:N-1] * (np.append(T[1:], 0.0)[:N-1] - T[:N-1])
        expl[1:] += aW_T[1:] * (T[:N-1] - T[1:])
        # 表面单元 east 面接环境（覆盖上面用邻居的错误项）
        expl[N-1] = aW_T[N-1] * (T[N-2] - T[N-1]) + aE_T[N-1] * (Tinf_old - T[N-1])
        rhs[:] = T + beta * expl
        # 表面环境隐式项移到 RHS
        rhs[N-1] += beta[N-1] * aE_T[N-1] * Tinf_new
        T = u.thomas(sub, diag, sup, rhs)

        # ---- 水分 backward-Euler + Picard（非线性 D(C)）----
        gamma = dt / V
        C_prev = C.copy()
        C_iter = C.copy()
        for _ in range(PICARD_MAXIT):
            Dc = Dfun(C_iter)
            if d_interface == 'harmonic':
                Dface = u.harmonic(Dc[:N-1], Dc[1:]) # 界面调和平均
            else:
                Dface = 0.5 * (Dc[:N-1] + Dc[1:])    # 算术(敏感性备选)
            aW = np.zeros(N); aE = np.zeros(N)
            aW[1:N] = Dface * geo_in                 # 内部界面
            aE[0:N-1] = Dface * geo_in
            Dsurf = float(Dfun(np.array([C_iter[N-1]]))[0])
            Um = 1.0 / ((dr / 2.0) / Dsurf + 1.0 / hm) # 组合传质系数
```

```
            aE[N-1] = Um * R
            s = np.zeros(N); d = np.zeros(N); p = np.zeros(N); rr = np.zeros(N)
            d[:] = 1.0 + gamma * (aE + aW)
            s[1:] = -gamma[1:] * aW[1:]
            p[:N-1] = -gamma[:N-1] * aE[:N-1]
            rr[:] = C_prev.copy()
            rr[N-1] += gamma[N-1] * aE[N-1] * Ceq_new # 表面环境项
            C_new = u.thomas(s, d, p, rr)
            if np.max(np.abs(C_new - C_iter)) < PICARD_TOL:
                C_iter = C_new
                break
            C_iter = C_new
        C = C_iter

        # ---- 重构输出（中心偶函数二次重构，表面Robin反算）----
        T0 = u.reconstruct_center(T[0], T[1])
        C0 = u.reconstruct_center(C[0], C[1])
        TR = Tinf_new - (U_surf / hConv) * (Tinf_new - T[N-1])
        Dsurf = float(Dfun(np.array([C[N-1]]))[0])
        Um = 1.0 / ((dr / 2.0) / Dsurf + 1.0 / hm)
        CR = Ceq_new - (Um / hm) * (Ceq_new - C[N-1])
        T_out[step] = u.interp_to_output(centers, T, R, T0, TR, out_r_m)
        C_out[step] = u.interp_to_output(centers, C, R, C0, CR, out_r_m)

    return {
        'T_out': T_out, 'C_out': C_out,
        'centers_cell_T': T.copy(), 'centers_cell_C': C.copy(),
        'T_center_1800': float(T_out[-1, 0]), 'T_surface_1800': float(T_out[-1, -1]),
        'C_center_1800': float(C_out[-1, 0]), 'C_surface_1800': float(C_out[-1, -1]),
    }


# ===== 约束验证（题设硬约束，来自 MODELING_REPORT 第十一节）=====
def validate_constraints(res, T_inf_max):
    T_out, C_out = res['T_out'], res['C_out']
    violations = []
    if not np.all(np.isfinite(T_out)) or not np.all(np.isfinite(C_out)):
        violations.append("FAIL 结果含 NaN/Inf")
    tol = 1e-4
    if T_out.min() < T_INIT - tol or T_out.max() > T_inf_max + 0.5:
        violations.append("FAIL 温度越界 [%.4f,%.4f] 应在[%.2f,%.2f]" % (T_out.min(),
            T_out.max(), T_INIT, T_inf_max))
    if C_out.max() > C_INIT + tol or C_out.min() < 0:
        violations.append("FAIL 含水率越界 [%.5f,%.5f] 应在[0,%.2f]" % (C_out.min(),
            C_out.max(), C_INIT))
    # 径向单调：T随r不减，C随r不增（逐时层，容差1e-4）
    dT = np.diff(T_out, axis=1)
```

```
    dC = np.diff(C_out, axis=1)
    if dT.min() < -1e-3:
        violations.append("FAIL 温度径向非单调(min dT=%.2e)" % dT.min())
    if dC.max() > 1e-3:
        violations.append("FAIL 含水率径向非单调(max dC=%.2e)" % dC.max())
    # 中心温度时间单调不减
    if np.diff(T_out[:, 0]).min() < -1e-3:
        violations.append("FAIL 中心温度随时间下降")
    ok = len(violations) == 0
    print("[validate_constraints] %s (n_violations=%d)" % ("PASS" if ok else "FAIL",
        len(violations)))
    for v in violations[:5]:
        print("  ", v)
    return ok


def validate_capability(res):
    # 能力：真解出时空演化的传热-传质场，非退化常数/未耦合边界
    T_out, C_out = res['T_out'], res['C_out']
    assert T_out.shape[1] == 21 and C_out.shape[1] == 21, "[能力未交付]
        输出网格非21点(0..2cm每0.1)"
    assert T_out.shape[0] == 1800, "[能力未交付] 时序非1800步(每1s)"
    # 表面响应快于中心：表面升温幅度 > 中心升温幅度
    surf_rise = T_out[-1, -1] - T_INIT
    center_rise = T_out[-1, 0] - T_INIT
    assert surf_rise > center_rise + 1e-3, "[能力不成立]
        表面未比中心响应更快，边界/扩散未真实作用"
    # 水分主要在表层变化：表面失水 > 中心失水
    surf_loss = C_INIT - C_out[-1, -1]
    center_loss = C_INIT - C_out[-1, 0]
    assert surf_loss > center_loss, "[能力不成立] 表层失水未大于中心，传质边界未真实驱动"
    # 中心1800s水分接近初值( _m 1800s)
    assert center_loss < 0.05, "[能力可疑] 中心失水过大，1800s内中心应几乎不变(见逻辑合同bounds)"
    print("[validate_capability] PASS surf_rise=%.3f center_rise=%.3f surf_loss=%.4f
        center_loss=%.4f"
         % (surf_rise, center_rise, surf_loss, center_loss))


if __name__ == '__main__':
    t0 = time.time()
    print("[problem1] 求解主网格 N=%d ..." % N_CV)
    _, T_inf_arr, _, _, _ = load_boundary()
    T_inf_max = float(T_inf_arr.max())
    res = solve(N=N_CV, return_full=True)
    print("主网格求解完成，耗时 %.1fs" % (time.time() - t0))
    print(" T_center(1800)=%.4f T_surface(1800)=%.4f" % (res['T_center_1800'],
        res['T_surface_1800']))
```

```
    print(" C_center(1800)=%.4f C_surface(1800)=%.4f" % (res['C_center_1800'],
        res['C_surface_1800']))

    ok1 = validate_constraints(res, T_inf_max)
    validate_capability(res)
    if not ok1:
        raise ValueError("约束验证失败，必须修正后重跑")

    # ---- 网格加密自检 N=50/100/200 ----
    print("[problem1] 网格加密自检 N=50/100/200 ...")
    grid_conv = {}
    key_r_cm = [0.0, 0.5, 1.0, 1.5, 2.0]
    key_idx = [int(round(x / OUTPUT_DR_CM)) for x in key_r_cm]
    for Ng in (50, 100, 200):
        rg = solve(N=Ng, return_full=True)
        grid_conv[str(Ng)] = {
            'T_1800': [round(float(rg['T_out'][-1, i]), 4) for i in key_idx],
            'C_1800': [round(float(rg['C_out'][-1, i]), 4) for i in key_idx],
        }
    # 相邻网格最大相对差
    def maxreldiff(a, b):
        a, b = np.array(a), np.array(b)
        return float(np.max(np.abs(a - b) / (np.abs(b) + 1e-12)))
    conv_T = maxreldiff(grid_conv['200']['T_1800'], grid_conv['100']['T_1800'])
    conv_C = maxreldiff(grid_conv['200']['C_1800'], grid_conv['100']['C_1800'])
    print(" 相邻网格(200 vs 100)最大相对差: T=%.3e C=%.3e" % (conv_T, conv_C))

    # ---- 逻辑方向探针: Ceq 朝更干推 ，中心C变化应极小 ----
    res_probe = solve(N=N_CV, ceq_scale=0.98, return_full=True)
    dC_center = res_probe['C_center_1800'] - res['C_center_1800']
    probe_sign = 1 if dC_center > 1e-9 else (-1 if dC_center < -1e-9 else 0)

    # ---- 保存结果 JSON（全精度写盘，供 paper-figure）----
    out = {
        'meta': {'seed': SEED, 'N_CV': N_CV, 'dt': DT, 'T_end': T_END,
                 'method': 'FVM cylindrical + Crank-Nicolson(T) + backward-Euler/Picard(C)',
                 'T_inf_max': T_inf_max, 'python': sys.version.split()[0], 'numpy':
                     np.__version__},
        'output_grid': {'r_cm': OUTPUT_R_CM.tolist(), 't_s': OUTPUT_T.tolist()},
        'T_field': np.round(res['T_out'], 4).tolist(), # [1800 x 21]
        'C_field': np.round(res['C_out'], 6).tolist(),
        'key_nodes': {
            't_s': [100, 300, 600, 900, 1200, 1500, 1800],
            'r_cm': key_r_cm,
        },
        'grid_convergence': grid_conv,
        'grid_conv_maxreldiff': {'T': conv_T, 'C': conv_C},
```

```
        'logic_probes': {'bounds': [
            {'quantity': 'C_center_1800', 'claim': 'upper', 'probe_delta_sign': probe_sign,
             'delta_value': dC_center, 'note': 'Ceq朝更干推2%, 中心C变化极小'}
        ]},
        'must_features': {'T_inf_t': True, 'Ceq_t': True},
    }
    # 表2/表3 关键节点数值
    tbl_t = [100, 300, 600, 900, 1200, 1500, 1800]
    tbl_idx_t = [t - 1 for t in tbl_t] # OUTPUT_T 从1起
    out['table_T'] = {str(t): [round(float(res['T_out'][ti, i]), 4) for i in key_idx]
                      for t, ti in zip(tbl_t, tbl_idx_t)}
    out['table_C'] = {str(t): [round(float(res['C_out'][ti, i]), 4) for i in key_idx]
                      for t, ti in zip(tbl_t, tbl_idx_t)}

    os.makedirs('figures', exist_ok=True)
    with open('figures/problem_1_results.json', 'w', encoding='utf-8') as fh:
        json.dump(out, fh, ensure_ascii=False)
    print("[problem1] 已保存 figures/problem_1_results.json")
    print(" logic probe: Ceq朝干推2%%, ΔC_center=%.3e (sign=%d, 期望 0)" % (dC_center,
        probe_sign))
```

1.4 问题一二维轴对称求解 problem1_axisym.py

```
# 问题一（轴向扩展）：二维轴对称 (r,z) 传热-传质求解器
# 在一维径向模型基础上加入长度方向 z：两方程各加一个轴向二阶扩散项，
# 新增 z=0 中截面对称、z=H 端面 Robin 边界（h_b= • h，h_mb= • h_m）。
# 圆柱+圆截面+均匀初值+周向均匀边界下，(r,z) 轴对称即三维精确形式（ 被对称性消去）。
#
# 离散口径与一维逐项同构（见 code/problem1.py）：
#   - FVM 柱坐标，(Nr,Nz)=(100,50)，dz=H/Nz；控制体权重 V_{i}=½(f_{i+1}²-f_i²)，单元体积
    V_{i} • dz。
#   - 温度：Crank－Nicolson 隐式（常物性 → 五对角稀疏矩阵时间不变，splu 一次分解）。
#   - 水分：backward-Euler + Picard（界面 D(C) 调和平均，容差 1e-8，上限 20 次），spsolve 五对角。
#   - Robin 边界用组合热阻/传质阻（单元中心 表面 环境）闭合，与一维完全相同。
#   - dt=1 s，0－1800 s，附件1 时变环境分段线性插值逐 1 s 驱动。
import os, sys, time, json
_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)

import numpy as np
import scipy.sparse as sp
from scipy.sparse.linalg import splu, spsolve
from scipy.linalg import solve_banded

```

```python
from params import *
import utils as u
import problem1 as p1 # 复用一维求解器与附件1读取（模块级仅设随机种子，无副作用运行）

np.random.seed(SEED)

# --- 轴向几何（params 未含半长/z 网格，此处按题面局部定义，不改 params）---
H_HALF = L / 2.0     # 半域长度 0.125 m（z=0 中截面，z=H 端面，两端条件相同）
NZ = 50              # 轴向控制体数，dz=H/Nz=2.5 mm


def _build_connectivity(Nr, Nz):
    """预计算五对角连接结构（几何固定，与迭代无关）。idx = i*Nz + k。"""
    ii = np.arange(1, Nr)      # 径向内部界面索引 1..Nr-1
    karr = np.arange(Nz)
    IIr, KKr = np.meshgrid(ii, karr, indexing='ij')   # (Nr-1, Nz)
    rowsR_A = ((IIr - 1) * Nz + KKr).ravel()          # 西侧单元 (i-1,k)
    rowsR_B = (IIr * Nz + KKr).ravel()                # 东侧单元 (i,k)
    faceR_i = IIr.ravel()                             # 界面几何索引

    iarr = np.arange(Nr)
    kk = np.arange(1, Nz)
    IIz, KKz = np.meshgrid(iarr, kk, indexing='ij')   # (Nr, Nz-1)
    rowsZ_A = (IIz * Nz + (KKz - 1)).ravel()          # 下侧单元 (i,k-1)
    rowsZ_B = (IIz * Nz + KKz).ravel()                # 上侧单元 (i,k)
    radZ_i = IIz.ravel()                              # 轴向面所属径向索引

    rR_cells = (Nr - 1) * Nz + np.arange(Nz)          # r=R 侧面单元 (i=Nr-1)
    zH_cells = np.arange(Nr) * Nz + (Nz - 1)          # z=H 端面单元 (k=Nz-1)
    Ntot = Nr * Nz
    diag_idx = np.arange(Ntot)
    # 预拼装 COO 行列（结构固定，仅数据随迭代更新）
    rows = np.concatenate([rowsR_A, rowsR_B, rowsZ_A, rowsZ_B, diag_idx])
    cols = np.concatenate([rowsR_B, rowsR_A, rowsZ_B, rowsZ_A, diag_idx])
    return dict(Ntot=Ntot, rowsR_A=rowsR_A, rowsR_B=rowsR_B, faceR_i=faceR_i,
                rowsZ_A=rowsZ_A, rowsZ_B=rowsZ_B, radZ_i=radZ_i,
                rR_cells=rR_cells, zH_cells=zH_cells,
                coo_rows=rows, coo_cols=cols)


def _assemble(cn, gR, gZ, bcR, bcZ):
    """由面导纳装配五对角算子 M: M•  = Σ_faces G(_nb-_cell) + 边界 -G_bc•_cell。"""
    Ntot = cn['Ntot']
    diag = -(np.bincount(cn['rowsR_A'], gR, Ntot) + np.bincount(cn['rowsR_B'], gR, Ntot)
             + np.bincount(cn['rowsZ_A'], gZ, Ntot) + np.bincount(cn['rowsZ_B'], gZ, Ntot)
             + np.bincount(cn['rR_cells'], bcR, Ntot) + np.bincount(cn['zH_cells'], bcZ, Ntot))
    data = np.concatenate([gR, gR, gZ, gZ, diag])
```

```
    return sp.coo_matrix((data, (cn['coo_rows'], cn['coo_cols'])), shape=(Ntot, Ntot)).tocsc()


def _solve_banded(cn, Nz, gR, gZ, bcR, bcZ, diag_add, rhs):
    """解 A·x=rhs，A=diag(diag_add)-M（M 五对角，带宽 Nz）。LAPACK 带状直接解，远快于反复稀疏 LU。

    idx=i*Nz+k：径向耦合偏移 ±Nz（值 gR），轴向耦合偏移 ±1（值 gZ）。
    """
    Ntot = cn['Ntot']
    A_diag = diag_add + (np.bincount(cn['rowsR_A'], gR, Ntot) + np.bincount(cn['rowsR_B'], gR,
        Ntot)
                         + np.bincount(cn['rowsZ_A'], gZ, Ntot) + np.bincount(cn['rowsZ_B'], gZ,
                             Ntot)
                         + np.bincount(cn['rR_cells'], bcR, Ntot) + np.bincount(cn['zH_cells'],
                             bcZ, Ntot))
    ab = np.zeros((2 * Nz + 1, Ntot))
    ab[Nz, :] = A_diag
    rA = cn['rowsR_A']                              # 径向：rowsR_B = rA + Nz
    ab[0, rA + Nz] = -gR                            # 上带 offset +Nz
    ab[2 * Nz, rA] = -gR                            # 下带 offset -Nz
    zA = cn['rowsZ_A']                              # 轴向：rowsZ_B = zA + 1
    ab[Nz - 1, zA + 1] = -gZ                        # 上带 offset +1
    ab[Nz + 1, zA] = -gZ                            # 下带 offset -1
    return solve_banded((Nz, Nz), ab, rhs, check_finite=False, overwrite_ab=True)


def solve(Nr=N_CV, Nz=NZ, dt=DT, beta=1.0, hConv=H_CONV, hm=HM_MASS,
          d_interface='harmonic', return_fields=True):
    """二维轴对称 (r,z) 求解 0..1800 s。beta=端面暴露系数（h_b= β·h，h_mb= β·h_m）。

    返回中截面(z=0)/端面(z=H)径向剖面时序（21点，cm）、1800 s 单元场与轴向剖面。
    """
    _, _, _, T_inf_f, Ceq_f = p1.load_boundary()
    dr, faces, centers, Vr = u.build_grid(Nr, R) # Vr：柱坐标径向控制体权重（每单位 z）
    dz = H_HALF / Nz
    z_centers = (np.arange(Nz) + 0.5) * dz

    Ntot = Nr * Nz
    cn = _build_connectivity(Nr, Nz)
    use_banded = Nz <= 60 # 带宽=Nz：小网格带状直接解更快，大网格改稀疏 LU（带宽无关）

    # 单元体积 V_{i,k} = Vr_i * dz（(Nr,Nz) 展平 idx=i*Nz+k）
    Vol = np.repeat(Vr * dz, Nz)

    # 径向内部界面几何权重 f_j·dz/dr（面积=2πf·dz，去掉共同 2π）；沿 z 复制
    geoR = (faces[1:Nr] * dz / dr)           # (Nr-1,)
    geoR_full = np.repeat(geoR, Nz)          # 展平顺序与 rowsR 对齐 (i,k)
```

```
    # 轴向内部界面几何权重 Vr_i/dz（面积权重=½(f²)差，去掉共同 2）；沿内部 z 界面复制
    geoZ_full = np.repeat(Vr / dz, Nz - 1).reshape(Nr, Nz - 1).ravel()

    T = np.full(Ntot, T_INIT)
    C = np.full(Ntot, C_INIT)

    # ---- 温度：常物性 → M_T 时间不变，装配一次并 splu ----
    aR_T = K_THERM * geoR_full                # 径向内部面导纳 k·f/dr
    aZ_T = K_THERM * geoZ_full                # 轴向内部面导纳 k·Vr/dz
    U_surf = 1.0 / ((dr / 2.0) / K_THERM + 1.0 / hConv) # 侧面组合换热(中心 环境)
    bcR_T = np.full(Nz, U_surf * R * dz)      # r=R 面接环境，面积权重 R·dz
    hb = beta * hConv
    if hb > 0:
        U_end = 1.0 / ((dz / 2.0) / K_THERM + 1.0 / hb) # 端面组合换热(中心 环境)
        bcZ_T = U_end * Vr                    # z=H 面接环境，面积权重 Vr_i
    else:
        U_end = 0.0
        bcZ_T = np.zeros(Nr)
    M_T = _assemble(cn, aR_T, aZ_T, bcR_T, bcZ_T)
    massT = RHO * CP * Vol
    A_T = sp.diags(massT) - (dt / 2.0) * M_T
    lu_T = splu(A_T.tocsc())

    Dfun = lambda cc: u.D_moisture(cc, D0, EA_OVER, D_FLOOR)

    def src_T(Tinf):
        s = np.zeros(Ntot)
        np.add.at(s, cn['rR_cells'], bcR_T * Tinf)
        np.add.at(s, cn['zH_cells'], bcZ_T * Tinf)
        return s

    def src_C(Ceq, bcR_C, bcZ_C):
        s = np.zeros(Ntot)
        np.add.at(s, cn['rR_cells'], bcR_C * Ceq)
        np.add.at(s, cn['zH_cells'], bcZ_C * Ceq)
        return s

    nt = int(round(T_END / dt))
    out_r_m = OUTPUT_R_CM / 100.0
    nout = len(out_r_m)
    Tmid = np.empty((nt, nout)); Cmid = np.empty((nt, nout))
    Tend = np.empty((nt, nout)); Cend = np.empty((nt, nout))

    def interp_row(cell_row, val0, valR):
        return u.interp_to_output(centers, cell_row, R, val0, valR, out_r_m)

    for step in range(nt):
```

```
        tt = (step + 1) * dt
        Tinf_new = float(T_inf_f(tt)); Tinf_old = float(T_inf_f(tt - dt))
        Ceq_new = float(Ceq_f(tt))

        # ---- 温度 Crank–Nicolson: (massD - dt/2 M) T^{n+1} = massD T^n + dt/2 M T^n + dt/2
            (s_old+s_new)
        rhs = massT * T + (dt / 2.0) * (M_T.dot(T) + src_T(Tinf_old) + src_T(Tinf_new))
        T = lu_T.solve(rhs)

        # ---- 水分 backward-Euler + Picard ----
        C_prev = C.copy(); C_iter = C.copy()
        for _ in range(PICARD_MAXIT):
            Dc = Dfun(C_iter)
            Dgrid = Dc.reshape(Nr, Nz)
            if d_interface == 'harmonic':
                DfaceR = u.harmonic(Dgrid[:-1, :], Dgrid[1:, :]).ravel() # 径向界面 (Nr-1,Nz)
                DfaceZ = u.harmonic(Dgrid[:, :-1], Dgrid[:, 1:]).ravel() # 轴向界面 (Nr,Nz-1)
            else:
                DfaceR = (0.5 * (Dgrid[:-1, :] + Dgrid[1:, :])).ravel()
                DfaceZ = (0.5 * (Dgrid[:, :-1] + Dgrid[:, 1:])).ravel()
            gR = DfaceR * geoR_full
            gZ = DfaceZ * geoZ_full
            # r=R 侧面: 表面单元 D 组合传质阻
            Dsurf_R = Dgrid[Nr - 1, :]
            UmR = 1.0 / ((dr / 2.0) / Dsurf_R + 1.0 / hm)
            bcR_C = UmR * R * dz
            # z=H 端面: h_mb= •h_m
            hmb = beta * hm
            if hmb > 0:
                Dsurf_Z = Dgrid[:, Nz - 1]
                UmZ = 1.0 / ((dz / 2.0) / Dsurf_Z + 1.0 / hmb)
                bcZ_C = UmZ * Vr
            else:
                UmZ = np.zeros(Nr); bcZ_C = np.zeros(Nr)
            rr = (Vol / dt) * C_prev + src_C(Ceq_new, bcR_C, bcZ_C)
            if use_banded:
                C_new = _solve_banded(cn, Nz, gR, gZ, bcR_C, bcZ_C, Vol / dt, rr)
            else:
                M_C = _assemble(cn, gR, gZ, bcR_C, bcZ_C)
                C_new = splu((sp.diags(Vol / dt) - M_C).tocsc()).solve(rr)
            if np.max(np.abs(C_new - C_iter)) < PICARD_TOL:
                C_iter = C_new; break
            C_iter = C_new
        C = C_iter

        # ---- 重构输出剖面（与一维同口径）----
        Tg = T.reshape(Nr, Nz); Cg = C.reshape(Nr, Nz)
```

```
        # 中截面 z=0: 轴向对称，取 k=0 行（O(dz²) 即中截面值）
        row = 0
        T0 = u.reconstruct_center(Tg[0, row], Tg[1, row])
        C0 = u.reconstruct_center(Cg[0, row], Cg[1, row])
        TR = Tinf_new - (U_surf / hConv) * (Tinf_new - Tg[Nr - 1, row])
        DsR = float(Dfun(np.array([Cg[Nr - 1, row]]))[0])
        UmR0 = 1.0 / ((dr / 2.0) / DsR + 1.0 / hm)
        CR = Ceq_new - (UmR0 / hm) * (Ceq_new - Cg[Nr - 1, row])
        Tmid[step] = interp_row(Tg[:, row], T0, TR)
        Cmid[step] = interp_row(Cg[:, row], C0, CR)
        # 端面 z=H: 先 Robin 反算各径向单元的端面表面值，再径向重构
        if hb > 0:
            Tsurf_z = Tinf_new - (U_end / hb) * (Tinf_new - Tg[:, Nz - 1])
        else:
            Tsurf_z = Tg[:, Nz - 1].copy()
        if beta * hm > 0:
            DsZ = Dfun(Cg[:, Nz - 1])
            UmZ_r = 1.0 / ((dz / 2.0) / DsZ + 1.0 / (beta * hm))
            Csurf_z = Ceq_new - (UmZ_r / (beta * hm)) * (Ceq_new - Cg[:, Nz - 1])
        else:
            Csurf_z = Cg[:, Nz - 1].copy()
        Te0 = u.reconstruct_center(Tsurf_z[0], Tsurf_z[1])
        Ce0 = u.reconstruct_center(Csurf_z[0], Csurf_z[1])
        TeR = Tinf_new - (U_surf / hConv) * (Tinf_new - Tsurf_z[Nr - 1])
        DeR = float(Dfun(np.array([Csurf_z[Nr - 1]]))[0])
        UmeR = 1.0 / ((dr / 2.0) / DeR + 1.0 / hm)
        CeR = Ceq_new - (UmeR / hm) * (Ceq_new - Csurf_z[Nr - 1])
        Tend[step] = interp_row(Tsurf_z, Te0, TeR)
        Cend[step] = interp_row(Csurf_z, Ce0, CeR)

    res = {'Tmid': Tmid, 'Cmid': Cmid, 'Tend': Tend, 'Cend': Cend,
           'z_cen_cm': (z_centers * 100.0), 'centers_cm': (centers * 100.0),
           'beta': beta, 'Nr': Nr, 'Nz': Nz}
    if return_fields:
        res['T_field'] = T.reshape(Nr, Nz).copy()
        res['C_field'] = C.reshape(Nr, Nz).copy()
    return res


# ===== 误差指标（2D vs 1D）与验证 =====
def error_series(res2d, res1d):
    """逐时刻中截面/端面径向最大偏差 E(t)（21点输出网格）。"""
    T1, C1 = res1d['T_out'], res1d['C_out']
    ET_mid = np.max(np.abs(res2d['Tmid'] - T1), axis=1)
    EC_mid = np.max(np.abs(res2d['Cmid'] - C1), axis=1)
    ET_end = np.max(np.abs(res2d['Tend'] - T1), axis=1)
    EC_end = np.max(np.abs(res2d['Cend'] - C1), axis=1)
```

```
    return ET_mid, EC_mid, ET_end, EC_end


def dom_max_depth(res2d, res1d):
    """1800 s 全域单元级最大偏差与轴向侵入深度（相对 1D 中心单元值，沿 z 广播）。"""
    T1c = res1d['centers_cell_T']; C1c = res1d['centers_cell_C'] # (Nr,) 单元中心解
    Tg = res2d['T_field']; Cg = res2d['C_field']
    dT = Tg - T1c[:, None]; dC = Cg - C1c[:, None]
    iT, kT = np.unravel_index(np.argmax(np.abs(dT)), dT.shape)
    iC, kC = np.unravel_index(np.argmax(np.abs(dC)), dC.shape)
    zc = res2d['z_cen_cm']
    # 侵入深度: 从端面 z=H 起, |Δ|>阈值 的最深 z（换算成"距端面深度 = H - z"）
    H_cm = H_HALF * 100.0
    maxT_by_z = np.max(np.abs(dT), axis=0) # 各 z 层全 r 最大
    maxC_by_z = np.max(np.abs(dC), axis=0)
    depthT = H_cm - zc[maxT_by_z > 0.05].min() if np.any(maxT_by_z > 0.05) else 0.0
    depthC = H_cm - zc[maxC_by_z > 0.01].min() if np.any(maxC_by_z > 0.01) else 0.0
    return {
        'dT_max': float(np.abs(dT).max()), 'dT_at': [float(res2d['centers_cm'][iT]),
            float(zc[kT])],
        'dC_max': float(np.abs(dC).max()), 'dC_at': [float(res2d['centers_cm'][iC]),
            float(zc[kC])],
        'depth_T_gt0.05_cm': float(depthT), 'depth_C_gt0.01_cm': float(depthC),
    }


def validate_constraints(res2d, T_inf_max):
    v = []
    for k in ('Tmid', 'Cmid', 'Tend', 'Cend'):
        if not np.all(np.isfinite(res2d[k])):
            v.append("FAIL %s 含 NaN/Inf" % k)
    tol = 1e-3
    Tall = np.concatenate([res2d['Tmid'].ravel(), res2d['Tend'].ravel()])
    Call = np.concatenate([res2d['Cmid'].ravel(), res2d['Cend'].ravel()])
    if Tall.min() < T_INIT - tol or Tall.max() > T_inf_max + 0.5:
        v.append("FAIL 温度越界 [%.4f,%.4f]" % (Tall.min(), Tall.max()))
    if Call.max() > C_INIT + tol or Call.min() < -tol:
        v.append("FAIL 含水率越界 [%.5f,%.5f]" % (Call.min(), Call.max()))
    # 端面偏差应  中截面偏差（轴向影响集中端面）
    ok = len(v) == 0
    print("[validate_constraints] %s (n=%d)" % ("PASS" if ok else "FAIL", len(v)))
    for x in v[:5]:
        print("  ", x)
    return ok


def validate_capability(res_b0, res1d, res_c1):
```

```
    """能力: =0 端面绝热必须逐位复现一维; =1 端面必须比中截面偏差大且非零。"""
    ETm, ECm, ETe, ECe = error_series(res_b0, res1d)
    reg = max(ETe.max(), ETm.max(), ECe.max(), ECm.max())
    assert reg < 5e-3, "[能力不成立] =0 未复现一维（max E=%.2e）" % reg
    ETm1, ECm1, ETe1, ECe1 = error_series(res_c1, res1d)
    assert ETm1.max() < 1e-2 and ECm1.max() < 1e-2, "[能力可疑] =1 中截面偏差过大"
    assert ETe1[-1] > ETm1[-1] + 0.5, "[能力不成立] =1 端面温度偏差未大于中截面"
    assert ECe1[-1] > 0.1, "[能力不成立] =1 端面失水偏差过小，端面传质未真实驱动"
    print("[validate_capability] PASS  0_reg=%.2e  1_end_ET(1800)=%.3f  1_end_EC(1800)=%.3f"
          % (reg, ETe1[-1], ECe1[-1]))


def _prof_1800(res2d):
    """1800 s 轴心 (r=0) 沿 z 的温度/含水率剖面（单元中心值，Nz 点）。"""
    return {'z_cm': res2d['z_cen_cm'].tolist(),
            'T_axis': res2d['T_field'][0, :].tolist(),
            'C_axis': res2d['C_field'][0, :].tolist()}


def _run_scenario(task):
    """进程池任务（Windows spawn 要求模块级函数）：求解一个  情景或一套收敛网格。"""
    tag, b = task
    tc = time.time()
    if b is None: # 收敛网格, =1
        nr, nz = (50, 25) if tag == 'conv50' else (200, 100)
        rc = solve(Nr=nr, Nz=nz, beta=1.0, return_fields=True)
    else:
        rc = solve(Nr=N_CV, Nz=NZ, beta=b, return_fields=True)
    return tag, rc, time.time() - tc


if __name__ == '__main__':
    t0 = time.time()
    print("[axisym] 载入一维基准 (N=%d) ..." % N_CV)
    _, T_inf_arr, _, _, _ = p1.load_boundary()
    T_inf_max = float(T_inf_arr.max())
    res1d = p1.solve(N=N_CV, return_full=True)
    # 一维单元中心解（用于全域偏差广播）
    res1d['centers_cell_T'] = res1d['centers_cell_T']
    print(" 1D: T_c=%.4f T_s=%.4f C_c=%.4f C_s=%.4f"
          % (res1d['T_center_1800'], res1d['T_surface_1800'],
             res1d['C_center_1800'], res1d['C_surface_1800']))

    # ---- 并行求解: 4 个  情景 + 2 套收敛网格（互不相关的独立正演 → 进程池任务级并行）----
    print("[axisym] 并行求解  =0/0.5/1/2 + 收敛网格 (50,25)/(200,100) ...")
    from multiprocessing import Pool

```

```
    scen, conv = {}, {}
    nproc = min(6, os.cpu_count() or 4)
    with Pool(nproc) as pool:
        results = pool.map(_run_scenario, [('B', 0.0), ('C05', 0.5), ('C1', 1.0), ('C2', 2.0),
                                           ('conv50', None), ('conv200', None)])
    for tag, r, wall in results:
        if tag.startswith('conv'):
            conv[tag] = r
            print(" %s done %.1fs" % (tag, wall))
        else:
            scen[tag] = r
            ETm, ECm, ETe, ECe = error_series(r, res1d)
            b = {'B': 0.0, 'C05': 0.5, 'C1': 1.0, 'C2': 2.0}[tag]
            print("  =%-4s done %.1fs | ET_mid=%.4f EC_mid=%.4f | ET_end(1800)=%.3f
                EC_end(1800)=%.3f"
                 % (b, wall, ETm.max(), ECm.max(), ETe[-1], ECe[-1]))

    ok = validate_constraints(scen['C1'], T_inf_max)
    validate_capability(scen['B'], res1d, scen['C1'])
    if not ok:
        raise ValueError("约束验证失败，必须修正后重跑")

    def maxreldiff(a, b):
        a, b = np.asarray(a), np.asarray(b)
        return float(np.max(np.abs(a - b) / (np.abs(b) + 1e-12)))

    # 主网格 C1 vs 相邻网格（同插值到21点输出网格的 1800 s 剖面）
    c1 = scen['C1']
    conv_report = {
        'Tmid_100_50_vs_50_25': maxreldiff(c1['Tmid'][-1], conv['conv50']['Tmid'][-1]),
        'Cmid_100_50_vs_50_25': maxreldiff(c1['Cmid'][-1], conv['conv50']['Cmid'][-1]),
        'Tend_100_50_vs_50_25': maxreldiff(c1['Tend'][-1], conv['conv50']['Tend'][-1]),
        'Cend_100_50_vs_50_25': maxreldiff(c1['Cend'][-1], conv['conv50']['Cend'][-1]),
        'Tmid_200_100_vs_100_50': maxreldiff(conv['conv200']['Tmid'][-1], c1['Tmid'][-1]),
        'Cmid_200_100_vs_100_50': maxreldiff(conv['conv200']['Cmid'][-1], c1['Cmid'][-1]),
        'Tend_200_100_vs_100_50': maxreldiff(conv['conv200']['Tend'][-1], c1['Tend'][-1]),
        'Cend_200_100_vs_100_50': maxreldiff(conv['conv200']['Cend'][-1], c1['Cend'][-1]),
    }
    print(" 收敛(端面C): 100/50 vs 50/25=%.3e | 200/100 vs 100/50=%.3e"
          % (conv_report['Cend_100_50_vs_50_25'], conv_report['Cend_200_100_vs_100_50']))

    # ---- 端面特征值与全域偏差（C1, =1, 1800 s）----
    dm = dom_max_depth(scen['C1'], res1d)
    end_axis_T = float(scen['C1']['Tend'][-1, 0])
    end_axis_C = float(scen['C1']['Cend'][-1, 0])
    end_corner_T = float(scen['C1']['Tend'][-1, -1])
    end_corner_C = float(scen['C1']['Cend'][-1, -1])
```

```
    print(" 端面轴心 T=%.4f(+%.2f) C=%.4f(%.2f) | 角点 T=%.4f C=%.4f | 全域|ΔT|max=%.3f
        |ΔC|max=%.3f"
         % (end_axis_T, end_axis_T - res1d['T_center_1800'],
            end_axis_C, end_axis_C - res1d['C_center_1800'],
            end_corner_T, end_corner_C, dm['dT_max'], dm['dC_max']))

    # ----  敏感性（端面轴心偏差, 1800 s）----
    beta_sens = {}
    for tag, b in [('C05', 0.5), ('C1', 1.0), ('C2', 2.0)]:
        beta_sens[str(b)] = {
            'end_axis_T': float(scen[tag]['Tend'][-1, 0]),
            'dT': float(scen[tag]['Tend'][-1, 0] - res1d['T_center_1800']),
            'end_axis_C': float(scen[tag]['Cend'][-1, 0]),
            'dC': float(scen[tag]['Cend'][-1, 0] - res1d['C_center_1800']),
        }

    # ---- 组织输出 JSON（全精度）----
    key_t = [100, 300, 600, 900, 1200, 1500, 1800]
    key_ti = [t - 1 for t in key_t]

    def series_dict(res2d):
        ETm, ECm, ETe, ECe = error_series(res2d, res1d)
        return {
            'ET_mid': [round(float(ETm[i]), 6) for i in key_ti],
            'EC_mid': [round(float(ECm[i]), 6) for i in key_ti],
            'ET_end': [round(float(ETe[i]), 6) for i in key_ti],
            'EC_end': [round(float(ECe[i]), 6) for i in key_ti],
        }

    out = {
        'meta': {'seed': SEED, 'Nr': N_CV, 'Nz': NZ, 'dt': DT, 'T_end': T_END,
                 'H_half_m': H_HALF, 'dz_m': H_HALF / NZ,
                 'method': 'FVM axisym (r,z) + CN(T splu once) + backward-Euler/Picard(C
                     spsolve), 5-diag sparse',
                 'beta_scenarios': [0.0, 0.5, 1.0, 2.0], 'beta_baseline': 1.0,
                 'T_inf_max': T_inf_max, 'python': sys.version.split()[0],
                 'numpy': np.__version__},
        'output_grid': {'r_cm': OUTPUT_R_CM.tolist(), 't_key_s': key_t},
        'baseline_1d': {'T_center_1800': res1d['T_center_1800'], 'T_surface_1800':
            res1d['T_surface_1800'],
                        'C_center_1800': res1d['C_center_1800'], 'C_surface_1800':
                            res1d['C_surface_1800']},
        'error_series': {tag: series_dict(scen[tag]) for tag in ('B', 'C05', 'C1', 'C2')},
        'end_face_1800_C1': {
            'axis_T': end_axis_T, 'axis_T_dev': end_axis_T - res1d['T_center_1800'],
            'axis_C': end_axis_C, 'axis_C_dev': end_axis_C - res1d['C_center_1800'],
            'corner_T': end_corner_T, 'corner_T_dev': end_corner_T - res1d['T_surface_1800'],
```

```
            'corner_C': end_corner_C, 'corner_C_dev': end_corner_C - res1d['C_surface_1800'],
        },
        'domain_max_1800_C1': dm,
        'beta_sens_1800': beta_sens,
        'grid_convergence': conv_report,
        'mid_profile_1800_C1': {
            'r_cm': OUTPUT_R_CM.tolist(),
            'T': np.round(scen['C1']['Tmid'][-1], 4).tolist(),
            'C': np.round(scen['C1']['Cmid'][-1], 6).tolist(),
        },
        'end_profile_1800_C1': {
            'r_cm': OUTPUT_R_CM.tolist(),
            'T': np.round(scen['C1']['Tend'][-1], 4).tolist(),
            'C': np.round(scen['C1']['Cend'][-1], 6).tolist(),
        },
        'axial_profile_1800_C1': _prof_1800(scen['C1']),
        'field_1800_C1': {
            'r_cm': scen['C1']['centers_cm'].tolist(),
            'z_cm': scen['C1']['z_cen_cm'].tolist(),
            'T': np.round(scen['C1']['T_field'], 4).tolist(),
            'C': np.round(scen['C1']['C_field'], 6).tolist(),
        },
    }
    os.makedirs('figures', exist_ok=True)
    with open('figures/problem1_axisym_results.json', 'w', encoding='utf-8') as fh:
        json.dump(out, fh, ensure_ascii=False)
    print("[axisym] 已保存 figures/problem1_axisym_results.json 总耗时 %.1fs" % (time.time() -
        t0))
```

1.5 问题二至四求解 solve_q234.py

```
# -*- coding: utf-8 -*-
# 问题2/3/4 求解引擎（修正版）：节点型有限体积 + 自适应BDF + 事件定位 + 守恒审计
# 模型：干基含水率 U、温度 T 在物质坐标 x=r/R(t) 上的耦合扩散（附录3/4 物性）：
#   U/ t = (1/(R²x))· (xD U/ x),  cp· T/ t = (1/(R²x))· (xk T/ x)
#  —— 无对流项：链式法则变换项 -Ṙx· U/ r 与欧拉对流项 +Ṙx· U/ r 精确对消（论文 8.3 节）
#    advVar=True 恢复早期推导的对流项（仅变体对照，t*=52.66h，见论文表8）
#    dilVar=True 为"体积浓度守恒解释"探针（稀释项 -2ṘU/R，见论文模型空间表）
# 离散口径：
#  - 节点 x_j=j/N 落边界（表面节点即药材表面，Robin 直接施加）；控制体体积 x dx 圆柱权重
#  - 面系数：水分 D(C,T) 沿节点间线性剖面做 Q 点 Gauss 积分（均匀温度下即 Kirchhoff 割线，
#    精确处理指数型 D(C)）；热导 k 调和平均（k 变化平缓，调和平均足够——已由 q8/q16
    与谐均值对照验证）
#  - 轴 r=0：通量为零（控制体积口径，无需处理 1/x 奇异）
#  - 时间：scipy BDF（带稀疏 Jacobian 模式），rtol=1e-8；事件=全场最大 U 降至 0.15（dense output
```

```
    求根）；
#    达标后用无事件积分检查下一整秒/整分钟的严格"<0.15"
#  - 守恒审计：水分（体积平均 U 的变化恒等于边界流失）+ 能量（含 cp 随 C 变化项与域收缩项）
# 结果落盘 figures/problem_{2,3,4}_results.json（与 fill_results.py 的旧格式兼容）。
import os, sys, time, json
import numpy as np
from numpy.polynomial.legendre import leggauss
from scipy.integrate import solve_ivp
from scipy.interpolate import PchipInterpolator
from scipy.sparse import diags, bmat
import pandas as pd
import utils as u

R0 = 0.02          # 初始半径 m
T_INIT = 28.0      # ℃
C_INIT = 2.55      # kg/kg
H_CONV = 25.0      # W/(m²K)
HM = 8e-7          # m/s
C_THRESHOLD = 0.15
ANNEX1 = 'user_data/附件1.xlsx'
ANNEX2 = 'user_data/附件2.xlsx'


# ---------- 物性（附录2/3/4，保持旧 API）----------
def props(C, T, pset):
    """返回 rho, cp, k, D。pset in {a2,a3,a4}。a3/a4 的 D 用开尔文温度。"""
    C = np.maximum(C, 1e-9)
    if pset == 'a2':
        rho = np.full_like(C, 820.0)
        cp = np.full_like(C, 2600.0)
        k = np.full_like(C, 0.36)
        D = 7e-9 * np.exp(-0.89 / C)
    elif pset == 'a3':
        rho = 650.0 + 128.0 * C
        cp = 1450.0 + 2736.0 * C / (C + 1.0)
        k = 0.21 + 0.38 * C / (C + 1.0)
        TK = T + 273.15
        D = 2.4e-3 * np.exp(-0.45 / C) * np.exp(-3850.0 / TK)
    elif pset == 'a4':
        rho = 760.0 + 90.0 * C
        cp = 1850.0 + 2150.0 * C / (C + 1.0)
        k = 0.12 + 0.20 * C / (C + 1.0)
        TK = T + 273.15
        D = 4.2e-4 * np.exp(-0.30 / C) * np.exp(-3850.0 / TK)
    else:
        raise ValueError(pset)
    return rho, cp, k, D
```

```python


# ---------- 环境（附件1 + 尾延拓口径）----------
def load_env(tail='nominal'):
    """tail: 'nominal'=恒温段平台
        50℃/0.05（生产口径）; 'last'=附件1末值恒定; 'mean'=末小时均值。"""
    df = pd.read_excel(ANNEX1, sheet_name='Sheet1')
    t = df['时间'].to_numpy(float)
    Tinf = df['温度'].to_numpy(float)
    Ceq = df['水分浓度'].to_numpy(float)
    if tail == 'nominal':
        t = np.concatenate([t, [t[-1] + 1.0, 1e9]])
        Tinf = np.concatenate([Tinf, [50.0, 50.0]])
        Ceq = np.concatenate([Ceq, [0.05, 0.05]])
    elif tail == 'mean':
        m = t >= 10800.0
        t = np.concatenate([t, [t[-1] + 1.0, 1e9]])
        Tinf = np.concatenate([Tinf, [Tinf[m].mean(), Tinf[m].mean()]])
        Ceq = np.concatenate([Ceq, [Ceq[m].mean(), Ceq[m].mean()]])
    fT = u.make_boundary_interp(t, Tinf) # 分段线性, 超界取端值
    fC = u.make_boundary_interp(t, Ceq)
    return fT, fC, float(Tinf.max()), float(Tinf[-1]), float(Ceq[-1])


# ---------- 半径（附件2）----------
def load_radius(mode='linear'):
    """返回 (fR, fRdot, tmax)。fRdot 为中心差弦斜率插值（仅变体对照/独立验证用;
    正确模型不需要 Ṙ——物质坐标方程只依赖 R(t) 本身）。"""
    df = pd.read_excel(ANNEX2, sheet_name='Sheet1')
    t = df['时间'].to_numpy(float)
    Rcm = df['半径'].to_numpy(float)
    Rm = Rcm / 100.0
    if mode == 'linear':
        fR = u.make_boundary_interp(t, Rm)
    elif mode == 'pchip':
        fR = PchipInterpolator(t, Rm, extrapolate=False)
    else:
        raise ValueError(mode)
    Rdot = np.gradient(Rm, t)
    fRdot = u.make_boundary_interp(t, Rdot)
    return fR, fRdot, t.max()


class DryingCylinder:
    """节点型有限体积的烘干圆柱求解器（固定域或随体收缩域, 附录2/3/4 物性）。"""

    def __init__(self, n=640, pset='a4', FixR=False, EnvTail='nominal',
```

```
                rMode='linear', quadrature=8, advVar=False,
                dilVar=False, bcMoist='robin', bcHeat='robin',
                hConv=H_CONV, LatHeat=False):
        self.n = n
        self.pset = pset
        self.FixR = FixR
        self.EnvTail = EnvTail
        self.quadrature = quadrature
        self.advVar = advVar
        self.dilVar = dilVar
        self.bcMoist = bcMoist
        self.bcHeat = bcHeat
        self.hConv = hConv      # 有效对流换热系数（辐射对照时可取 25+6.5）
        self.LatHeat = LatHeat
        self.L_vap = 2.4e6        # J/kg（蒸发潜热量级对照用）
        # 节点与几何
        self.x = np.linspace(0.0, 1.0, n + 1)
        self.dx = 1.0 / n
        self.faces = (self.x[:-1] + self.x[1:]) / 2.0 # 内部面位置 x_{j+1/2}
        edges = np.concatenate(([0.0], self.faces, [1.0]))
        self.vol = np.diff(edges ** 2) / 2.0          # 控制体体积权重  x dx
        # Gauss 节点/权重（沿面线性剖面）
        g, w = leggauss(quadrature)
        self.gfrac = ((g + 1.0) / 2.0)[:, None]
        self.gw = (w / 2.0)[:, None]
        # 环境与半径
        self.fT, self.fC, self.Tinf_max, self.Tinf_last, self.Ceq_last = load_env(EnvTail)
        self.fR, self.fRdot, self.radius_tmax = load_radius(rMode)
        self.rMode = rMode
        # 雅可比稀疏模式（两场各自的块三对角 + 全场互耦合）
        m = n + 1
        band = diags([np.ones(m - 1), np.ones(m), np.ones(m - 1)], [-1, 0, 1], format='csc')
        self.jac_sp = bmat([[band, band], [band, band]], format='csc')

    # ---- 环境 / 半径 ----
    def _env(self, t):
        return float(self.fT(t)), float(self.fC(t))

    def _radius(self, t):
        if self.FixR:
            return R0
        return float(self.fR(t))

    def _rdot(self, t):
        """中心差弦斜率（仅 advect/dilution 变体对照用）。"""
        return float(self.fRdot(t))

```

```
    # ---- 面系数 ----
    def _face_d(self, c, temp):
        """水分面系数：D(C,T) 沿节点线性剖面的 Gauss 积分（均匀温度下=Kirchhoff 割线）。"""
        cq = c[:-1][None, :] + self.gfrac * np.diff(c)[None, :]
        tq = temp[:-1][None, :] + self.gfrac * np.diff(temp)[None, :]
        _, _, _, dq = props(cq, tq, self.pset)
        return np.sum(self.gw * dq, axis=0)

    @staticmethod
    def _face_k(k):
        return 2.0 * k[:-1] * k[1:] / (k[:-1] + k[1:]) # 调和平均（k 变化平缓）

    # ---- 通量与右端 ----
    def _flux(self, u, coeff_face, radius, transfer, external, bc):
        """返回面通量数组 F[0..n+1]（单位角）：内部面 x·D·Δu/Δx；表面按 bc 口径。"""
        flux = np.zeros(self.n + 2)
        flux[1:-1] = self.faces * coeff_face * np.diff(u) / self.dx
        if bc == 'robin':
            flux[-1] = -radius * transfer * (u[-1] - external)
        elif bc == 'dirichlet':
            # 强汇：表面值=环境值，半胞传导通量 F=2D(u_external-u_n)/dx
            flux[-1] = -2.0 * coeff_face[-1] * (u[-1] - external) / self.dx
        else:
            raise ValueError(bc)
        return flux

    def rhs(self, t, y):
        temp, c = y.reshape(2, self.n + 1)
        rho, cp, k, _ = props(c, temp, self.pset)
        radius = self._radius(t)
        tinf, cinf = self._env(t)
        # 温度（Robin 换热；advect 变体另加对流项——见论文 8.3）
        kf = self._face_k(k)
        ft = self._flux(temp, kf, radius, self.hConv, tinf, self.bcHeat)
        if self.LatHeat:
            # 蒸发潜热冷却：表面热入流减去 L·_d·hm·(U_surf-C∞)（量级对照，非生产口径）
            rho_d = rho / (1.0 + c)
            ft[-1] -= radius * self.L_vap * rho_d[-1] * HM * (c[-1] - cinf)
        rhs_t = np.diff(ft) / (radius ** 2 * self.vol)
        if self.advVar:
            rdot = self._rdot(t)
            # 早期推导的迎风对流项 Ṙ/R· u（Ṙ<0 迎风=右邻；表面节点无对流）——仅变体对照
            adv = rdot / radius * self.x[:-1] * np.diff(temp) / self.dx
            rhs_t += np.concatenate([adv, [0.0]])
        rhs_t = rhs_t / (rho * cp)
        # 水分
        df = self._face_d(c, temp)
```

```
        fc = self._flux(c, df, radius, HM, cinf, self.bcMoist)
        rhs_c = np.diff(fc) / (radius ** 2 * self.vol)
        if self.advVar:
            rdot = self._rdot(t)
            adv = rdot / radius * self.x[:-1] * np.diff(c) / self.dx
            rhs_c += np.concatenate([adv, [0.0]])
        if self.dilVar:
            rdot = self._rdot(t)
            rhs_c += -2.0 * rdot / radius * c          # 稀释项 -2ṘU/R（体积浓度解释探针）
        return np.concatenate([rhs_t, rhs_c])

    # ---- 时间推进与事件 ----
    def run(self, rtol=1e-8, horizon=200 * 3600.0,
            stopEvt=True, audit=True, outDt=60.0):
        m = self.n + 1
        y0 = np.concatenate([np.full(m, T_INIT), np.full(m, C_INIT)])
        atol = np.concatenate([np.full(m, rtol), np.full(m, rtol * 0.01)])
        opts = dict(method='BDF', rtol=rtol, atol=atol, jac_sparsity=self.jac_sp,
                    dense_output=True)
        if stopEvt:
            def ev(t, y):
                return np.max(y[m:]) - C_THRESHOLD
            ev.terminal = True
            ev.direction = -1
            opts['events'] = ev
        s1 = solve_ivp(self.rhs, (0.0, min(14400.0, horizon)), y0, **opts)
        if not s1.success:
            raise RuntimeError(s1.message)
        s2 = None
        if horizon > 14400.0:
            s2 = solve_ivp(self.rhs, (14400.0, horizon), s1.y[:, -1], **opts)
            if not s2.success:
                raise RuntimeError(s2.message)
        if stopEvt:
            ev_arr = s2.t_events[0] if s2 is not None else s1.t_events[0]
            if len(ev_arr) == 0:
                raise RuntimeError('未触发烘干事件: horizon 内 Cmax 未降至 %.3f' % C_THRESHOLD)
            te = float(ev_arr[0])
        else:
            te = None

        def evaluate(t):
            tt = np.atleast_1d(t).astype(float)
            out = np.empty((2 * m, len(tt)))
            first = tt <= 14400.0
            if first.any():
                out[:, first] = s1.sol(tt[first])
```

```
            if (~first).any():
                out[:, ~first] = s2.sol(tt[~first])
            return out

        # 严格"<0.15": 事件后无事件积分到下一整秒/整分钟
        strict = None
        if stopEvt:
            after = solve_ivp(self.rhs, (te, te + 60.0), s2.y[:, -1], method='BDF',
                              rtol=rtol, atol=atol, jac_sparsity=self.jac_sp,
                              t_eval=[np.ceil(te), np.ceil(te / 60) * 60])
            strict = dict(seconds=after.t.tolist(),
                          cmax=[float(np.max(after.y[m:, i])) for i in range(after.y.shape[1])])

        # 输出时间序列（Q2 用 outDt=1s，Q3/Q4 用 60s；末行恒为事件时刻；首行=outDt）
        if stopEvt:
            times = np.concatenate([np.arange(outDt, te, outDt), [te]])
        else:
            times = np.arange(outDt, horizon + 1e-9, outDt)
        states = evaluate(times)
        temp, c = states[:m], states[m:]

        summary = dict(n=self.n, pset=self.pset, EnvTail=self.EnvTail,
                       FixR=self.FixR, quadrature=self.quadrature,
                       advVar=self.advVar,
                       dilVar=self.dilVar,
                       rtol=rtol,
                       event_s=te, event_h=(te / 3600.0) if te else None,
                       ev_r_cm=(self._radius(te) * 100) if te else None,
                       end_center=float(c[0, -1]), end_surface=float(c[-1, -1]),
                       strict_check=strict)
        if audit and stopEvt:
            summary.update(self._audit(times, evaluate, c))
        return summary, dict(times=times, x=self.x, temp=temp.T, c=c.T,
                             radius=np.array([self._radius(t) for t in times]))

    # ---- 守恒审计（连续口径，区间 Gauss 积分）----
    def _audit(self, times, evaluate, c):
        m = self.n + 1
        mean = 2.0 * (c.T @ self.vol)              # 体积平均 U（时间序列）
        g, w = leggauss(5)
        cum_out = 0.0
        et = np.concatenate([[0.0], times])        # 从 t=0 起积（首区间不可漏）
        for lo, hi in zip(et[:-1], et[1:]):
            qt = (lo + hi) / 2 + (hi - lo) / 2 * g
            qc = evaluate(qt)[-1]                   # 表面 U(qt)
            qr = np.array([self._radius(t) for t in qt])
            cinf = np.array([self._env(t)[1] for t in qt])
```

```
            cum_out += (hi - lo) / 2 * np.dot(w, 2.0 * HM / qr * (qc - cinf))
        balance = float(mean[-1] - C_INIT + cum_out)
        # 能量审计: E=R²Σ cp·T·V; dE/dt = h(T∞-T_surf)R + 2ṘE/R + R²Σ(d(cp)/dC·C/t)·T·V
        rho0, cp0, _, _ = props(np.full(m, C_INIT), np.full(m, T_INIT), self.pset)
        E0 = R0 ** 2 * float(np.sum(rho0 * cp0 * T_INIT * self.vol))
        y_end = evaluate(times[-1:])[:, 0]
        rho_e, cp_e, _, _ = props(c[:, -1], y_end[:m], self.pset)
        E_end = self._radius(times[-1]) ** 2 * float(np.sum(rho_e * cp_e * y_end[:m] * self.vol))
        qh_int = volc_int = cfx_int = 0.0
        dt_e = 300.0
        et = np.arange(0.0, times[-1], dt_e)
        if et[-1] < times[-1]:
            et = np.concatenate([et, [times[-1]]])
        for lo, hi in zip(et[:-1], et[1:]):
            qt = (lo + hi) / 2 + (hi - lo) / 2 * g
            yq = evaluate(qt)
            tq, cq = yq[:m], yq[m:]
            qr = np.array([self._radius(t) for t in qt])
            tinf = np.array([self._env(t)[0] for t in qt])
            rho_q, cp_q, _, _ = props(cq, tq, self.pset)
            E_q = (qr ** 2) * np.sum(rho_q * cp_q * tq * self.vol[:, None], axis=0)
            heat_q = H_CONV * (tinf - tq[-1]) * qr
            qrdot = np.array([0.0 if self.FixR else self._rdot(t) for t in qt])
            volc_q = 2.0 * qrdot / qr * E_q
            rhs_cq = np.array([self.rhs(t, yq[:, i])[m:] for i, t in enumerate(qt)]).T
            eps = 1e-6
            rp, cpp, _, _ = props(cq + eps, tq, self.pset)
            drhocp = (rp * cpp - rho_q * cp_q) / eps
            cfx_q = (qr ** 2) * np.sum(drhocp * rhs_cq * tq * self.vol[:, None], axis=0)
            qh_int += (hi - lo) / 2 * np.dot(w, heat_q)
            volc_int += (hi - lo) / 2 * np.dot(w, volc_q)
            cfx_int += (hi - lo) / 2 * np.dot(w, cfx_q)
        e_res = E_end - E0 - qh_int - volc_int - cfx_int
        return dict(wm_end=float(mean[-1]),
                    bnd_out=float(cum_out),
                    bal_res=balance,
                    e_res=float(e_res),
                    e_rel=abs(e_res) / E0)


# ---------- 固定域求解包装（问题2/3，旧 API 兼容）----------
def solve_fixed(pset='a3', dt=1.0, t_end=None, stopBelow=None,
                tMaxCap=200 * 3600.0, N=640, EnvTail='nominal',
                progress=None):
    """固定域烘干。dt=输出步长；stopBelow 给定时事件求根并返回 t_star_s。"""
    model = DryingCylinder(n=N, pset=pset, FixR=True, EnvTail=EnvTail)
    horizon = t_end if t_end is not None else tMaxCap
```

```
    summ, arr = model.run(rtol=1e-8, horizon=horizon,
                          stopEvt=(stopBelow is not None), outDt=dt)
    times = arr['times']; temp = arr['temp']; c = arr['c']
    # 输出到固定 0..2cm/0.1cm 网格（问题2 需要温度+水分）
    out_r_m = np.round(np.arange(0.0, 2.0 + 1e-9, 0.1), 4) / 100.0
    T_rows = np.array([np.interp(out_r_m / R0, model.x, temp[i]) for i in range(len(times))])
    C_rows = np.array([np.interp(out_r_m / R0, model.x, c[i]) for i in range(len(times))])
    return {
        'r_cm': (out_r_m * 100.0).round(4).tolist(),
        't_s': times.tolist(),
        'T': T_rows, 'C': C_rows,
        't_star_s': summ['event_s'],
        'Tinf_max': model.Tinf_max,
        'audit': {kk: summ.get(kk) for kk in ('bal_res', 'wm_end',
                                              'bnd_out', 'e_rel')},
    }


# ---------- 动边界求解包装（问题4，旧 API 兼容）----------
def solve_moving(pset='a4', dt=60.0, stopBelow=C_THRESHOLD,
                 tMaxCap=200 * 3600.0, N=1280, progress=None, advect=False,
                 audit=True, EnvTail='nominal', rMode='linear',
                 dilution=False, bcMoist='robin'):
    """随体收缩域（物质坐标 x=r/R(t)，正确模型无对流项）。
    advect=True/dilution=True 为论文 8.3 模型空间对照变体。"""
    model = DryingCylinder(n=N, pset=pset, FixR=False, EnvTail=EnvTail,
                           rMode=rMode, quadrature=8,
                           advVar=advect, dilVar=dilution,
                           bcMoist=bcMoist)
    summ, arr = model.run(rtol=1e-8, horizon=tMaxCap,
                          stopEvt=(stopBelow is not None), audit=audit,
                          outDt=dt)
    times = arr['times']; c = arr['c']; radius = arr['radius']
    out_r_m = np.round(np.arange(0.0, 2.0 + 1e-9, 0.1), 4) / 100.0
    C_rows = np.zeros((len(times), len(out_r_m)))
    for i in range(len(times)):
        ri = radius[i]
        row = np.interp(out_r_m, model.x * ri, c[i])
        row[out_r_m > ri + 1e-12] = np.nan
        C_rows[i] = row
    out = {
        'r_cm': (out_r_m * 100.0).round(4).tolist(),
        't_s': times.tolist(), 'C': C_rows,
        'C_surf': [float(v) for v in c[:, -1]],
        'R_cm': [float(v * 100) for v in radius],
        't_star_s': summ['event_s'],
        'Tinf_max': model.Tinf_max,
```

```
    }
    if audit:
        out['audit'] = {kk: summ.get(kk) for kk in ('bal_res', 'wm_end',
                                                    'bnd_out', 'e_rel')}
    return out


if __name__ == '__main__':
    os.chdir(os.path.dirname(os.path.abspath(__file__)))
    key_r = [0.0, 0.5, 1.0, 1.5, 2.0]
    key_idx = [int(round(x / 0.1)) for x in key_r]

    # ===== 问题2: 附录3 固定域 3h, dt=1s 输出 =====
    print("[Q2] 附录3 变物性 0..3h (节点FVM+BDF) ...", flush=True)
    t0 = time.time()
    q2 = solve_fixed(pset='a3', dt=1.0, t_end=3 * 3600.0, N=640)
    print(" Q2 完成 %.1fs, 步数=%d" % (time.time() - t0, len(q2['t_s'])), flush=True)
    q2_tbl_t = [0.5, 1.0, 1.5, 2.0, 2.5, 3.0]
    tbl_T = {}; tbl_C = {}
    for th in q2_tbl_t:
        i = int(np.argmin(np.abs(np.array(q2['t_s']) - th * 3600.0)))
        tbl_T["%.1f" % th] = [round(float(q2['T'][i, j]), 4) for j in key_idx]
        tbl_C["%.1f" % th] = [round(float(q2['C'][i, j]), 4) for j in key_idx]
    out2 = {
        'meta': {'pset': 'a3', 'method': 'node_FVM+BDF+Gauss_face', 'EnvTail': 'nominal',
                 'Tinf_max': q2['Tinf_max']},
        'r_cm': q2['r_cm'], 't_s': q2['t_s'],
        'T_field': np.round(q2['T'], 4).tolist(), 'C_field': np.round(q2['C'], 4).tolist(),
        'table_T': tbl_T, 'table_C': tbl_C, 'key_r_cm': key_r,
    }
    with open('figures/problem_2_results.json', 'w', encoding='utf-8') as fh:
        json.dump(out2, fh, ensure_ascii=False)
    print(" 已存 figures/problem_2_results.json", flush=True)

    # ===== 问题3: 附录3 固定域至 C<0.15 (n=1280, rtol=1e-8) =====
    print("[Q3] 附录3 长时程至全场 C<0.15 ...", flush=True)
    t0 = time.time()
    q3 = solve_fixed(pset='a3', dt=60.0, stopBelow=C_THRESHOLD, N=1280)
    tstar3 = q3['t_star_s']
    print(" Q3 完成 %.1fs, t*=%.3fh 水分收支残差=%.3e" %
          (time.time() - t0, tstar3 / 3600.0, q3['audit']['bal_res']), flush=True)
    tbl5 = {}
    ts3 = np.array(q3['t_s'])
    for th in range(6, int(tstar3 / 3600.0) + 1, 6):
        i = int(np.argmin(np.abs(ts3 - th * 3600.0)))
        tbl5["%d" % th] = [round(float(q3['C'][i, j]), 4) for j in key_idx]
    tbl5["t_star=%.3fh" % (tstar3 / 3600.0)] = [round(float(q3['C'][-1, j]), 4) for j in
```

```
        key_idx]
maxC = np.max(q3['C'], axis=1).tolist()
out3 = {
    'meta': {'pset': 'a3', 'method': 'node_FVM+BDF+Gauss_face', 'EnvTail': 'nominal',
             't_star_s': tstar3, 't_star_h': tstar3 / 3600.0,
             'C_threshold': C_THRESHOLD, 'Tinf_max': q3['Tinf_max']},
    'r_cm': q3['r_cm'], 't_s': q3['t_s'],
    'C_field': np.round(q3['C'], 4).tolist(),
    'maxC_t': [round(x, 6) for x in maxC],
    'table_C': tbl5, 'key_r_cm': key_r,
    'audit': q3['audit'],
}
with open('figures/problem_3_results.json', 'w', encoding='utf-8') as fh:
    json.dump(out3, fh, ensure_ascii=False)
print(" 已存 figures/problem_3_results.json", flush=True)

# ===== 问题4: 附录4 + 附件2 收缩域至 C<0.15（正确模型, n=1280, rtol=1e-8）=====
print("[Q4] 附录4 收缩域(物质坐标模型) 至全场 C<0.15 ...", flush=True)
t0 = time.time()
q4 = solve_moving(pset='a4', dt=60.0, stopBelow=C_THRESHOLD, N=1280)
tstar4 = q4['t_star_s']
print(" Q4 完成 %.1fs, t*_shrink=%.3fh 水分收支残差=%.3e" %
      (time.time() - t0, tstar4 / 3600.0, q4['audit']['bal_res']), flush=True)
tbl6 = {}
for th in range(6, int(tstar4 / 3600.0) + 1, 6):
    i = int(np.argmin(np.abs(np.array(q4['t_s']) - th * 3600.0)))
    row = [round(float(q4['C'][i, j]), 4) if np.isfinite(q4['C'][i, j]) else None
           for j in key_idx]
    row.append(round(float(q4['C_surf'][i]), 4))
    tbl6["%d" % th] = row
lastrow = [round(float(q4['C'][-1, j]), 4) if np.isfinite(q4['C'][-1, j]) else None
           for j in key_idx]
lastrow.append(round(float(q4['C_surf'][-1]), 4))
tbl6["t_star=%.3fh" % (tstar4 / 3600.0)] = lastrow
out4 = {
    'meta': {'pset': 'a4', 'method': 'node_FVM+BDF+Gauss_face',
             'moving_boundary': 'landau_no_adv', 'EnvTail': 'nominal',
             't_star_s': tstar4, 't_star_h': tstar4 / 3600.0,
             'C_threshold': C_THRESHOLD, 'Tinf_max': q4['Tinf_max']},
    'r_cm': q4['r_cm'], 't_s': q4['t_s'],
    'C_field': [[float(x) if np.isfinite(x) else None for x in row]
                for row in np.round(q4['C'], 4)],
    'C_surf': [round(float(x), 4) for x in q4['C_surf']],
    'R_cm': [round(float(x), 4) for x in q4['R_cm']],
    'table_C': tbl6, 'key_r_cm': key_r,
    'audit': q4['audit'],
}
```

```
    with open('figures/problem_4_results.json', 'w', encoding='utf-8') as fh:
        json.dump(out4, fh, ensure_ascii=False)
    print(" 已存 figures/problem_4_results.json", flush=True)

    print("[cross] Q3 t*=%.3fh Q4 t*_shrink=%.3fh 收缩加速=%s" %
          (tstar3 / 3600.0, tstar4 / 3600.0, "是" if tstar4 <= tstar3 else "否"), flush=True)
    print("[done] Q2/Q3/Q4 全部落盘。", flush=True)
```

1.6 灵敏度分析 sensitivity_analysis.py

```
# 灵敏度分析：h(±10%)、hm(±10%)、Ceq口径(±20%)、界面D平均方式 对中心/表面 T,C 的影响
import os, sys, json
_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)
import numpy as np
import problem1 as p1
from params import H_CONV, HM_MASS


def run():
    base = p1.solve()
    keys = ['T_center_1800', 'T_surface_1800', 'C_center_1800', 'C_surface_1800']
    baseline = {k: base[k] for k in keys}

    out = {'baseline': baseline, 'parameters': {}}

    # h ±10%
    hv = [H_CONV * f for f in (0.9, 1.0, 1.1)]
    out['parameters']['hConv'] = {
        'values': hv,
        **{k: [p1.solve(hConv=h)[k] for h in hv] for k in keys}}

    # hm ±10%
    hmv = [HM_MASS * f for f in (0.9, 1.0, 1.1)]
    out['parameters']['hm_mass'] = {
        'values': hmv,
        **{k: [p1.solve(hm=hm)[k] for hm in hmv] for k in keys}}

    # Ceq 口径 ±20%（核心软肋）
    cs = [0.8, 1.0, 1.2]
    out['parameters']['ceq_scale'] = {
        'values': cs,
        **{k: [p1.solve(ceq_scale=s)[k] for s in cs] for k in keys}}

```

```
    # 界面D平均方式 调和 vs 算术
    r_arith = p1.solve(d_interface='arithmetic')
    out['parameters']['d_interface'] = {
        'values': ['harmonic', 'arithmetic'],
        **{k: [base[k], r_arith[k]] for k in keys}}

    with open('figures/sensitivity_results.json', 'w', encoding='utf-8') as fh:
        json.dump(out, fh, ensure_ascii=False, indent=1)
    print("[sensitivity] 已保存 figures/sensitivity_results.json")
    for pname, pd in out['parameters'].items():
        print(" %s: values=%s" % (pname, pd['values']))
        print("    C_surface_1800=%s" % [round(x, 4) for x in pd['C_surface_1800']])
        print("    C_center_1800=%s" % [round(x, 4) for x in pd['C_center_1800']])
    return out


if __name__ == '__main__':
    run()
```

1.7 四问收敛性与模型空间复算 convergence.py

```
# -*- coding: utf-8 -*-
# 论文第10节复算程序: 收敛性与模型空间（新求解器口径: 节点FVM + BDF + Gauss面系数）
# A: Q2 表值空间稳定性(n=320/640)
# B: Q3 空间收敛 n=200/400/800 + 时间rtol + 最大步长 + 求积阶（生产 n=1280 复用
    problem_3_results.json）
# C: Q4 同矩阵（生产 n=1280 复用 problem_4_results.json）
# D: 模型空间变体（论文8.3: advect/dilution/fixed/dirichlet/环境尾/半径插值）
# E: 与外部独立实现的交叉对照汇总
import os, sys, time, json
_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)
import numpy as np
from solve_q234 import (DryingCylinder, solve_fixed, solve_moving, C_THRESHOLD,
                        R0, T_INIT, C_INIT)
FIG = os.path.join(_HERE, 'figures')


def _tstar(problem, n, rtol, quad, **kw):
    if problem == 'q3':
        model = DryingCylinder(n=n, pset='a3', FixR=True, EnvTail='nominal',
                               quadrature=quad)
        summ, _ = model.run(rtol=rtol, horizon=200 * 3600.0,
                            stopEvt=True, audit=False)
```

```
    else:
        model = DryingCylinder(n=n, pset='a4', FixR=False, EnvTail='nominal',
                               quadrature=quad, **kw)
        summ, _ = model.run(rtol=rtol, horizon=200 * 3600.0,
                            stopEvt=True, audit=False)
    return summ['event_h']


def _variant(kw, n=640, rtol=1e-8):
    model = DryingCylinder(n=n, pset='a4', FixR=False, quadrature=8, **kw)
    summ, _ = model.run(rtol=rtol, horizon=200 * 3600.0,
                        stopEvt=True, audit=False)
    return summ['event_h']


def _worker(cfg):
    t0 = time.time()
    tag = cfg.pop('tag')
    val = _tstar(**cfg)
    return tag, val, time.time() - t0


def _worker_var(cfg):
    t0 = time.time()
    tag = cfg.pop('tag')
    kw = dict(cfg.pop('kw'))
    val = _variant(kw, n=cfg.get('n', 640), rtol=cfg.get('rtol', 1e-8))
    return tag, val, time.time() - t0


if __name__ == '__main__':
    os.chdir(_HERE)
    out = {}

    print('=' * 70, flush=True)
    print('[A0] Q1 时间步收敛 (N=100, dt=2/1/0.5 s, 1800s 公共时刻剖面)', flush=True)
    import problem1 as p1
    r1 = {}
    for dtx in (2.0, 1.0, 0.5):
        r1[dtx] = p1.solve(N=100, dt=dtx, return_full=True)
    for key in ('T_out', 'C_out'):
        ref = np.asarray(r1[1.0][key])
        d21 = float(np.abs(np.asarray(r1[2.0][key]) - ref[1::2]).max())
        d1h = float(np.abs(np.asarray(r1[0.5][key][3::4]) - ref[1::2]).max())
        print(' %s 公共时刻 max|Delta|: dt2-vs-1=%.3e dt0.5-vs-1=%.3e (比值=%.1f)'
              % (key, d21, d1h, d21 / max(d1h, 1e-30)), flush=True)
        out['q1_time_%s' % key] = dict(dt2_vs_1=d21, dt05_vs_1=d1h)
```

```
71
72  print('=' * 70, flush=True)
73  print('[A] Q2 表值空间稳定性 (n=320/640, 3h 五位置)', flush=True)
74  from multiprocessing import Pool
75  for n in (320, 640):
76      model = DryingCylinder(n=n, pset='a3', FixR=True, EnvTail='nominal')
77      summ, arr = model.run(rtol=1e-8, horizon=3 * 3600.0,
78                            stopEvt=False, audit=False, outDt=1.0)
79      i3 = int(np.argmin(np.abs(arr['times'] - 3 * 3600.0)))
80      row = arr['c'][i3]
81      vals = [float(np.interp(r / 100 / R0, model.x, row)) for r in (0, .5, 1, 1.5, 2)]
82      print(' n=%3d: 3h C = [%s]' % (n, ', '.join('%.4f' % v for v in vals)), flush=True)
83      out['q2_n%d' % n] = vals
84
85  print('=' * 70, flush=True)
86  print('[B/C] Q3/Q4 空间收敛 (rtol=1e-8, q8)', flush=True)
87  sweeps = [
88      dict(tag='q3_n200', problem='q3', n=200, rtol=1e-8, quad=8),
89      dict(tag='q3_n400', problem='q3', n=400, rtol=1e-8, quad=8),
90      dict(tag='q3_n800', problem='q3', n=800, rtol=1e-8, quad=8),
91      dict(tag='q4_n200', problem='q4', n=200, rtol=1e-8, quad=8),
92      dict(tag='q4_n400', problem='q4', n=400, rtol=1e-8, quad=8),
93      dict(tag='q4_n800', problem='q4', n=800, rtol=1e-8, quad=8),
94  ]
95  with Pool(3) as pool:
96      rows = pool.map(_worker, sweeps)
97  for tag, val, sec in rows:
98      print(' %-8s t*=%.6f h (%.1fs)' % (tag, val, sec), flush=True)
99      out[tag] = val
100
101 print('=' * 70, flush=True)
102 print('[B/C] 时间容差/最大步长/求积阶 敏感性 (n=400)', flush=True)
103 sweeps2 = [
104     dict(tag='q3_rtol1e-7', problem='q3', n=400, rtol=1e-7, quad=8),
105     dict(tag='q3_rtol1e-9', problem='q3', n=400, rtol=1e-9, quad=8),
106     dict(tag='q3_q4pt', problem='q3', n=400, rtol=1e-8, quad=4),
107     dict(tag='q3_q16pt', problem='q3', n=400, rtol=1e-8, quad=16),
108     dict(tag='q4_rtol1e-7', problem='q4', n=400, rtol=1e-7, quad=8),
109     dict(tag='q4_rtol1e-9', problem='q4', n=400, rtol=1e-9, quad=8),
110     dict(tag='q4_q4pt', problem='q4', n=400, rtol=1e-8, quad=4),
111     dict(tag='q4_q16pt', problem='q4', n=400, rtol=1e-8, quad=16),
112 ]
113 with Pool(3) as pool:
114     rows2 = pool.map(_worker, sweeps2)
115 for tag, val, sec in rows2:
116     print(' %-12s t*=%.6f h (%.1fs)' % (tag, val, sec), flush=True)
117     out[tag] = val
```

```
118
119     print('=' * 70, flush=True)
120     print('[D] 模型空间变体 (论文8.3; n=640, rtol=1e-8)', flush=True)
121     varcfgs = [
122         dict(tag='advect_last', kw=dict(advVar=True, EnvTail='last')),
123         dict(tag='advect_nominal', kw=dict(advVar=True, EnvTail='nominal')),
124         dict(tag='dilution_nominal', kw=dict(dilVar=True, EnvTail='nominal')),
125         dict(tag='correct_last', kw=dict(EnvTail='last')),
126         dict(tag='correct_mean', kw=dict(EnvTail='mean')),
127         dict(tag='dirichlet', kw=dict(bcMoist='dirichlet', EnvTail='nominal')),
128     ]
129     with Pool(3) as pool:
130         rows3 = pool.map(_worker_var, varcfgs)
131     for tag, val, sec in rows3:
132         print(' %-16s t*=%.4f h (%.1fs)' % (tag, val, sec), flush=True)
133         out[tag] = val
134     # 固定域 a4（无收缩上界）与 pchip 半径插值
135     mf = DryingCylinder(n=640, pset='a4', FixR=True, EnvTail='nominal')
136     sf, _ = mf.run(rtol=1e-8, horizon=200 * 3600.0, stopEvt=True, audit=False)
137     print(' fixed_a4       t*=%.4f h' % sf['event_h'], flush=True)
138     out['fixed_a4'] = sf['event_h']
139     mp = DryingCylinder(n=640, pset='a4', FixR=False, EnvTail='nominal', rMode='pchip')
140     sp, _ = mp.run(rtol=1e-8, horizon=200 * 3600.0, stopEvt=True, audit=False)
141     print(' pchip_radius   t*=%.4f h' % sp['event_h'], flush=True)
142     out['pchip_radius'] = sp['event_h']
143
144     print('=' * 70, flush=True)
145     print('[E] 与外部独立实现对照 (外部收敛值: Q3=57.5394, Q4=51.0906, 表值四位小数)', flush=True)
146     j3 = json.load(open(os.path.join(FIG, 'problem_3_results.json'), encoding='utf-8'))
147     j4 = json.load(open(os.path.join(FIG, 'problem_4_results.json'), encoding='utf-8'))
148     print(' 生产(本实现 n=1280): Q3 t*=%.4f h (外部 57.5394) | Q4 t*=%.4f h (外部 51.0906)'
149           % (j3['meta']['t_star_h'], j4['meta']['t_star_h']), flush=True)
150     print(' 水分收支残差: Q3=%.3e Q4=%.3e (外部 ~7e-6 量级)' %
151           (j3['audit']['bal_res'], j4['audit']['bal_res']), flush=True)
152     print(' 能量收支相对残差: Q4=%.3e' % j4['audit'].get('e_rel', float('nan')), flush=True)
153     with open(os.path.join(FIG, 'convergence_new.json'), 'w', encoding='utf-8') as fh:
154         json.dump(out, fh, ensure_ascii=False, indent=1)
155     print('[done] 全部落盘 figures/convergence_new.json', flush=True)
```

1.8 问题四独立有限差分核验 crossval/fd_q4.py

```
1 # -*- coding: utf-8 -*-
2 # 问题4 交叉验证(算法层): 节点型有限差分 — 修正模型(物质坐标无对流项)
3 #  U/ t = (1/(R²x)) · (xD U/ x), 表面节点落边界、Robin 直接施加; BE + Picard。
4 # 与生产(单元中心FVM)、外部实现(节点FVM+BDF)构成第三套独立代码路径。
```

```
import os, sys, time
_HERE = os.path.dirname(os.path.abspath(__file__))
_CODE = os.path.dirname(_HERE)
if _CODE not in sys.path:
    sys.path.insert(0, _CODE)
import numpy as np
import solve_q234 as q


def _thomas(a, b, c, d):
    n = len(b)
    cp = np.empty(n); dp = np.empty(n)
    cp[0] = c[0] / b[0]; dp[0] = d[0] / b[0]
    for i in range(1, n):
        m = b[i] - a[i] * cp[i - 1]
        cp[i] = c[i] / m
        dp[i] = (d[i] - a[i] * dp[i - 1]) / m
    x = np.empty(n); x[-1] = dp[-1]
    for i in range(n - 2, -1, -1):
        x[i] = dp[i] - cp[i] * x[i + 1]
    return x


def solve_fd(N=200, dt=15.0, stopBelow=0.15, tMaxCap=200 * 3600.0):
    """节点型 FD:  [0,1] n+1 节点, 面在节点中点, 面系数调和平均, 轴 r=0 用对称格式。
    表面节点 n: 半胞体积 + Robin 通量(口径 ×1/R²)。无对流项(修正模型)。"""
    fT, fC, _, _, _ = q.load_env()
    fR, _, _ = q.load_radius()
    n = N
    dxi = 1.0 / N
    x = np.linspace(0.0, 1.0, n + 1)
    xf = (x[:-1] + x[1:]) / 2.0           # 内部面位置
    V = np.zeros(n + 1)
    V[0] = (xf[0] ** 2) / 2.0             # [0, xf0]
    for j in range(1, n):
        V[j] = (xf[j] ** 2 - xf[j - 1] ** 2) / 2.0
    V[n] = (1.0 - xf[-1] ** 2) / 2.0      # [xf_last, 1]

    T = np.full(n + 1, 28.0); C = np.full(n + 1, 2.55)
    ts, Cmax_s, Csurf_s = [], [], []
    t_star = None
    cm_prev = 2.55
    step = 0; tt = 0.0
    while tt < tMaxCap - 1e-9:
        tt = (step + 1) * dt
        Tinf = float(fT(tt)); Ceq = float(fC(tt))
        Rt = float(fR(tt))
```

```
        invR2 = 1.0 / (Rt * Rt)
        Tk, Ck = T.copy(), C.copy()
        for _ in range(30):
            rho, cp, kth, _ = q.props(Ck, Tk, 'a4')
            kf = 2.0 * kth[:-1] * kth[1:] / (kth[:-1] + kth[1:])
            aW = np.zeros(n + 1); aE = np.zeros(n + 1)
            aW[1:n] = invR2 * xf[:-1] * kf[:-1] / dxi
            aE[1:n] = invR2 * xf[1:] * kf[1:] / dxi
            aW[0] = 0.0
            aE[0] = invR2 * 4.0 * xf[0] * kf[0] / dxi # 轴对称: (2D/d²)(U1-U0)·V0/V0口径
            # 表面节点 n: Robin 口径
            aW[n] = invR2 * xf[-1] * kf[-1] / dxi
            aE[n] = 25.0 / Rt                       # H_CONV·(Tinf-Tn)/R
            cap = rho * cp * V / dt
            sub = np.zeros(n + 1); dia = np.zeros(n + 1); sup = np.zeros(n + 1)
            rhs = np.zeros(n + 1)
            dia[:] = cap + aE + aW
            sub[1:] = -aW[1:]
            sup[:n] = -aE[:n]
            rhs[:] = cap * T
            rhs[n] += aE[n] * Tinf
            T_new = _thomas(sub, dia, sup, rhs)
            _, _, _, Dc = q.props(Ck, T_new, 'a4')
            Df = 2.0 * Dc[:-1] * Dc[1:] / (Dc[:-1] + Dc[1:])
            bW = np.zeros(n + 1); bE = np.zeros(n + 1)
            bW[1:n] = invR2 * xf[:-1] * Df[:-1] / dxi
            bE[1:n] = invR2 * xf[1:] * Df[1:] / dxi
            bE[0] = invR2 * 4.0 * xf[0] * Df[0] / dxi
            bW[n] = invR2 * xf[-1] * Df[-1] / dxi
            bE[n] = 8e-7 / Rt
            capC = V / dt
            s = np.zeros(n + 1); d = np.zeros(n + 1); p = np.zeros(n + 1); rr = np.zeros(n + 1)
            d[:] = capC + bE + bW
            s[1:] = -bW[1:]
            p[:n] = -bE[:n]
            rr[:] = capC * C
            rr[n] += bE[n] * Ceq
            C_new = _thomas(s, d, p, rr)
            err = max(np.max(np.abs(T_new - Tk)), np.max(np.abs(C_new - Ck)) / 2.55)
            Tk, Ck = T_new, C_new
            if err < 1e-6:
                break
        T, C = Tk, Ck
        Cmax = float(C.max())
        ts.append(tt); Cmax_s.append(Cmax); Csurf_s.append(float(C[n]))
        if t_star is None and Cmax < stopBelow:
            frac = (stopBelow - cm_prev) / (Cmax - cm_prev)
```

```
            t_star = tt - dt * (1.0 - frac)
            break
        cm_prev = Cmax
        step += 1
    return {'t_star_h': t_star / 3600.0 if t_star else None,
            'ts': ts, 'Cmax': Cmax_s, 'Csurf': Csurf_s}


if __name__ == '__main__':
    os.chdir(_CODE)
    for N in (200, 400):
        t0 = time.time()
        r = solve_fd(N=N, dt=15.0)
        print('FD节点法 N=%3d t*=%.4f h Cmax(48h)=%.4f Csurf(48h)=%.4f (%.1fs)'
              % (N, r['t_star_h'], np.interp(48 * 3600.0, r['ts'], r['Cmax']),
                 np.interp(48 * 3600.0, r['ts'], r['Csurf']), time.time() - t0), flush=True)
    print('[done]', flush=True)
```

1.9 问题四切割单元有限体积核验 crossval/cutcell_q4.py

```
# -*- coding: utf-8 -*-
# 问题4 交叉验证(算法层): 固定欧拉网格 + 切割单元 FVM — 修正模型(无对流项的欧拉形式)
# 修正模型: U/ t|_r + v• U/ r =  •(D U), v=Ṙr/R (物质坐标变换后与生产口径 U/ t=(1/R²x)(xD xU) 等价)
# 离散口径:
#   - 固定网格 r∈[0,R0]; 活动单元=中心<R(t); 切割单元 k 右面精确落在 R(t) (宽 w∈(0.5,1.5]dr, 无小切单元)
#   - 扩散: 内部面调和平均; 切割面 Robin 闭合 (bc='res' 总热阻 / 'quad' 二次Lagrange / 'dirichlet' 强汇上界)
#   - 对流: 非守恒迎风 -v• U/ r (v<0 取右邻; 切割单元用表面值 U_R 与距离 Rt-r_k)
#     dia[i]+=-V•v/dr, sup[i]+=+V•v/dr (M矩阵; 若误用守恒对消形式则退化为被否定的稀释项模型, 见论文)
#   - BE + Picard 双向耦合 (容差同生产 1e-6); 落胞质量/热含量守恒合并; 逐步守恒审计
import os, sys, time, json
_HERE = os.path.dirname(os.path.abspath(__file__))
_CODE = os.path.dirname(_HERE)
if _CODE not in sys.path:
    sys.path.insert(0, _CODE)
import numpy as np
import solve_q234 as q

R0 = 0.02
H_CONV = 25.0
HM = 8e-7
C_THRESHOLD = 0.15
```

```
TOL = 1e-6
MAXIT = 30


def _thomas(a, b, c, d):
    n = len(b)
    cp = np.empty(n); dp = np.empty(n)
    cp[0] = c[0] / b[0]; dp[0] = d[0] / b[0]
    for i in range(1, n):
        m = b[i] - a[i] * cp[i - 1]
        cp[i] = c[i] / m
        dp[i] = (d[i] - a[i] * dp[i - 1]) / m
    x = np.empty(n); x[-1] = dp[-1]
    for i in range(n - 2, -1, -1):
        x[i] = dp[i] - cp[i] * x[i + 1]
    return x


def _lagrange3(rn, x):
    L = np.zeros(3); Lp = np.zeros(3)
    for j in range(3):
        others = [m for m in range(3) if m != j]
        den = (rn[j] - rn[others[0]]) * (rn[j] - rn[others[1]])
        num = (x - rn[others[0]]) * (x - rn[others[1]])
        nump = (x - rn[others[0]]) + (x - rn[others[1]])
        L[j] = num / den
        Lp[j] = nump / den
    return L, Lp


def _solve_field(u_old, u_base, k, Vnew, faces, dr, centers, Rt, Rdot, dt, bc,
                 Dcell, Hcoef, bcval, rcp):
    """BE 单场 (T/C 同构)。u_old=系数评估用迭代值; u_base=上一时间步值(右端项固定)。
    返回 (u_new, u_surf, flux_in, adv_sum)。"""
    n = k + 1
    w = Rt - faces[k]
    sub = np.zeros(n); dia = np.zeros(n); sup = np.zeros(n); rhs = np.zeros(n)
    Df = 2.0 * Dcell[:-1] * Dcell[1:] / (Dcell[:-1] + Dcell[1:])
    for i in range(1, k + 1):                  # 内部面扩散
        g = Df[i - 1] * faces[i] / dr
        dia[i - 1] += g; sup[i - 1] = -g
        sub[i] = -g; dia[i] += g
    # 非守恒对流 -v·∂u/∂r (v=ṙ/R<0, 迎风=右邻): 源项 = co·(U_{i+1}-U_i), co=-V·v/dr>0
    # BE 行: dia[i] += co, sup[i] += -co (M矩阵); adv_sum 累计源项值供审计
    adv_sum = 0.0
    for i in range(k):
        vi = Rdot * centers[i] / Rt
```

```
        co = -Vnew[i] * vi / dr
        dia[i] += co; sup[i] -= co
        adv_sum += co * (u_old[i + 1] - u_old[i])
    extra = None
    vk = Rdot * centers[k] / Rt
    dk = Rt - centers[k]                    # 切割单元中心到边界距离
    if bc == 'res':
        U = 1.0 / ((w / 2.0) / Dcell[k] + 1.0 / Hcoef)
        dia[k] += U * Rt; rhs[k] += U * Rt * bcval
        alpha = U / Hcoef
        beta = (1.0 - alpha) * bcval
        co_k = -Vnew[k] * vk / dk
        dia[k] += -co_k * (alpha - 1.0)     # 源项 co_k[(-1)U_k+] → dia += -co_k(-1)
        rhs[k] += co_k * beta               # 常数部分 co_k·  移入右端(正号)
        # 退化极限 dk→0: U_k → /(1-)  +0.05 (表面值), 保证非负
        u_surf = alpha * u_old[k] + beta
        flux_in = U * Rt * (bcval - u_old[k])
        adv_sum += co_k * (u_surf - u_old[k]) # = co_k·[(-1)U_k+]
    elif bc == 'dirichlet':
        gR = Dcell[k] * Rt / (w / 2.0)
        dia[k] += gR; rhs[k] += gR * bcval
        co_k = -Vnew[k] * vk / dk
        dia[k] += co_k                      # 源项 co_k·(bcval-u_k) → dia += co_k
        rhs[k] += co_k * bcval              # 常数部分移入右端(正号)
        u_surf = bcval
        flux_in = gR * (bcval - u_old[k])
        adv_sum += co_k * (bcval - u_old[k])
    else: # quad: 二次Lagrange外推表面值 p(R), 通量=R·h·(bcval-p(R))
        rn = centers[k - 2:k + 1]
        L, Lp = _lagrange3(rn, Rt)
        if k < 2 or not np.all(np.isfinite(L)):
            L = np.array([0.0, 0.0, 1.0]) # 退化保护: 回退为表面值=U_k 口径
            Lp = np.array([0.0, 0.0, 1.0])
        pR = L[2] * u_old[k] + L[1] * u_old[k - 1] + L[0] * u_old[k - 2]
        c0, c1, c2 = -Rt * Hcoef * L[0], -Rt * Hcoef * L[1], -Rt * Hcoef * L[2]
        rhs[k] += Rt * Hcoef * bcval
        # 切割单元对流: 源项 co_k·(p(R)-u_k) → 行 k 耦合 k-2,k-1,k
        co_k = -Vnew[k] * vk / dk
        dia[k] += -c2 - co_k * (L[2] - 1.0)
        sup[k - 1] += -c1 - co_k * L[1]
        extra = (k - 2, -c0 - co_k * L[0])
        u_surf = pR
        flux_in = -Rt * Hcoef * (pR - bcval)
        adv_sum += co_k * (pR - u_old[k])
    cap = rcp * Vnew[:n] / dt
    dia += cap; rhs += cap * u_base
    if extra is None:
```

```
        u = _thomas(sub, dia, sup, rhs)
    else:
        col, x = extra
        s = np.zeros(k); s[k - 1] = sup[k - 1]
        u0 = _thomas(sub[:k], dia[:k], sup[:k], rhs[:k])
        u1 = _thomas(sub[:k], dia[:k], sup[:k], -s)
        uk = (rhs[k] - sub[k] * u0[k - 1] - x * u0[k - 2]) / (dia[k] + sub[k] * u1[k - 1] + x *
            u1[k - 2])
        u = np.concatenate([u0 + u1 * uk, [uk]])
    return u, u_surf, flux_in, adv_sum


def solve_cutcell(N=200, dt=15.0, bc='res', stopBelow=C_THRESHOLD,
                  tMaxCap=200 * 3600.0):
    fT, fC, _, _, _ = q.load_env()
    fR, fRdot, _ = q.load_radius()
    dr = R0 / N
    faces = np.arange(N + 1) * dr
    centers = (faces[:-1] + faces[1:]) / 2.0
    Vgeo = (faces[1:] ** 2 - faces[:-1] ** 2) / 2.0

    T = np.full(N, 28.0); C = np.full(N, 2.55)
    Vcur = np.zeros(N)
    k = N - 1
    Vcur[:k + 1] = Vgeo[:k + 1]

    ts, Cmax_s, Csurf_s = [], [], []
    W0 = float(np.sum(C * Vgeo))
    lossW_cum = volW_cum = advW_cum = 0.0
    max_resid = 0.0
    t_star = None
    cm_prev = 2.55
    step = 0; tt = 0.0

    while tt < tMaxCap - 1e-9:
        tt = (step + 1) * dt
        Tinf = float(fT(tt)); Ceq = float(fC(tt))
        Rt = float(fR(tt)); Rdot = float(fRdot(tt))
        k_new = int(np.searchsorted(centers, Rt, side='left')) - 1
        Vnew = np.zeros(N)
        Vnew[:k_new] = Vgeo[:k_new]
        Vnew[k_new] = (Rt * Rt - faces[k_new] ** 2) / 2.0

        dv = Vnew - Vcur
        volW = float(np.sum(dv * C))
        while k > k_new:                        # 落胞合并(质量/热含量守恒)
            cj = C[k]; Vj = Vcur[k]
```

```
            if Vj > 0:
                rho_j, cp_j, _, _ = q.props(np.array([cj]), np.array([T[k]]), 'a4')
                rho_k, cp_k, _, _ = q.props(np.array([C[k_new]]), np.array([T[k_new]]), 'a4')
                Hj = rho_j[0] * cp_j[0] * T[k] * Vj
                Hk = rho_k[0] * cp_k[0] * T[k_new] * Vnew[k_new]
                C[k_new] = (C[k_new] * Vnew[k_new] + cj * Vj) / Vnew[k_new]
                rho_m, cp_m, _, _ = q.props(np.array([C[k_new]]), np.array([T[k_new]]), 'a4')
                T[k_new] = (Hk + Hj) / (rho_m[0] * cp_m[0] * Vnew[k_new])
            k -= 1

        W_old = float(np.sum(C[:k_new + 1] * Vcur[:k_new + 1]))
        T_base = T[:k_new + 1].copy() # 上一时间步值：右端项固定
        C_base = C[:k_new + 1].copy()
        Tk, Ck = T_base.copy(), C_base.copy()
        for _ in range(MAXIT):
            rho, cp, kth, _ = q.props(Ck, Tk, 'a4')
            T_new, T_surf, fluxT, advT = _solve_field(
                Tk, T_base, k_new, Vnew, faces, dr, centers, Rt, Rdot, dt, bc,
                kth, H_CONV, Tinf, rho * cp)
            _, _, _, Dc = q.props(Ck, T_new, 'a4')
            C_new, C_surf, fluxC, advC = _solve_field(
                Ck, C_base, k_new, Vnew, faces, dr, centers, Rt, Rdot, dt, bc,
                Dc, HM, Ceq, np.ones(k_new + 1))
            if not (np.all(np.isfinite(T_new)) and np.all(np.isfinite(C_new))):
                # 非有限回退：二次外推闭合脆弱时改用总热阻两点闭合(robust口径)
                T_new, T_surf, fluxT, advT = _solve_field(
                    Tk, T_base, k_new, Vnew, faces, dr, centers, Rt, Rdot, dt, 'res',
                    kth, H_CONV, Tinf, rho * cp)
                _, _, _, Dc2 = q.props(Ck, T_new, 'a4')
                C_new, C_surf, fluxC, advC = _solve_field(
                    Ck, C_base, k_new, Vnew, faces, dr, centers, Rt, Rdot, dt, 'res',
                    Dc2, HM, Ceq, np.ones(k_new + 1))
            err = max(np.max(np.abs(T_new - Tk)), np.max(np.abs(C_new - Ck)) / 2.55)
            Tk, Ck = T_new, C_new
            if err < TOL:
                break
        T[:k_new + 1] = Tk; C[:k_new + 1] = Ck
        Vcur = Vnew.copy()

        lossW = dt * Rt * HM * (C_surf - Ceq)
        W_new = float(np.sum(C[:k_new + 1] * Vnew[:k_new + 1]))
        resid = W_new - W_old + lossW - volW - dt * advC
        max_resid = max(max_resid, abs(resid))
        lossW_cum += lossW; volW_cum += volW; advW_cum += dt * advC

        C0 = (9.0 * C[0] - C[1]) / 8.0
        Cmax = max(C0, float(C[:k_new + 1].max()))
```

```
        ts.append(tt); Cmax_s.append(Cmax); Csurf_s.append(C_surf)
        if t_star is None and Cmax < stopBelow:
            frac = (stopBelow - cm_prev) / (Cmax - cm_prev)
            t_star = tt - dt * (1.0 - frac)
            break
        cm_prev = Cmax
        step += 1

    return {
        't_star_h': t_star / 3600.0 if t_star else None,
        'ts': ts, 'Cmax': Cmax_s, 'Csurf': Csurf_s,
        'audit': {'max_residW': max_resid,
                  'relW_final': abs(W_new + lossW_cum - volW_cum - advW_cum - W0) / W0},
    }


if __name__ == '__main__':
    os.chdir(_CODE)
    for N, bc in ((200, 'res'), (400, 'res'), (200, 'quad'), (200, 'dirichlet')):
        t0 = time.time()
        r = solve_cutcell(N=N, dt=15.0, bc=bc)
        print('cutcell N=%3d bc=%-9s t*=%.4f h max_residW=%.3e relW=%.3e (%.1fs)'
              % (N, bc, r['t_star_h'], r['audit']['max_residW'], r['audit']['relW_final'],
                 time.time() - t0), flush=True)
        if N == 200 and bc == 'res':
            for th in (48.0, 51.0):
                print('   Cmax(%dh)=%.4f Csurf=%.4f' %
                      (th, np.interp(th * 3600.0, r['ts'], r['Cmax']),
                       np.interp(th * 3600.0, r['ts'], r['Csurf'])), flush=True)
    print('[done]', flush=True)
```

1.10 问题一至三第三套独立实现核验 crossval/ext_check_q1q3.py

```
# -*- coding: utf-8 -*-
# 交叉核验（跨实现、跨面系数口径）：用 external_verify/solve_q4.py 的独立实现
# 核验生产 Q1/Q2 表值与 Q3 t*。
#   Q1: 附录2 常物性固定域，对比 1800s 五位置 T/C（生产参考=problem1 表1/表2）；
#   Q2: 附录3 变物性固定域，对比 3h 五位置 T/C（生产参考=problem_2_results.json 表3/表4）；
#   Q3: 附录3 变物性固定域，对比 t*（生产参考=problem_3_results.json，nominal 尾）。
# 注意口径差（不是错误）：本实现的面系数为**端点调和平均**，生产引擎为沿节点线性剖面的
# 8 点 Gauss 积分（指数型 D(C) 下的 Kirchhoff 割线）。在干燥后期干壳层内调和平均会抹平
# D 的指数坍缩，故本实现的 t* 系统性偏慢；两口径在网格加密下收敛到同一极限
# （见论文"收敛性与数值质量"章节）。
import sys, json
from pathlib import Path
```

```python
import numpy as np
from scipy.integrate import solve_ivp

_HERE = Path(__file__).resolve().parents[1] # 支撑材料/code
_EXT = _HERE / 'external_verify'
sys.path.insert(0, str(_EXT))
import solve_q4
solve_q4.DATA = _HERE / 'user_data'         # 附件1.xlsx / 附件2.xlsx
from solve_q4 import ShrinkingCylinder


class FixedA2(ShrinkingCylinder):
    """附录2 常物性 + 固定域 (Q1 口径)。"""
    def __init__(self, n=640):
        super().__init__(n=n, env='nominal', radius='linear', FixR=True)

    @staticmethod
    def properties(c, temp):
        c = np.maximum(c, 1e-8)
        rho = np.full_like(c, 820.0)
        cp = np.full_like(c, 2600.0)
        k = np.full_like(c, 0.36)
        d = 7e-9 * np.exp(-0.89 / c)
        return rho, cp, k, d


class FixedA3(ShrinkingCylinder):
    """附录3 变物性 + 固定域 (Q2/Q3 口径)。"""
    def __init__(self, n=640, env='nominal'):
        super().__init__(n=n, env=env, radius='linear', FixR=True)

    @staticmethod
    def properties(c, temp):
        c = np.maximum(c, 1e-8)
        rho = 650.0 + 128.0 * c
        cp = 1450.0 + 2736.0 * c / (c + 1.0)
        k = 0.21 + 0.38 * c / (c + 1.0)
        d = 2.4e-3 * np.exp(-0.45 / c) * np.exp(-3850.0 / (temp + 273.15))
        return rho, cp, k, d


def q1_check():
    m = FixedA2(n=640)
    y0 = np.r_[np.full(m.n + 1, 28.0), np.full(m.n + 1, 2.55)]
    s = solve_ivp(m.rhs, (0., 1800.), y0, method='BDF', rtol=1e-7,
                  atol=np.r_[np.full(m.n + 1, 1e-7), np.full(m.n + 1, 1e-9)],
                  jac_sparsity=m.jac, dense_output=True)
```

```
    u = s.sol(1800.0)
    T, C = u[:m.n + 1], u[m.n + 1:]
    prod = json.load(open(_HERE / 'figures' / 'problem_1_results.json', encoding='utf-8'))
    ref_T = prod['table_T']['1800']
    ref_C = prod['table_C']['1800']
    print('Q1 1800s 五位置对比 (外部BDF n=640 vs 生产表1/表2):', flush=True)
    maxdT = maxdC = 0.0 # 逐位置取最大绝对偏差
    for r_cm, rt, rc in zip([0, .5, 1, 1.5, 2], ref_T, ref_C):
        x = r_cm / 100 / 0.02
        eT = float(np.interp(x, m.x, T)); eC = float(np.interp(x, m.x, C))
        maxdT = max(maxdT, abs(eT - rt)); maxdC = max(maxdC, abs(eC - rc))
        print(' r=%4g cm T: %8.4f (ref %8.4f, delta %+7.4f) | C: %8.4f (ref %8.4f, delta %+7.4f)'
              % (r_cm, eT, rt, eT - rt, eC, rc, eC - rc), flush=True)
    print(' max|delta|: T=%.4f C C=%.4f kg/kg' % (maxdT, maxdC), flush=True)


def q2_check():
    m = FixedA3(n=640, env='nominal') # 3h < 4h, 尾延拓无影响
    y0 = np.r_[np.full(m.n + 1, 28.0), np.full(m.n + 1, 2.55)]
    s = solve_ivp(m.rhs, (0., 3 * 3600.), y0, method='BDF', rtol=1e-7,
                  atol=np.r_[np.full(m.n + 1, 1e-7), np.full(m.n + 1, 1e-9)],
                  jac_sparsity=m.jac, dense_output=True)
    u = s.sol(3 * 3600.0)
    T, C = u[:m.n + 1], u[m.n + 1:]
    ref = json.load(open(_HERE / 'figures' / 'problem_2_results.json', encoding='utf-8'))
    tT = ref['table_T']['3.0']; tC = ref['table_C']['3.0']
    print('Q2 3h 五位置对比 (外部BDF n=640 vs 生产表3/表4):', flush=True)
    maxdT = maxdC = 0.0
    for r_cm, rt, rc in zip([0, .5, 1, 1.5, 2], tT, tC):
        x = r_cm / 100 / 0.02
        eT = float(np.interp(x, m.x, T)); eC = float(np.interp(x, m.x, C))
        maxdT = max(maxdT, abs(eT - rt)); maxdC = max(maxdC, abs(eC - rc))
        print(' r=%4g cm T: %8.4f (ref %8.4f, delta %+7.4f) | C: %8.4f (ref %8.4f, delta %+7.4f)'
              % (r_cm, eT, rt, eT - rt, eC, rc, eC - rc), flush=True)
    print(' max|delta|: T=%.4f C C=%.4f kg/kg' % (maxdT, maxdC), flush=True)


def q3_check():
    """Q3 跨口径核验: 生产引擎（节点 FVM + BDF + Gauss/Kirchhoff 面系数, n=1280）
    vs 本实现（继承 solve_q4.py, 端点调和平均面系数, n=640）。"""
    prod = json.load(open(_HERE / 'figures' / 'problem_3_results.json', encoding='utf-8'))
    for env in ('nominal', 'last'):
        m = FixedA3(n=640, env=env)
        summ, _ = m.run(rtol=1e-7, horizon=200 * 3600.)
        if env == 'nominal':
            ref = float(prod['meta']['t_star_h'])
            print('Q3 env=%-7s: 外部(调和平均口径) t*=%.4f h | 生产(Gauss口径) t*=%.4f h
```

```
                Delta=%+.4f h'
107              % (env, summ['event_h'], ref, summ['event_h'] - ref), flush=True)
108       else:
109           print('Q3 env=%-7s: 外部(调和平均口径) t*=%.4f h (生产表值采用 nominal 口径)'
110                 % (env, summ['event_h']), flush=True)
111
112
113 if __name__ == '__main__':
114     q1_check()
115     q2_check()
116     q3_check()
117     print('[done]', flush=True)
```

1.11 结果文件生成 fill_results.py

```
1  # -*- coding: utf-8 -*-
2  # 按官方附件3模板填充 result1-4.xlsx（模板在 user_data/，只覆盖数据区，保留模板结构与表头）
3  # result1: 温度/水分浓度，t=1..1800 s；result2: 温度/水分浓度，t=1..10800 s (dt=1)
4  # result3: Sheet1 水分浓度，t=60..t* 每60 s；result4: Sheet1 + 药材表面列，t=60..t*_shrink 每60 s
5  # 数据区统一 number_format='0.0000'，与题面"所有结果保留四位小数（下同）"一致。
6  # 注：模板（result1-4.xlsx）与附件1/2均为题目提供材料，不随支撑材料打包；
7  #   复现时从 01_原始题目 复制到 code/user_data/ 再运行本脚本。
8  import os, sys, json
9  import numpy as np
10 from openpyxl import load_workbook
11
12 _HERE = os.path.dirname(os.path.abspath(__file__))
13 if _HERE not in sys.path:
14     sys.path.insert(0, _HERE)
15
16 OUTDIR = os.path.join(_HERE, '..', '..', '03_结果文件')
17 TMPL_DIR = os.path.join(_HERE, 'user_data')
18 R_CM = np.round(np.arange(0.0, 2.0 + 1e-9, 0.1), 4)
19
20
21 def _open_template(name):
22     """载入官方模板（保留全部格式），仅数据区会被覆盖。"""
23     p = os.path.join(TMPL_DIR, name)
24     if not os.path.exists(p):
25         raise FileNotFoundError('模板缺失：%s（请从 01_原始题目 复制到 user_data/）' % p)
26     return load_workbook(p)
27
28
29 def _fill_sheet(ws, t_arr, field, surf=None):
30     # 表头展开：A1 时间标注；B1..V1 = 0..2.0 cm（0.1 步）；result4 末尾再加 W1=药材表面
```

```
    ws.cell(row=1, column=1, value='时间\\到药材中心的距离')
    for j, rc in enumerate(R_CM):
        ws.cell(row=1, column=2 + j, value=round(float(rc), 4))
    if surf is not None:
        ws.cell(row=1, column=2 + len(R_CM), value='药材表面')
    for i, tt in enumerate(t_arr):
        ws.cell(row=2 + i, column=1, value=int(round(tt)))
        for j in range(len(R_CM)):
            v = field[i, j]
            if v is None or (isinstance(v, float) and np.isnan(v)):
                continue
            cell = ws.cell(row=2 + i, column=2 + j, value=round(float(v), 4))
            cell.number_format = '0.0000'
        if surf is not None:
            cell = ws.cell(row=2 + i, column=2 + len(R_CM), value=round(float(surf[i]), 4))
            cell.number_format = '0.0000'


def write_result1():
    d = json.load(open(os.path.join(_HERE, 'figures', 'problem_1_results.json'),
        encoding='utf-8'))
    T = np.array(d['T_field']); C = np.array(d['C_field'])
    t = np.arange(1, 1801)
    wb = _open_template('result1.xlsx')
    for name, field in (('温度', T), ('水分浓度', C)):
        _fill_sheet(wb[name], t, field)
    wb.save(os.path.join(OUTDIR, 'result1.xlsx'))
    print('[result1] %d行×21列 两表四位小数' % len(t))


def write_result2():
    # 直接使用 solve_q234.py 的 dt=1s 生产数据（problem_2_results.json，每隔1s真算）
    d = json.load(open(os.path.join(_HERE, 'figures', 'problem_2_results.json'),
        encoding='utf-8'))
    T = np.array(d['T_field']); C = np.array(d['C_field'])
    t = np.array(d['t_s'], dtype=int)
    wb = _open_template('result2.xlsx')
    for name, field in (('温度', T), ('水分浓度', C)):
        _fill_sheet(wb[name], t, field)
    wb.save(os.path.join(OUTDIR, 'result2.xlsx'))
    print('[result2] %d行×21列 两表四位小数 (dt=1s)' % len(t))


def _subsample60(t, *fields):
    """生产求解 dt=15s; xlsx 每 60s 一行，末行恒为烘干结束时刻 t*"""
    t = np.asarray(t, dtype=int)
    keep = (t % 60 == 0)
```

```
    keep[-1] = True # 末行 t* 恒保留
    idx = np.where(keep)[0]
    return t[idx], [np.asarray(f)[idx] for f in fields]


def write_result3():
    d = json.load(open(os.path.join(_HERE, 'figures', 'problem_3_results.json'),
        encoding='utf-8'))
    C = np.array(d['C_field'])
    t = np.array(d['t_s'])
    t, (C,) = _subsample60(t, C)
    wb = _open_template('result3.xlsx')
    _fill_sheet(wb['Sheet1'], t, C)
    wb.save(os.path.join(OUTDIR, 'result3.xlsx'))
    print('[result3] %d行×21列 每60s至t*=%.0fs' % (len(t), d['meta']['t_star_s']))


def write_result4():
    d = json.load(open(os.path.join(_HERE, 'figures', 'problem_4_results.json'),
        encoding='utf-8'))
    C = np.array(d['C_field'], dtype=float)
    t = np.array(d['t_s'])
    surf = np.array(d['C_surf'], dtype=float)
    t, (C, surf) = _subsample60(t, C, surf)
    wb = _open_template('result4.xlsx')
    _fill_sheet(wb['Sheet1'], t, C, surf=surf)
    wb.save(os.path.join(OUTDIR, 'result4.xlsx'))
    print('[result4] %d行×21列+表面列 每60s至t*_shrink=%.0fs' % (len(t), d['meta']['t_star_s']))


if __name__ == '__main__':
    os.makedirs(OUTDIR, exist_ok=True)
    write_result1()
    write_result2()
    write_result3()
    write_result4()
    print('[fill_results] 附件3 四个文件已按官方模板填充')
```

1.12 烘干时长灵敏度补充 hm_tstar_check.py

```
# -*- coding: utf-8 -*-
# 灵敏度补充: 传质系数 hm ±10% 对烘干时长 t* 的影响（论文灵敏度分析一节引用）
#   生产口径: n=1280、BDF rtol=1e-8、EnvTail='nominal'、audit 关闭（仅取事件时刻）
#   Q3: pset='a3' 固定域; Q4: pset='a4' 随体收缩域
import os, sys
```

```
_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)
os.chdir(_HERE)
import solve_q234 as s

BASE = s.HM # 8e-7 m/s

results = {}
for label, mult in [('hm-10%', 0.9), ('hm基准', 1.0), ('hm+10%', 1.1)]:
    s.HM = BASE * mult
    row = {}
    for q, kw in [('Q3', dict(pset='a3', FixR=True)),
                  ('Q4', dict(pset='a4', FixR=False))]:
        model = s.DryingCylinder(n=1280, EnvTail='nominal', **kw)
        summ, arr = model.run(rtol=1e-8, stopEvt=True,
                              audit=False, outDt=60.0)
        row[q] = round(summ['event_h'], 6)
    results[label] = row
    print('%s: Q3 t*=%.6f h Q4 t*=%.6f h' % (label, row['Q3'], row['Q4']), flush=True)

print(results)
```

1.13 附件数据校验与有限性扫描 data_check.py、sanity_check.py

```
# 数据读取验证：确认附件1结构、前31行区间、无缺失
import os, sys
_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)
import pandas as pd
import params as P

# 显式读全部 sheet，暴露多表情况（附件1单表，读全部再取 Sheet1）
_all = pd.read_excel(P.ANNEX1_PATH, sheet_name=None)
print("附件1 共 %d 个 sheet: %s" % (len(_all), ", ".join("%s(%d行)" % (n, len(d)) for n, d in
    _all.items())))

df = _all['Sheet1']
print("列名:", list(df.columns))
print("总行数:", len(df))
print("缺失值:\n", df.isnull().sum().to_dict())

sub = df.iloc[:P.ANNEX1_NROWS]
print("\n前31行(t=0..1800s):")
```

```
print(" 时间: %s -> %s, 间隔=%s" % (sub['时间'].iloc[0], sub['时间'].iloc[-1],
    sub['时间'].iloc[1]-sub['时间'].iloc[0]))
print(" 温度范围: %.4f -> %.4f" % (sub['温度'].min(), sub['温度'].max()))
print(" 水分浓度范围: %.5f -> %.5f" % (sub['水分浓度'].min(), sub['水分浓度'].max()))
assert sub['时间'].iloc[0] == 0 and sub['时间'].iloc[-1] == 1800, "时间区间不符"
assert not sub[['时间','温度','水分浓度']].isnull().any().any(), "前31行存在缺失"
print("\n[data_check] 附件1 前31行完整、区间正确 OK")
```

```
# 自动化数值 sanity check: 扫描 figures/*_results.json 的数值有限性/范围
import os, sys, json, math
_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)

errors, warnings = [], []


def walk(obj, path=""):
    if isinstance(obj, dict):
        for k, v in obj.items():
            walk(v, f"{path}.{k}")
    elif isinstance(obj, list):
        # 大数组只抽样检查有限性, 避免打印
        for i, v in enumerate(obj):
            if isinstance(v, (list, dict)):
                walk(v, f"{path}[{i}]")
            elif isinstance(v, float):
                if math.isnan(v):
                    errors.append(f"NaN at {path}[{i}]")
                elif math.isinf(v):
                    errors.append(f"Inf at {path}[{i}]")
    elif isinstance(obj, float):
        if math.isnan(obj):
            errors.append(f"NaN at {path}")
        elif math.isinf(obj):
            errors.append(f"Inf at {path}")


for f in ('figures/problem_1_results.json', 'figures/all_results.json',
    'figures/sensitivity_results.json'):
    if os.path.exists(f):
        walk(json.load(open(f, encoding='utf-8')), os.path.basename(f))

if errors:
    print(f"[sanity_check] {len(errors)} 个错误")
    for e in errors[:5]:
```

```
        print("  ", e)
    sys.exit(1)
else:
    print(f"[sanity_check] 所有数值有限、无 NaN/Inf ({len(warnings)} 警告) PASS")
```

1.14 物理对照量化 crossval/q1_variants.py

```
# -*- coding: utf-8 -*-
# Q1(1800s) 三种物理对照的量化（B级待办收尾，论文假设/限制一节引用）：
#  1) 收缩对照：附录2 物性 + 附件2 收缩(正确模型) vs 固定域 → 题目Q1固定参数口径的偏差量级
#  2) 辐射对照：有效对流系数 h=25 vs 25+6.5(  • 4T³, T 323K) → 辐射并入有效h的量级
#  3) 蒸发潜热量级：表面能量平衡加 L • _d • hm • (U-Uω) 冷却项 → 潜热省略的方向与量级
import os, sys, time
_HERE = os.path.dirname(os.path.abspath(__file__))
_CODE = os.path.dirname(_HERE)
if _CODE not in sys.path:
    sys.path.insert(0, _CODE)
import numpy as np
from solve_q234 import DryingCylinder, R0, T_INIT, C_INIT


def run1800(**kw):
    model = DryingCylinder(n=640, pset='a2', EnvTail='nominal', **kw)
    summ, arr = model.run(rtol=1e-8, horizon=1800.0,
                          stopEvt=False, audit=False, outDt=30.0)
    return model, arr


def row_at_1800(model, arr):
    i = int(np.argmin(np.abs(arr['times'] - 1800.0)))
    c = arr['c'][i]
    t = arr['temp'][i]
    R = model._radius(1800.0)
    out = []
    for r_cm in (0, .5, 1, 1.5, 2):
        rr = r_cm / 100.0
        if rr <= R:
            out.append((float(np.interp(rr / R, model.x, t)),
                        float(np.interp(rr / R, model.x, c))))
        else:
            out.append(None)
    return out, R


if __name__ == '__main__':
```

```
    os.chdir(_CODE)
    ref_T = [33.5753, 33.7721, 34.3642, 35.3622, 36.7856]
    ref_C = [2.55, 2.5497, 2.5382, 2.3753, 1.5103]

    print('[1] 固定域基线 (h=25, 附录2)', flush=True)
    m0, a0 = run1800(FixR=True)
    r0, _ = row_at_1800(m0, a0)
    for k, rc in enumerate((0, .5, 1, 1.5, 2)):
        print(' r=%4gcm T=%8.4f C=%8.4f' % (rc, r0[k][0], r0[k][1]), flush=True)

    print('[2] 收缩对照 (附件2 收缩, 1800s 半径=%.4f cm)' % 0, flush=True)
    m1, a1 = run1800(FixR=False)
    r1, R1 = row_at_1800(m1, a1)
    print(' R(1800s)=%.4f cm (收缩 %.1f%%)' % (R1 * 100, 100 * (1 - R1 / R0)), flush=True)
    for k, rc in enumerate((0, .5, 1, 1.5, 2)):
        if r1[k] is None:
            print(' r=%4g cm 已收缩至域外' % rc, flush=True)
        else:
            print(' r=%4g cm T=%8.4f (Δ%+7.4f) C=%8.4f (Δ%+7.4f)'
                  % (rc, r1[k][0], r1[k][0] - r0[k][0], r1[k][1], r1[k][1] - r0[k][1]),
                      flush=True)

    print('[3] 辐射对照 (h_eff=31.5 vs 25)', flush=True)
    m2, a2 = run1800(FixR=True, hConv=31.5)
    r2, _ = row_at_1800(m2, a2)
    for k, rc in enumerate((0, .5, 1, 1.5, 2)):
        print(' r=%4g cm T=%8.4f (Δ%+7.4f) C=%8.4f (Δ%+7.4f)'
              % (rc, r2[k][0], r2[k][0] - r0[k][0], r2[k][1], r2[k][1] - r0[k][1]), flush=True)

    print('[4] 蒸发潜热量级 (L=2.4MJ/kg 表面冷却)', flush=True)
    m3, a3 = run1800(FixR=True, LatHeat=True)
    r3, _ = row_at_1800(m3, a3)
    for k, rc in enumerate((0, .5, 1, 1.5, 2)):
        print(' r=%4g cm T=%8.4f (Δ%+7.4f) C=%8.4f (Δ%+7.4f)'
              % (rc, r3[k][0], r3[k][0] - r0[k][0], r3[k][1], r3[k][1] - r0[k][1]), flush=True)
    print('[done]', flush=True)
```

附录 B　问题一物理对照：收缩、辐射与蒸发潜热

模型假设 4 与假设 5 涉及三个题面未规定的物理环节，此处给出它们的量化对照。以下全部由问题二至四的同一求解引擎（节点型有限体积，$n = 640$、BDF 相对容差 10^{-8}、附录 2 物性、1800 s）复算，脚本见支撑材料 `crossval/q1_variants.py`。

（1）收缩对照（假设 4）。固定域基线与附件 2 收缩域（$R(1800\,\mathrm{s}) = 1.8730\,\mathrm{cm}$，收

缩 6.3%）对比如下。基线五位置温度为 33.5753/33.7720/34.3642/35.3621/36.7856 ℃，含水率为 2.5500/2.5497/2.5383/2.3755/1.5102 kg/kg。收缩使温度升高、含水率降低，且越靠外越明显：中心 $+0.34$ ℃、$r = 1.5$ cm 处 $+0.45$ ℃；含水率中心不变，$r = 1.0$ cm 处 -0.0105、$r = 1.5$ cm 处 -0.11 kg/kg；$r = 2$ cm 已收缩至域外（半径 1.87 cm）。这说明预热阶段收缩影响有限（表面温度仅约 $+0.45$ ℃，约 1%），但已足以改变四位小数表值，故问题一严格按题面固定半径口径输出，收缩作为问题四的机制单独处理。

（2）辐射对照（假设 5）。以 $\varepsilon\sigma\cdot 4T^3$（$T \approx 323$ K）估算辐射换热并入有效对流系数，h 由 25 W/(m^2·K) 上浮 26% 至 31.5 W/(m^2·K) 重算：表面温度由 36.7856 ℃ 升至 37.5649 ℃（$+0.78$ ℃），含水率四位小数不变（单向弱耦合下水分方程不含温度）。有效 h 上浮 26% 仅使表面温度移动 0.8 ℃，与假设 5 的披露一致。

（3）蒸发潜热量级（假设 5）。若在表面能量平衡中显式加入汽化潜热冷却项（$L = 2.4$ MJ/kg），1800 s 时表面温度将被拉低至约 4.2 ℃（较基线低约 32.5 ℃）、中心低约 28.7 ℃——与换热系数 $h = 25$ W/(m^2·K) 所支撑的传热量级严重矛盾。因此题给的 h 应视为包含辐射等全部表面换热途径的有效系数，水分通量视为以干基含水率为驱动力的有效通量，与假设 5 的结论一致。

附录 C　计算效率、线性代数路线与并行策略

问题一二维离散的记账细节补充在此：径向界面权重取 $f_j\Delta z/\Delta r$、轴向界面权重取 $V_i/\Delta z$、控制体体积取 $V_i\Delta z$；水分界面扩散系数取两侧节点调和平均 $D_{i+1/2} = 2D_iD_{i+1}/(D_i + D_{i+1})$。完整实现见支撑材料 problem1_axisym.py。

我们的计算代价集中在问题一的 (r, z) 验证链与问题二至四的长时程积分上，实测为：一维问题一主网格约 1 s；问题二（节点有限体积 $n = 640$、每秒一行的 dense output）约 14 s；问题三（$n = 1280$、BDF 相对容差 10^{-8}、57.5 h 全程）约 91 s；问题四（同配置、51.1 h 全程、含守恒审计）约 99 s；(r, z) 验证链共 6 次独立二维求解（4 个 β 情景各约 1.7 min，$(200, 100)$ 收敛网格约 12 min），串行合计约 19 min。降维后生产模型仅含 $n = 1280$ 个节点，每步解块三对角线性系统，BDF 内嵌 Newton 迭代（带稀疏雅可比模式）处理非线性耦合，dense output 按需输出——空间与时间离散均已收敛（第 10 节），且收敛链、模型空间变体与三套独立实现的交叉验证互不相关、可任务级并行。

二维 (r, z) 五点格式的线性代数路线也曾权衡：按自然编号它是带宽 N_z 的带型矩阵而非经典三对角，可选带型直接解、块 Thomas 与 ADI 分块 Thomas 三条路线。我们用带型直接解（追赶法的带宽推广）：$O(N_{\text{tot}}N_z^2) \approx 2.5 \times 10^7$ 次运算、M 矩阵免主元、实测单次 17 ms，已是该结构最优；块 Thomas 同阶但常数更大、无优势；ADI 单步 $O(N_{\text{tot}})$、理论快 50–100 倍，但（a）分裂误差需 Douglas/Peaceman–Rachford 变体维持精度，且破坏 $D(C)$ 指数非线性下的离散极值原理；（b）零增益——表2/3取中截面，其剖面恒等于

一维解（$\beta = 0$ 回归机器精度 2.25×10^{-12}，$\beta = 1$ 时水分场的轴向影响按 $\mathrm{erfc}(L/2\ell)$ 在 1800 s 仅约 10^{-194}），任何一致格式都改变不了表值一位数字；（c）规模不值得——带型解 17 ms×3 次 Picard×1800 步 ≈92 s，ADI 只加速验证链、不改变任何提交答案。故保留带型直接解。

在此规模下，常见并行手段均不采用。时间并行（Parareal 类）受抛物型方程时间推进的串行因果链限制，且本题 $D(C)$ 指数强非线性、干燥后期刚性，粗传播子精度不足，校正迭代的开销无法抵消并行收益。空间区域分解适用于 10^5–10^6 以上未知数、直接解因内存与填充不可行的大系统；本题二维验证模型仅 5×10^3 个未知数（收敛网格 2×10^4 个），算子为带宽 101 的带状矩阵，LAPACK 带状直接解已是精确且最优的选择，引入 halo 通信与 Schwarz 迭代后通信计算比过高，反而更慢。ADI 类方法见前文二维五点格式对比，不采用。

真正有效且零风险的并行在任务级：网格收敛（$n = 200/400/800$）、β 情景（0/0.5/1/2）、环境尾三口径、模型空间变体与灵敏度扰动是互不相关的独立正演。我们用 multiprocessing 进程池并行这批独立求解（见 problem1_axisym.py 与 convergence.py），(r, z) 验证链墙钟由约 19 min 缩短至约 12 min，结果与串行逐位一致。单次正演已无需再加速：BDF 按误差控制自适应选步（全程仅数千步），Gauss 面系数为向量化表达式，当前计算量在秒至分钟量级。